

# 01. Clustering Algorithms: Comparative Analysis and Implementation

## Algorithms Covered:

1. **K-Means Clustering** (Standard Lloyd's Algorithm)
2. **Modified K-Means** (Bisecting K-Means & K-Means++ Initialization)
3. **Hierarchical Clustering** (Agglomerative Clustering with Dendrogram Visualization)
4. **Fuzzy C-Means (FCM)** (Soft Clustering with Membership Degrees)

**Dataset:** Real-World Mall Customers Dataset ( `data/mall_customers.csv` ) and Synthetic Benchmark Multi-Cluster Data.

```
In [1]: # ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, BisectingKMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (Linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")
```

Libraries successfully imported!

## 1. Data Loading and Exploratory Data Analysis (EDA)

We load the **Mall Customers** dataset, which consists of customer demographics, annual income (in \$k), and spending score (1-100).

```
In [2]: # ---- Load the Mall Customers dataset (path is relative to the notebooks/ folder) ----
df_mall = pd.read_csv('../data/mall_customers.csv')

# Inspect the shape (rows, columns) and preview the first five rows
print("Dataset Shape:", df_mall.shape)
df_mall.head()
```

Dataset Shape: (200, 5)

Out[2]:

|  | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|--|------------|--------|-----|---------------------|------------------------|
|--|------------|--------|-----|---------------------|------------------------|

|   |   |        |    |    |    |
|---|---|--------|----|----|----|
| 0 | 1 | Male   | 19 | 15 | 39 |
| 1 | 2 | Male   | 21 | 15 | 81 |
| 2 | 3 | Female | 20 | 16 | 6  |
| 3 | 4 | Female | 23 | 16 | 77 |
| 4 | 5 | Female | 31 | 17 | 40 |

```
In [3]: # ---- Data quality check: count missing values per column ----
print("Missing values per column:\n", df_mall.isnull().sum())

# ---- Descriptive statistics (mean, std, min, quartiles, max) for numeric columns ----
df_mall.describe()
```

Missing values per column:

```
CustomerID      0
Gender          0
Age             0
Annual Income (k$)  0
Spending Score (1-100)  0
dtype: int64
```

Out[3]:

|  | CustomerID | Age | Annual Income (k\$) | Spending Score (1-100) |
|--|------------|-----|---------------------|------------------------|
|--|------------|-----|---------------------|------------------------|

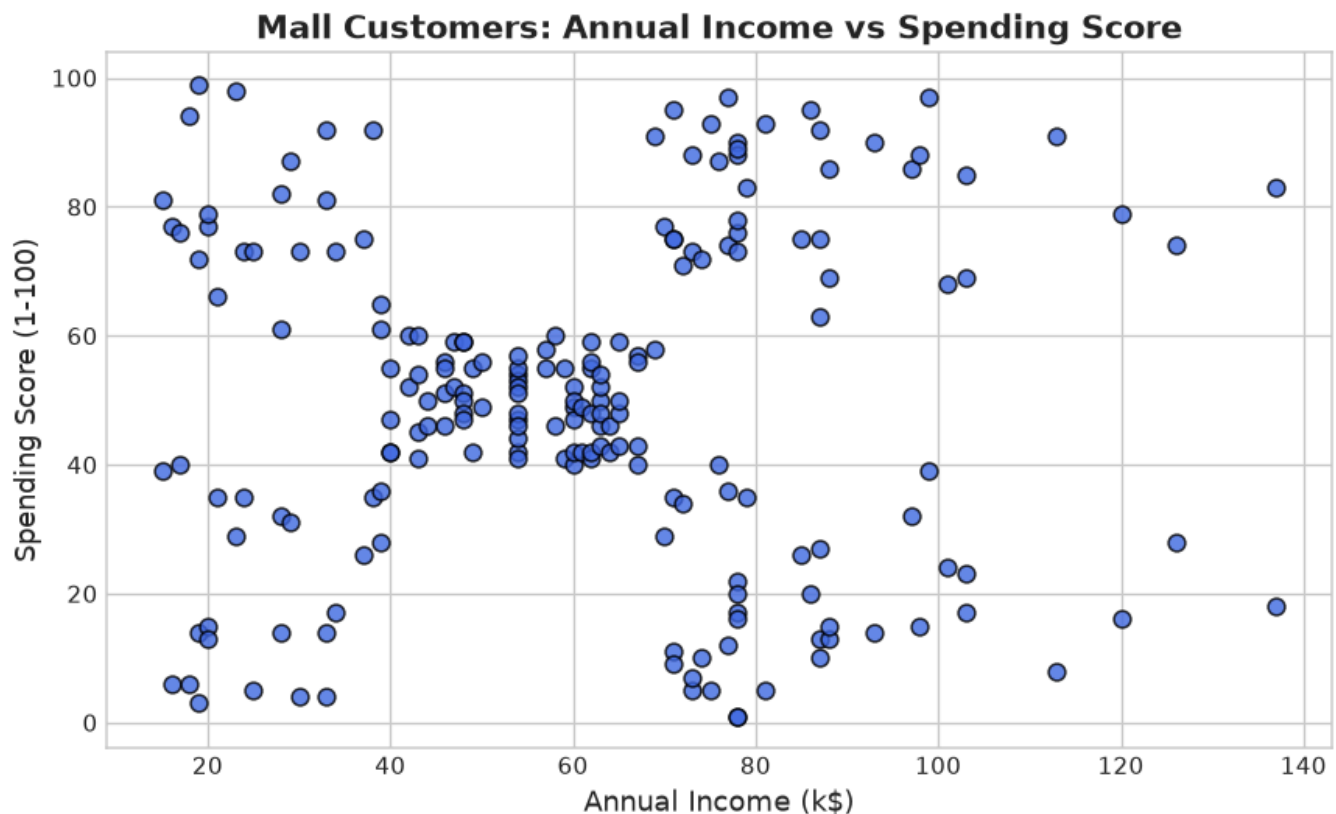
|       |            |            |            |            |
|-------|------------|------------|------------|------------|
| count | 200.000000 | 200.000000 | 200.000000 | 200.000000 |
| mean  | 100.500000 | 38.850000  | 60.560000  | 50.200000  |
| std   | 57.879185  | 13.969007  | 26.264721  | 25.823522  |
| min   | 1.000000   | 18.000000  | 15.000000  | 1.000000   |
| 25%   | 50.750000  | 28.750000  | 41.500000  | 34.750000  |
| 50%   | 100.500000 | 36.000000  | 61.500000  | 50.000000  |
| 75%   | 150.250000 | 49.000000  | 78.000000  | 73.000000  |
| max   | 200.000000 | 70.000000  | 137.000000 | 99.000000  |

```
In [4]: # ---- Feature selection and normalization ----
# We cluster on two informative dimensions so the result can be visualized in 2D:
# Annual Income and Spending Score.
feature_cols = ['Annual_Income_k$', 'Spending_Score'] if 'Annual_Income_k$' in df_mall.columns
else ['Annual Income (k$)', 'Spending Score (1-100)']
X_mall = df_mall[feature_cols].values

# Standardize features (mean = 0, std = 1) so both dimensions contribute equally
# to Euclidean distance - distance-based algorithms are scale-sensitive.
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)

# Scatter plot of the raw (unscaled) data to inspect its structure before clustering
plt.figure(figsize=(8, 5))
plt.scatter(X_mall[:, 0], X_mall[:, 1], c='royalblue', edgecolors='k', s=50, alpha=0.8)
plt.title("Mall Customers: Annual Income vs Spending Score", fontsize=14, fontweight='bold')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
```

```
plt.tight_layout()
plt.show()
```



## 2. Algorithm 1: Standard K-Means Clustering

### Mathematical Formulation

Given dataset  $X = \{x_1, x_2, \dots, x_n\}$ , K-Means partitions the data into  $K$  disjoint clusters  $C_1, C_2, \dots, C_K$  by minimizing the Within-Cluster Sum of Squares (WCSS / Inertia):

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where  $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$  is the centroid of cluster  $C_k$ .

### Determining Optimal $K$ : Elbow Method & Silhouette Analysis

#### Step-by-Step Algorithm (Lloyd's K-Means)

1. **Choose  $K$**  and initialize  $K$  centroids (here with K-Means++ for a strong start).
2. **Assignment step:** assign every sample to the nearest centroid (Euclidean distance).
3. **Update step:** move each centroid to the mean of the samples assigned to it.
4. **Repeat** steps 2-3 until assignments/centroids stop changing or the iteration limit is reached.
5. **Output:** final centroids, hard cluster labels, and the inertia (WCSS) used for model selection.

### How $K$ Is Selected in This Notebook

1. Fit K-Means for  $K = 2 \dots 10$ .
2. Record inertia (elbow curve) and silhouette score for each  $K$ .
3. Pick the  $K$  at the "elbow" / highest silhouette ( $K = 5$  here) and refit the final model.

```

In [5]: # ---- Selecting the optimal number of clusters K ----
# Try K = 2..10 and record two diagnostics:
# * Inertia (WCSS): total squared distance from points to their centroid
# * Silhouette score: cluster cohesion vs. separation
k_range = range(2, 11)
wcss = []
silhouette_scores = []

for k in k_range:
    # n_init=10 runs K-Means from 10 different seeds and keeps the best run
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    kmeans.fit(X_mall_scaled)
    wcss.append(kmeans.inertia_) # WCSS of the fitted model
    silhouette_scores.append(silhouette_score(X_mall_scaled, kmeans.labels_)) # partition quality

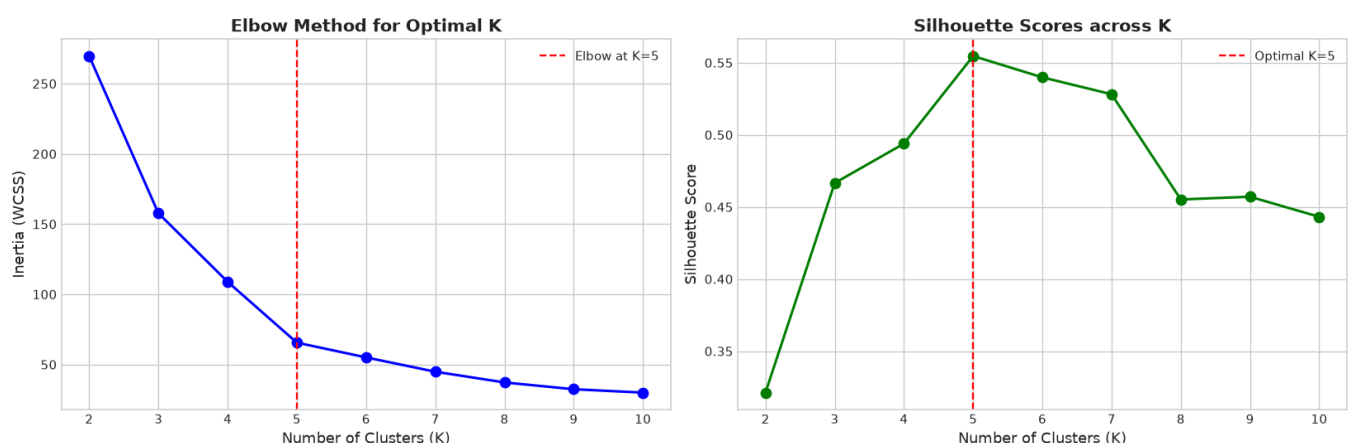
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# ---- Left: elbow curve (look for the "knee" where inertia stops falling quickly) ----
ax1.plot(k_range, wcss, 'bo-', linewidth=2, markersize=8)
ax1.set_title("Elbow Method for Optimal K", fontsize=14, fontweight='bold')
ax1.set_xlabel("Number of Clusters (K)", fontsize=12)
ax1.set_ylabel("Inertia (WCSS)", fontsize=12)
ax1.axvline(x=5, color='r', linestyle='--', label='Elbow at K=5')
ax1.legend()

# ---- Right: silhouette score for each K (higher is better) ----
ax2.plot(k_range, silhouette_scores, 'go-', linewidth=2, markersize=8)
ax2.set_title("Silhouette Scores across K", fontsize=14, fontweight='bold')
ax2.set_xlabel("Number of Clusters (K)", fontsize=12)
ax2.set_ylabel("Silhouette Score", fontsize=12)
ax2.axvline(x=5, color='r', linestyle='--', label='Optimal K=5')
ax2.legend()

plt.tight_layout()
plt.show()

```



```

In [6]: # ---- Final K-Means model with the selected K = 5 ----
optimal_k = 5
kmeans_model = KMeans(n_clusters=optimal_k, init='k-means++', n_init=20, random_state=42)
kmeans_labels = kmeans_model.fit_predict(X_mall_scaled) # hard cluster assignment per sample
kmeans_centers = scaler.inverse_transform(kmeans_model.cluster_centers_) # convert centroids back to original units

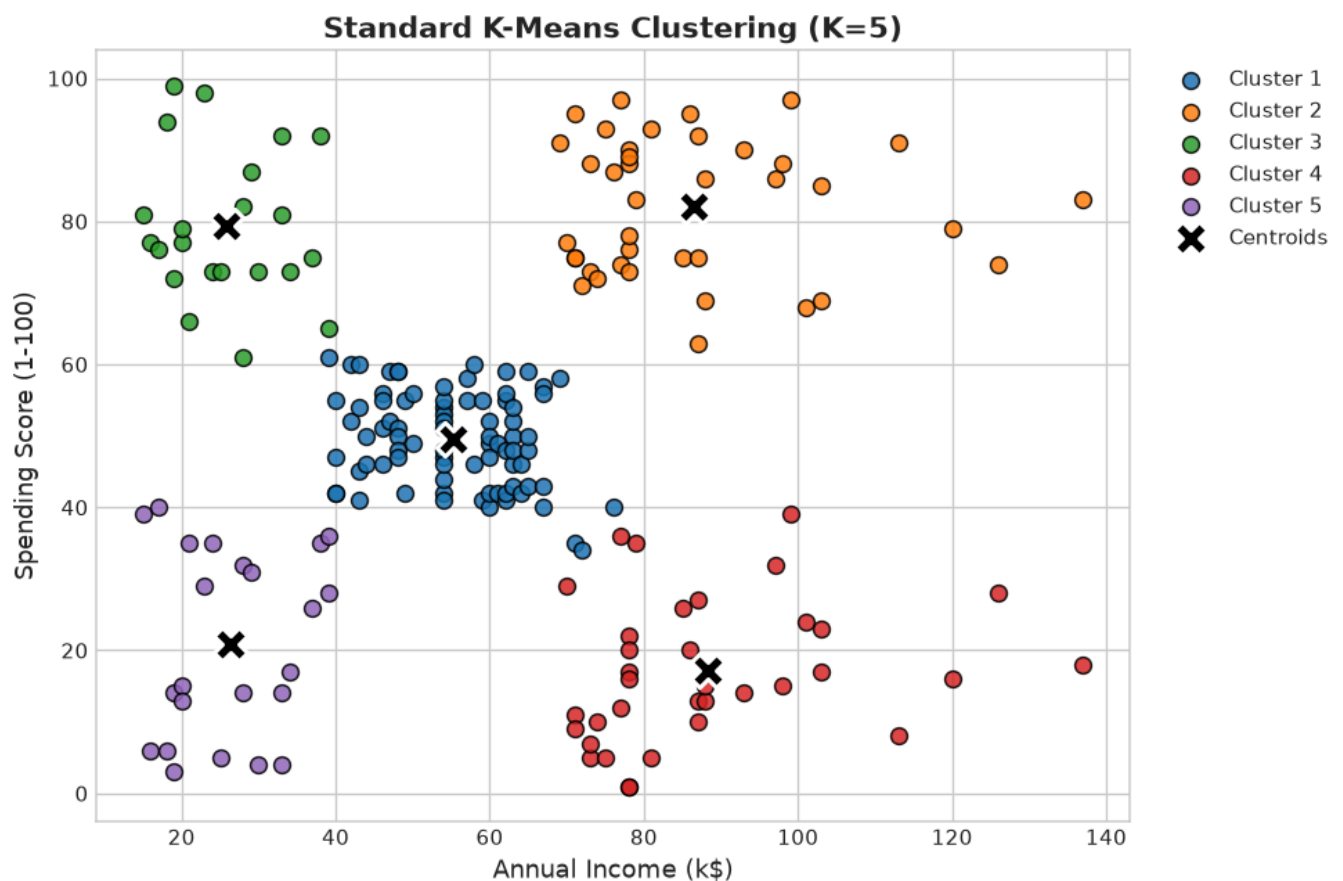
# ---- Visualize the clusters and centroids in the original feature space ----

```

```
plt.figure(figsize=(9, 6))
palette = sns.color_palette("deep", optimal_k)
for cluster_id in range(optimal_k):
    cluster_points = X_mall[kmeans_labels == cluster_id] # points belonging to this cluster
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1], label=f'Cluster {cluster_id + 1}',
s=60, edgecolors='k', alpha=0.85)

plt.scatter(kmeans_centers[:, 0], kmeans_centers[:, 1], c='black', marker='X', s=250,
label='Centroids', edgecolors='white', linewidths=2)
plt.title("Standard K-Means Clustering (K=5)", fontsize=14, fontweight='bold')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

# ---- Quantitative evaluation (computed on the standardized features) ----
print(f"K-Means Silhouette Score: {silhouette_score(X_mall_scaled, kmeans_labels):.4f}")
print(f"K-Means Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled,
kmeans_labels):.4f}")
```



K-Means Silhouette Score: 0.5547  
K-Means Davies-Bouldin Index: 0.5722

### 3. Algorithm 2: Modified K-Means

Modified variants of K-Means address known limitations of the standard algorithm:

1. **K-Means++ Initialization:** Selects initial cluster centers with probability proportional to the squared Euclidean distance to the nearest existing center, drastically reducing convergence time and sensitivity to local minima.
2. **Bisecting K-Means:** A hierarchical divisive variant that repeatedly splits clusters using 2-means until  $K$  clusters are obtained. It produces more uniform and consistent cluster sizes.

# Step-by-Step Algorithm (Modified K-Means Variants)

## A. K-Means++ initialization

1. Pick the first centroid uniformly at random from the data.
2. For every point compute  $D(x)^2$  = squared distance to the nearest chosen centroid.
3. Pick the next centroid randomly with probability proportional to  $D(x)^2$ .
4. Repeat steps 2-3 until K centroids are chosen, then run standard K-Means. *Benefit:* avoids poor random starts, converges faster and more consistently.

## B. Bisecting K-Means (divisive)

1. Start with all samples in a single cluster.
2. Select the cluster with the largest inertia (or largest size).
3. Split it into two using 2-means (try multiple restarts, keep the split with the lowest SSE).
4. Repeat steps 2-3 until exactly K clusters are obtained. *Benefit:* more balanced, stable cluster sizes and less sensitivity to initialization.

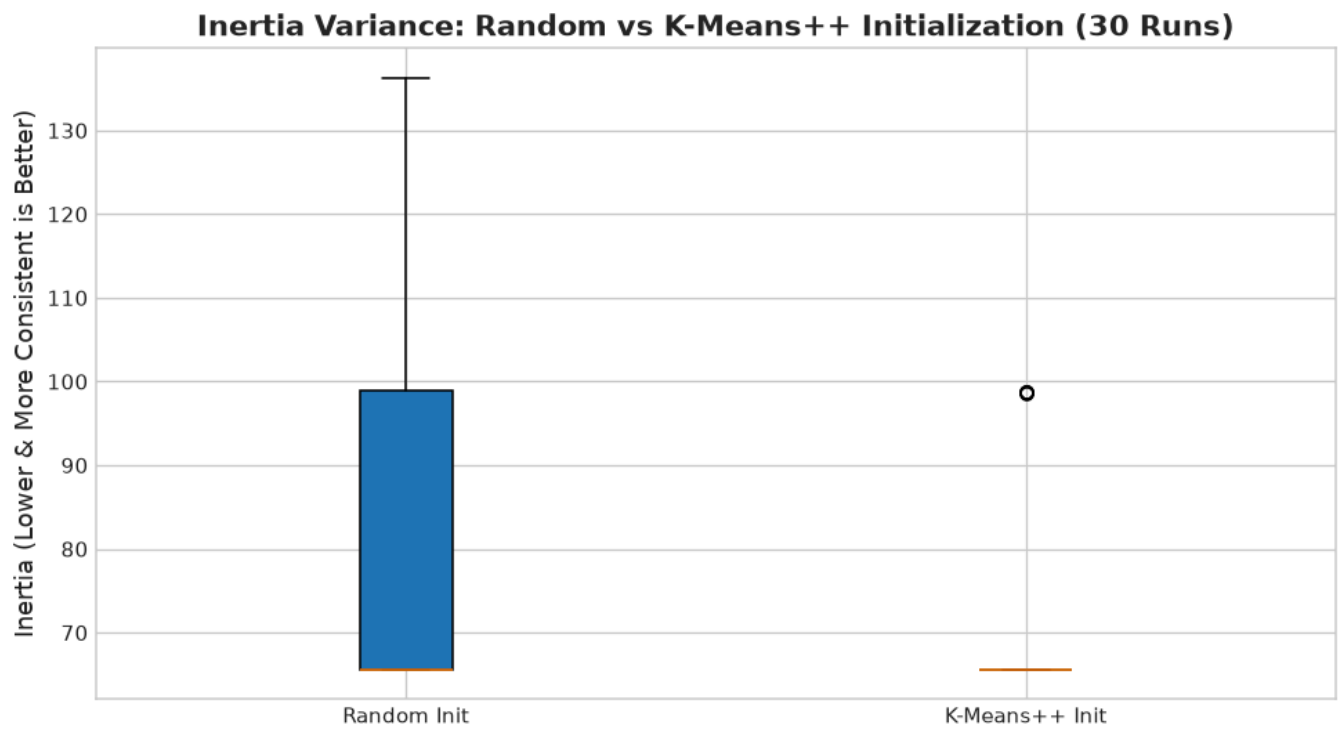
```
In [7]: # ---- Experiment: K-Means++ vs random initialization stability ----
# Run both initializations 30 times (one seed each) and compare the resulting inertia.
# A stable initialization should produce consistently low inertia across seeds.
n_trials = 30
inertia_random = []
inertia_kpp = []

for seed in range(n_trials):
    # (a) Plain random centroid initialization
    km_rnd = KMeans(n_clusters=5, init='random', n_init=1, random_state=seed)
    km_rnd.fit(X_mall_scaled)
    inertia_random.append(km_rnd.inertia_)

    # (b) K-Means++ smart initialization (spreads initial centroids apart)
    km_kpp = KMeans(n_clusters=5, init='k-means++', n_init=1, random_state=seed)
    km_kpp.fit(X_mall_scaled)
    inertia_kpp.append(km_kpp.inertia_)

# Boxplot compares the spread (variance) of the two strategies
plt.figure(figsize=(9, 5))
plt.boxplot([inertia_random, inertia_kpp], tick_labels=['Random Init', 'K-Means++ Init'],
            patch_artist=True)
plt.title("Inertia Variance: Random vs K-Means++ Initialization (30 Runs)", fontsize=14,
          fontweight='bold')
plt.ylabel("Inertia (Lower & More Consistent is Better)", fontsize=12)
plt.tight_layout()
plt.show()

# Mean and standard deviation summarize stability (lower std = more reliable)
print(f"Random Init Mean Inertia: {np.mean(inertia_random):.2f} (std: {np.std(inertia_random):.2f})")
print(f"K-Means++ Mean Inertia: {np.mean(inertia_kpp):.2f} (std: {np.std(inertia_kpp):.2f})")
```



Random Init Mean Inertia: 83.44 (std: 22.08)

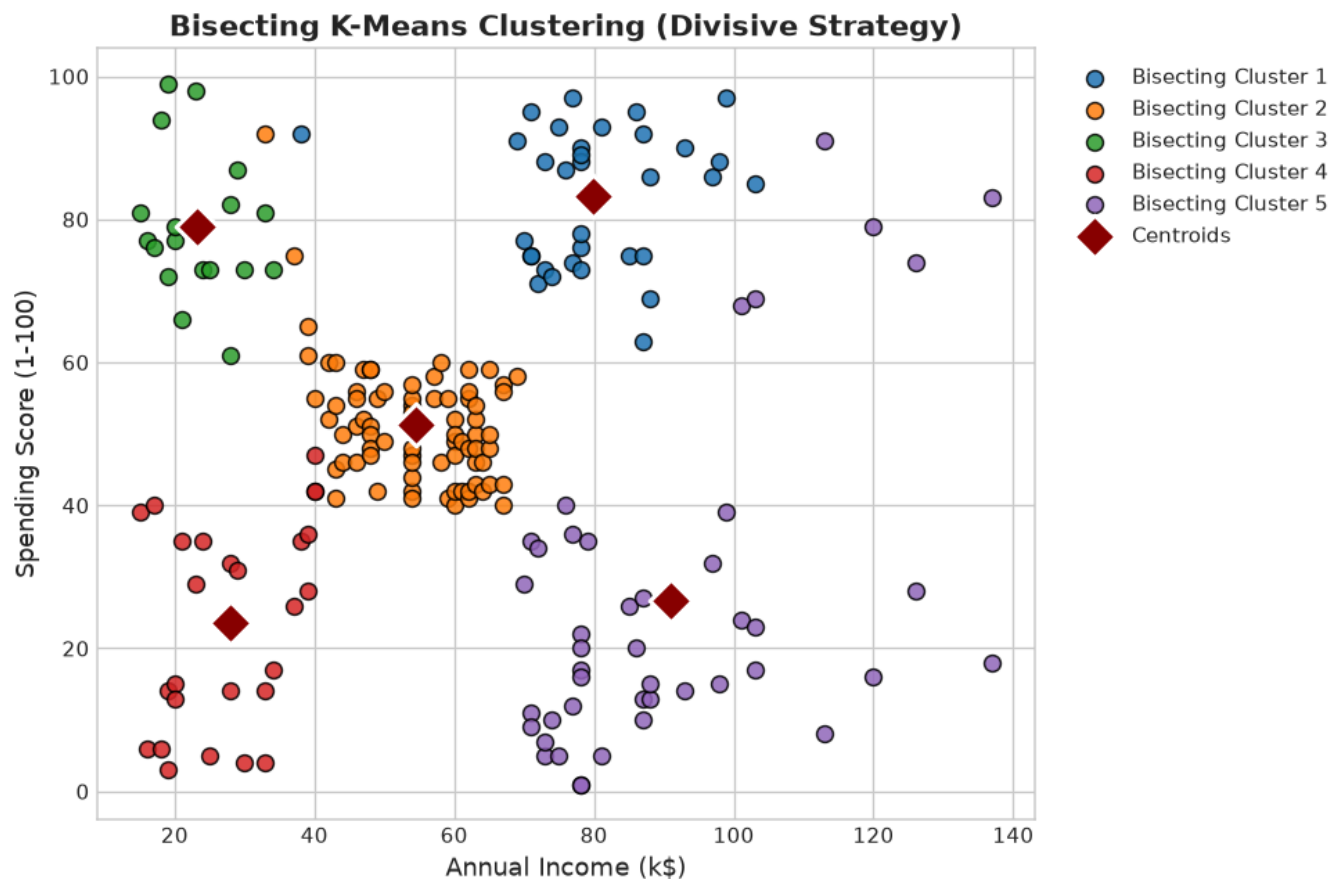
K-Means++ Mean Inertia: 69.99 (std: 11.26)

```
In [8]: # ---- Bisecting K-Means (divisive variant of K-Means) ----
# Repeatedly splits the cluster with the biggest inertia using 2-means until K clusters remain.
bisecting_km = BisectingKMeans(n_clusters=optimal_k, random_state=42,
bisecting_strategy='biggest_inertia')
bisecting_labels = bisecting_km.fit_predict(X_mall_scaled)
bisecting_centers = scaler.inverse_transform(bisecting_km.cluster_centers_)

# ---- Visualize the partition produced by the divisive strategy ----
plt.figure(figsize=(9, 6))
for cluster_id in range(optimal_k):
    cluster_points = X_mall[bisecting_labels == cluster_id]
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1], label=f'Bisecting Cluster
{cluster_id + 1}', s=60, edgecolors='k', alpha=0.85)

plt.scatter(bisecting_centers[:, 0], bisecting_centers[:, 1], c='darkred', marker='D', s=200,
label='Centroids', edgecolors='white', linewidths=2)
plt.title("Bisecting K-Means Clustering (Divisive Strategy)", fontsize=14, fontweight='bold')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

# ---- Evaluation of the bisecting partition ----
print(f"Bisecting K-Means Silhouette Score: {silhouette_score(X_mall_scaled,
bisecting_labels):.4f}")
print(f"Bisecting K-Means Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled,
bisecting_labels):.4f}")
```



Bisecting K-Means Silhouette Score: 0.4807

Bisecting K-Means Davies-Bouldin Index: 0.6788

## 4. Algorithm 3: Hierarchical Clustering (Agglomerative)

### Mathematical Formulation

Agglomerative clustering begins with each point as its own cluster and iteratively merges the closest pair of clusters until only one cluster remains. Using **Ward's Linkage**, the distance between cluster  $A$  and cluster  $B$  measures the increase in total within-cluster variance upon merging:

$$\Delta W(A, B) = \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2$$

### Step-by-Step Algorithm (Agglomerative / Ward)

1. Start with  $N$  singleton clusters (one per sample).
2. Compute pairwise distances between all clusters using Ward's criterion:  $\Delta W(A, B) = (n_A n_B) / (n_A + n_B) \cdot \|\mu_A - \mu_B\|^2$ .
3. Merge the two clusters with the smallest  $\Delta W$  (least increase in within-cluster variance).
4. Update distances between the new cluster and all remaining clusters.
5. Repeat steps 3-4 until a single cluster remains.
6. Cut the dendrogram at the chosen height (or choose  $K$ ) to obtain the final partition.

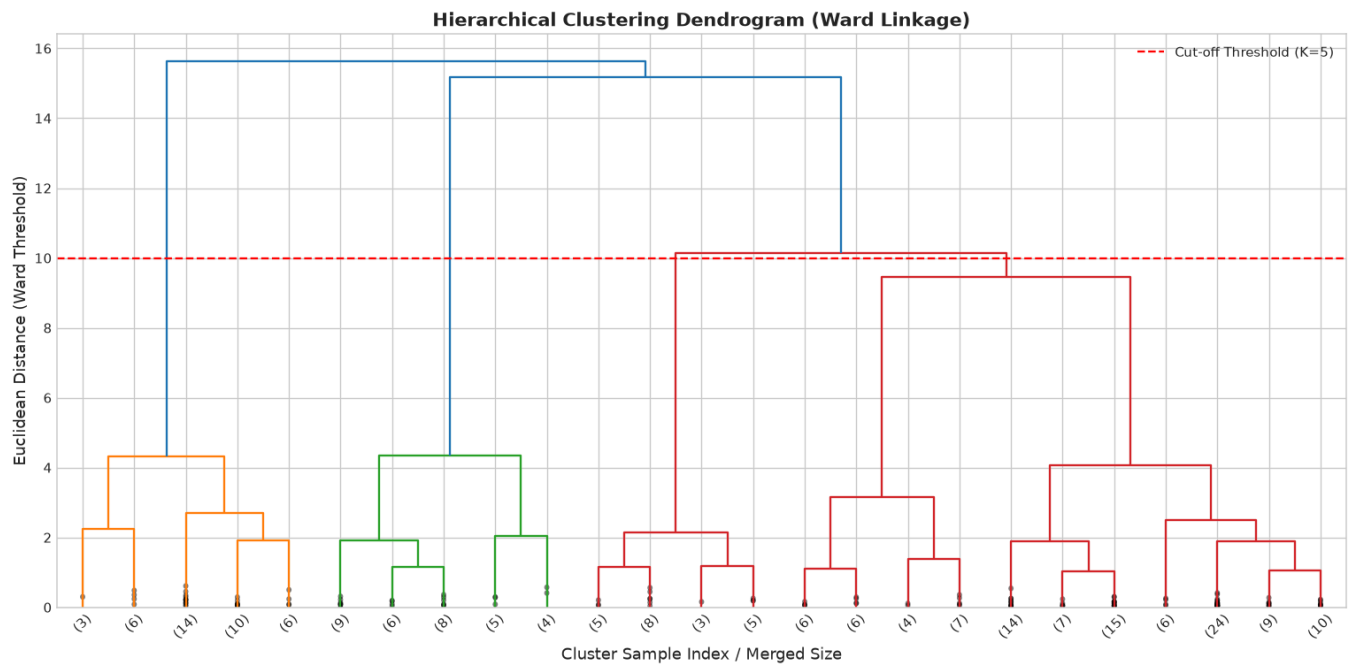
```
In [9]: # ---- Build the linkage matrix and draw the dendrogram ----
# 'ward' linkage merges the pair of clusters that increases within-cluster variance the least.
linkage_matrix = linkage(X_scaled, method='ward')

# The dendrogram shows every merge; we truncate it to the last 25 merges for readability.
plt.figure(figsize=(14, 7))
dendrogram(
```

```

linkage_matrix,
truncate_mode='lastp', # show only the last p merged clusters
p=25,
leaf_rotation=45,
leaf_font_size=10,
show_contracted=True # condensed leaves are shown as counts
)
plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)", fontsize=14, fontweight='bold')
plt.xlabel("Cluster Sample Index / Merged Size", fontsize=12)
plt.ylabel("Euclidean Distance (Ward Threshold)", fontsize=12)
plt.axhline(y=10.0, color='r', linestyle='--', label='Cut-off Threshold (K=5)') # horizontal
cut => cluster count
plt.legend()
plt.tight_layout()
plt.show()

```



```

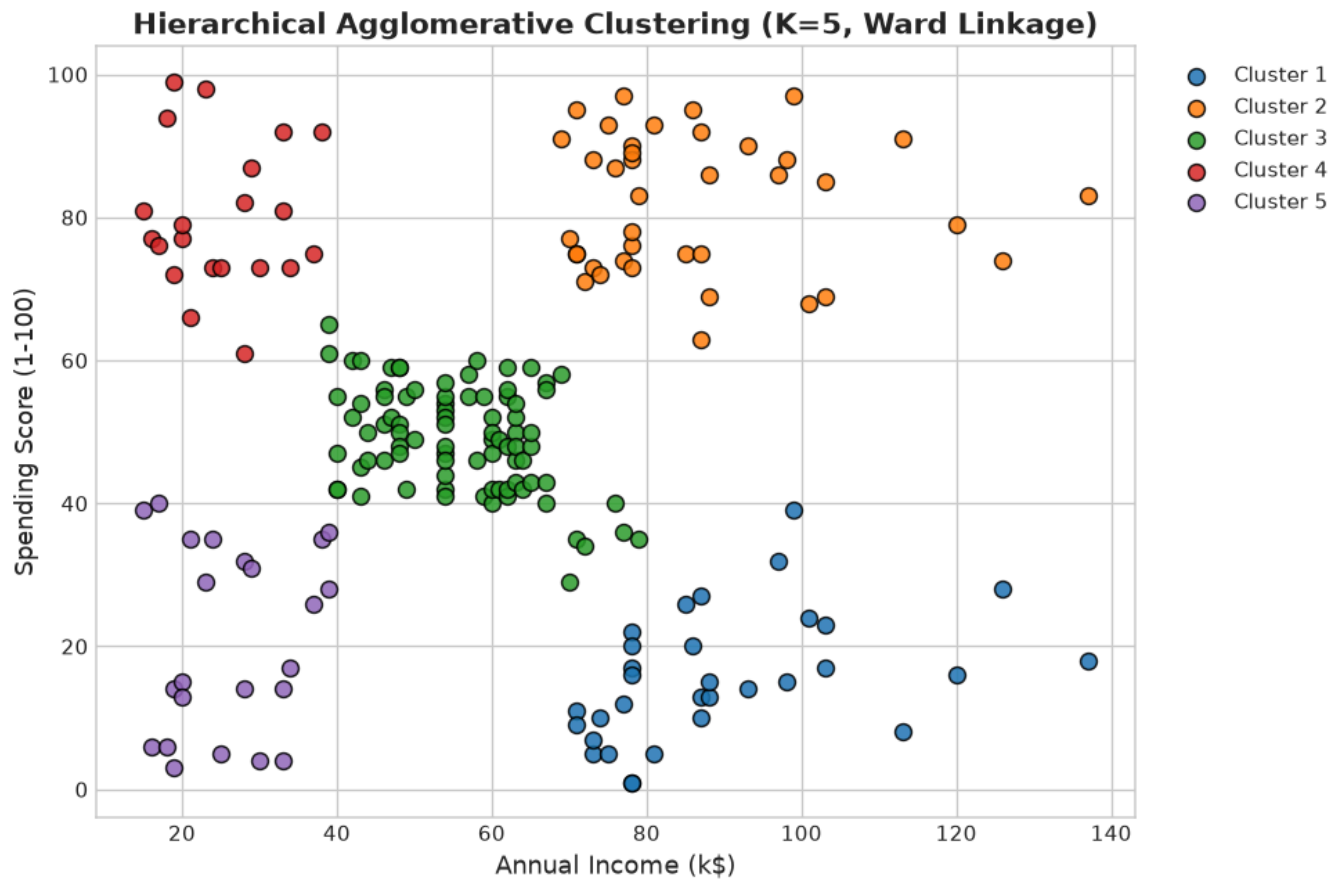
In [10]: # ---- Fit the final Agglomerative (bottom-up hierarchical) model with K = 5 ----
# metric='euclidean' + linkage='ward' implements Ward's minimum-variance criterion.
agg_model = AgglomerativeClustering(n_clusters=optimal_k, metric='euclidean', linkage='ward')
agg_labels = agg_model.fit_predict(X_mall_scaled)

# ---- Visualize the hierarchical partition ----
plt.figure(figsize=(9, 6))
for cluster_id in range(optimal_k):
    cluster_points = X_mall[agg_labels == cluster_id]
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1], label=f'Cluster {cluster_id + 1}',
s=60, edgecolors='k', alpha=0.85)

plt.title("Hierarchical Agglomerative Clustering (K=5, Ward Linkage)", fontsize=14,
fontweight='bold')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

# ---- Evaluation of the hierarchical partition ----
print(f"Hierarchical Silhouette Score: {silhouette_score(X_mall_scaled, agg_labels):.4f}")
print(f"Hierarchical Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled,
agg_labels):.4f}")

```



Hierarchical Silhouette Score: 0.5538  
 Hierarchical Davies-Bouldin Index: 0.5779

## 5. Algorithm 4: Fuzzy C-Means (FCM)

### Mathematical Formulation

Unlike hard clustering (K-Means), Fuzzy C-Means assigns each sample  $x_i$  a **fuzzy membership degree**  $u_{ik} \in [0, 1]$  across all  $K$  clusters such that  $\sum_{k=1}^K u_{ik} = 1$ . The objective function is:

$$J_m = \sum_{i=1}^N \sum_{k=1}^K u_{ik}^m \|x_i - v_k\|^2, \quad m > 1$$

Where:

- $m$  is the fuzzifier (typically  $m = 2.0$ ).
- Centroid update:  $v_k = \frac{\sum_{i=1}^N u_{ik}^m x_i}{\sum_{i=1}^N u_{ik}^m}$
- Membership update:  $u_{ik} = \frac{1}{\sum_{j=1}^K \left( \frac{\|x_i - v_k\|}{\|x_i - v_j\|} \right)^{\frac{2}{m-1}}}$

### Step-by-Step Algorithm (Fuzzy C-Means)

1. Choose  $K$ , fuzzifier  $m > 1$  and tolerance  $\epsilon$ ; initialize the membership matrix  $U$  randomly so each row sums to 1.
2. **Centroid update:**  $v_k = \sum_i u_{ik}^m x_i / \sum_i u_{ik}^m$ .
3. **Membership update:**  $u_{ik} = 1 / \sum_j ( \|x_i - v_k\| / \|x_i - v_j\| )^{2/(m-1)}$ .
4. **Repeat** steps 2-3 until  $\max |U_{\text{new}} - U_{\text{old}}| < \epsilon$  or the iteration limit is reached.

5. **Output:** fuzzy memberships  $u_{ik}$  (soft assignment) and centroids; hard labels come from  $\text{argmax}_k u_{ik}$ .

```
In [11]: class FuzzyCMeans:
    """
    Fuzzy C-Means (FCM) clustering - implemented from scratch with NumPy.

    Each sample belongs to every cluster with a membership degree  $u_{ik}$  in  $[0, 1]$ ,
    and the memberships over the  $K$  clusters sum to 1 per sample.

    Parameters
    -----
    n_clusters    : number of fuzzy clusters  $K$ 
    m             : fuzzifier exponent ( $m > 1$ ; larger  $m \Rightarrow$  fuzzier memberships)
    max_iter      : maximum number of alternating-optimization iterations
    tol           : convergence tolerance on the maximum membership change
    random_state  : seed for reproducible random initialization
    """

    def __init__(self, n_clusters=5, m=2.0, max_iter=150, tol=1e-5, random_state=42):
        self.n_clusters = n_clusters
        self.m = m
        self.max_iter = max_iter
        self.tol = tol
        self.random_state = random_state

    def fit(self, X):
        rng = np.random.RandomState(self.random_state)
        n_samples = X.shape[0]

        # Step 1: initialize the membership matrix  $U$  randomly and normalize every row to sum to
        1
        U = rng.rand(n_samples, self.n_clusters)
        U = U / U.sum(axis=1, keepdims=True)

        # Step 2: alternate between centroid and membership updates until convergence
        for iteration in range(self.max_iter):
            U_old = U.copy()

            # 2a: centroid update - membership-weighted mean of the data
            Um = U ** self.m                                #  $u_{ik}^m$ 
            centers = np.dot(Um.T, X) / Um.sum(axis=0)[:, np.newaxis]

            # 2b: compute the ( $n_{\text{samples}} \times n_{\text{clusters}}$ ) distance matrix to every centroid
            dist = np.linalg.norm(X[:, np.newaxis, :] - centers[np.newaxis, :, :], axis=2)
            dist = np.fmax(dist, 1e-10) # avoid division by zero

            # 2c: membership update - closer points receive higher membership
            #  $u_{ik} = 1 / \sum_j (d_{ik} / d_{ij})^{2/(m-1)}$ 
            inv_dist = 1.0 / dist
            power = 2.0 / (self.m - 1.0)
            inv_dist_p = inv_dist ** power
            U = inv_dist_p / inv_dist_p.sum(axis=1, keepdims=True)

            # 2d: stop when memberships stop changing (convergence)
            if np.max(np.abs(U - U_old)) < self.tol:
                break

        # Store results: final centroids, full membership matrix, and hard labels
        self.cluster_centers_ = centers
        self.u_ = U
```

```

        self.labels_ = np.argmax(U, axis=1) # hard assignment = cluster with maximum
membership
        return self

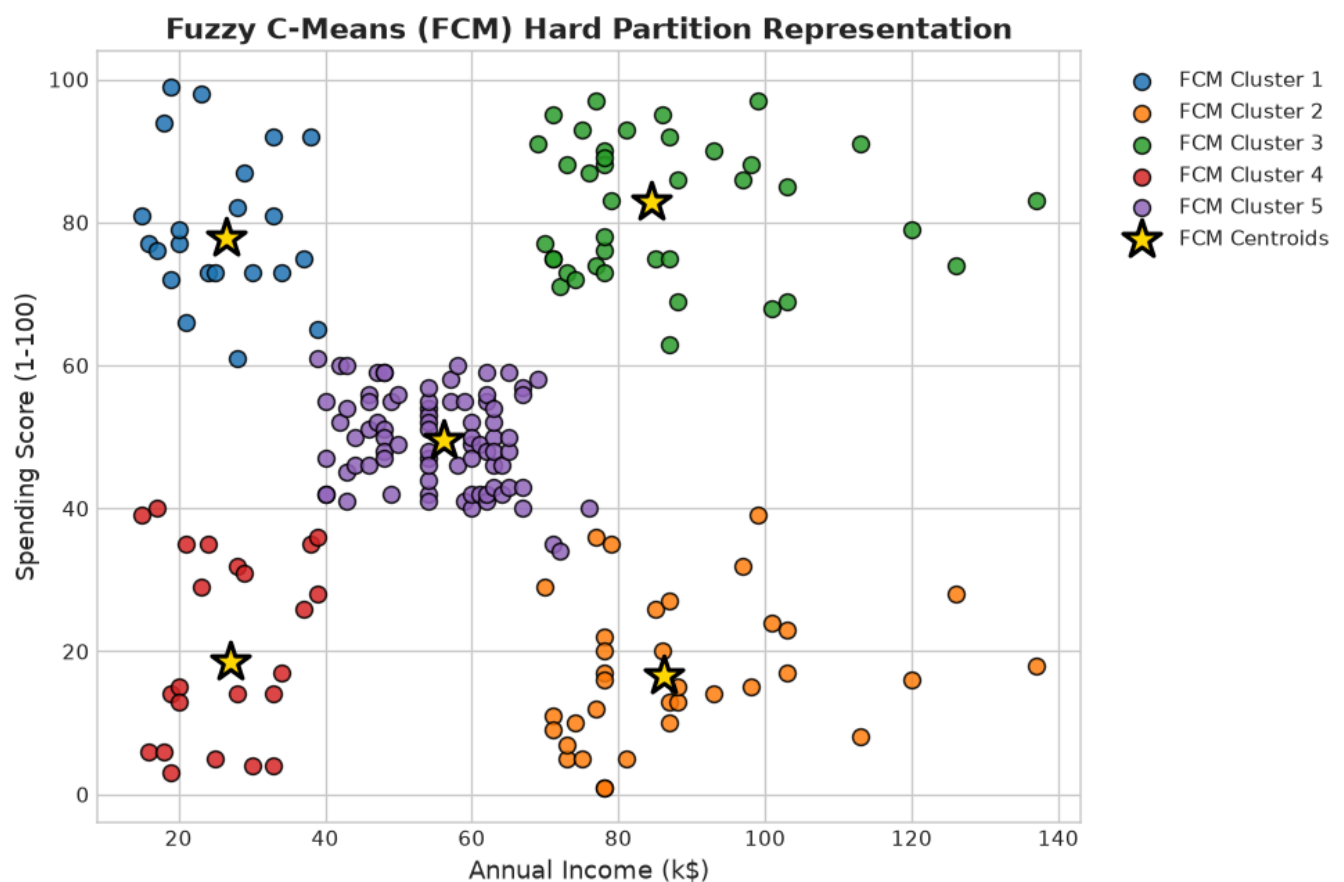
# Fit FCM with the same K as the other algorithms (m = 2 is the standard fuzzifier)
fcm = FuzzyCMeans(n_clusters=optimal_k, m=2.0, random_state=42)
fcm.fit(X_mall_scaled)
fcm_labels = fcm.labels_
fcm_centers = scaler.inverse_transform(fcm.cluster_centers_)

# ---- Visualize the hard partition derived from the soft memberships ----
plt.figure(figsize=(9, 6))
for cluster_id in range(optimal_k):
    cluster_points = X_mall[fcm_labels == cluster_id]
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1], label=f'FCM Cluster {cluster_id +
1}', s=60, edgecolors='k', alpha=0.85)

plt.scatter(fcm_centers[:, 0], fcm_centers[:, 1], c='gold', marker='*', s=350, label='FCM
Centroids', edgecolors='k', linewidths=2)
plt.title("Fuzzy C-Means (FCM) Hard Partition Representation", fontsize=14, fontweight='bold')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

# ---- Evaluation of the hard partition induced by the fuzzy memberships ----
print(f"FCM Silhouette Score: {silhouette_score(X_mall_scaled, fcm_labels):.4f}")
print(f"FCM Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled, fcm_labels):.4f}")

```



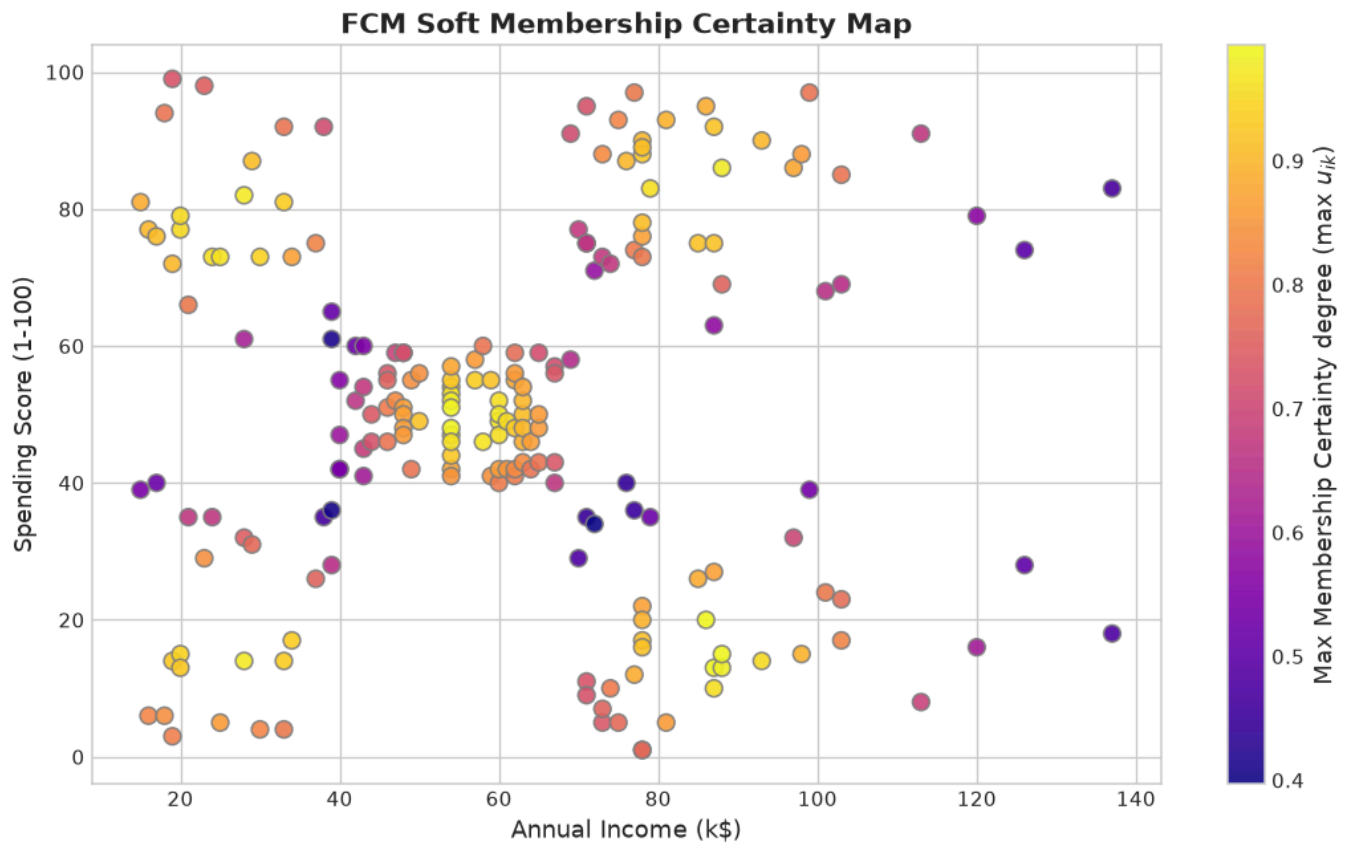
FCM Silhouette Score: 0.5547

FCM Davies-Bouldin Index: 0.5722

In [12]: # ---- Visualizing the soft membership degrees (the key FCM output) ----  
# For every customer take the maximum membership across clusters:

```
# * close to 1.0 -> the point clearly belongs to one cluster
# * near 1/K      -> the point is ambiguous (sits between clusters)
max_memberships = np.max(fcm.u_, axis=1)

plt.figure(figsize=(10, 6))
scatter = plt.scatter(X_mall[:, 0], X_mall[:, 1], c=max_memberships, cmap='plasma', s=70,
                     edgecolors='gray', alpha=0.9)
cbar = plt.colorbar(scatter)
cbar.set_label('Max Membership Certainty degree (max  $u_{ik}$ )', fontsize=12)
plt.title("FCM Soft Membership Certainty Map", fontsize=14, fontweight='bold')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
plt.tight_layout()
plt.show()
```



## 6. Comprehensive Quantitative Comparison

We evaluate all four clustering algorithms across three primary unsupervised metrics:

1. **Silhouette Score** (Higher is better: ranges from -1 to 1; measures intra-cluster cohesion vs. inter-cluster separation)
2. **Davies-Bouldin Index** (Lower is better: measures average similarity between clusters)
3. **Calinski-Harabasz Score** (Higher is better: ratio of between-clusters dispersion to within-cluster dispersion)

### Evaluation Process

1. Collect the final labels produced by each algorithm.
2. Compute Silhouette, Davies-Bouldin and Calinski-Harabasz on the standardized features.
3. Compare the results in a table (best values highlighted) and bar charts, then interpret differences.

```

In [13]: # ---- Collect the labels produced by the four algorithms ----
models = {
    "Standard K-Means": kmeans_labels,
    "Bisecting K-Means": bisecting_labels,
    "Hierarchical (Ward)": agg_labels,
    "Fuzzy C-Means": fcm_labels
}

# ---- Compute the three unsupervised quality metrics for each partition ----
results = []
for name, labels in models.items():
    results.append({
        "Algorithm": name,
        "Silhouette Score": silhouette_score(X_mall_scaled, labels),           # higher is
better
        "Davies-Bouldin Index": davies_bouldin_score(X_mall_scaled, labels),   # lower is
better
        "Calinski-Harabasz Score": calinski_harabasz_score(X_mall_scaled, labels) # higher is
better
    })

df_comparison = pd.DataFrame(results).set_index("Algorithm")
print("Clustering Performance Comparison on Mall Customers:")
# Highlight the best value of each metric in green for quick comparison
display(df_comparison.style.highlight_max(subset=['Silhouette Score', 'Calinski-Harabasz
Score'], color='lightgreen')
        .highlight_min(subset=['Davies-Bouldin Index'], color='lightgreen'))

```

Clustering Performance Comparison on Mall Customers:

|                     | Silhouette Score | Davies-Bouldin Index | Calinski-Harabasz Score |
|---------------------|------------------|----------------------|-------------------------|
| Algorithm           |                  |                      |                         |
| Standard K-Means    | 0.554657         | 0.572236             | 248.649320              |
| Bisecting K-Means   | 0.480684         | 0.678789             | 150.938872              |
| Hierarchical (Ward) | 0.553809         | 0.577862             | 244.410326              |
| Fuzzy C-Means       | 0.554657         | 0.572236             | 248.649320              |

```

In [14]: # ---- Bar-chart summary of the comparison table ----
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Panel 1: Silhouette Score (higher is better)
df_comparison['Silhouette Score'].plot(kind='bar', ax=axes[0], color='teal', edgecolor='k')
axes[0].set_title("Silhouette Score (Higher is Better)", fontweight='bold')
axes[0].set_ylabel("Score")
axes[0].tick_params(axis='x', rotation=30)

# Panel 2: Davies-Bouldin Index (Lower is better)
df_comparison['Davies-Bouldin Index'].plot(kind='bar', ax=axes[1], color='coral',
edgecolor='k')
axes[1].set_title("Davies-Bouldin Index (Lower is Better)", fontweight='bold')
axes[1].set_ylabel("Score")
axes[1].tick_params(axis='x', rotation=30)

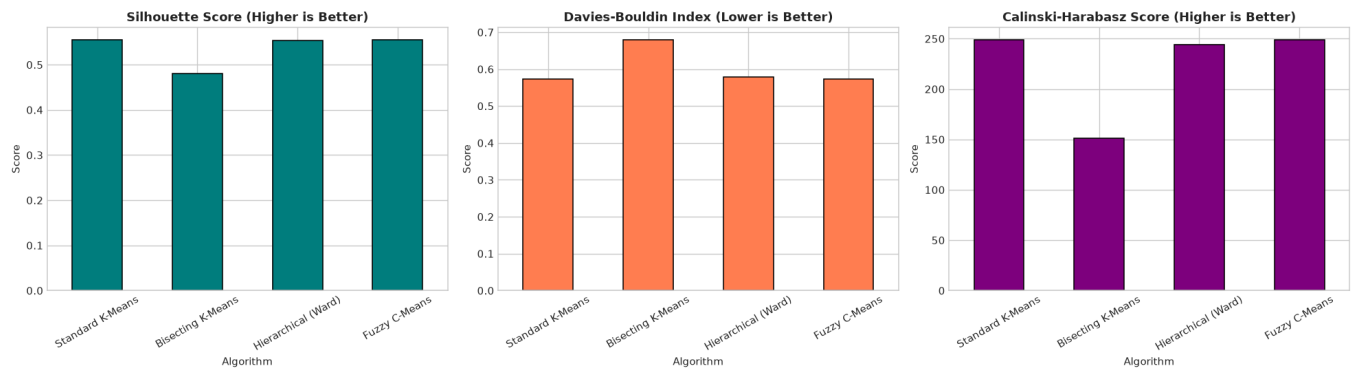
# Panel 3: Calinski-Harabasz Score (higher is better)
df_comparison['Calinski-Harabasz Score'].plot(kind='bar', ax=axes[2], color='purple',
edgecolor='k')
axes[2].set_title("Calinski-Harabasz Score (Higher is Better)", fontweight='bold')
axes[2].set_ylabel("Score")

```

```
axes[2].tick_params(axis='x', rotation=30)
```

```
plt.tight_layout()
```

```
plt.show()
```



## 7. Conclusions & Algorithm Selection Insights

- **Standard K-Means** provides an excellent balance of speed and cluster separation on globular, well-separated data.
- **Bisecting K-Means** prevents small outlier clusters from dominating the solution and yields stable, balanced partitions.
- **Hierarchical Clustering** requires no prior assumption about  $K$  when constructing the tree, and the dendrogram offers visual interpretability into cluster hierarchy.
- **Fuzzy C-Means** captures overlapping clusters and data ambiguity, providing actionable membership probabilities for points along boundary regions.

# 02. Density-Based Learning: DBSCAN and HDBSCAN

## Algorithms Covered:

1. **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise)
2. **HDBSCAN** (Hierarchical Density-Based Spatial Clustering of Applications with Noise)

## Core Focus:

- Discovering clusters of arbitrary shapes and variable densities
- Automatic anomaly and noise identification
- Systematic hyperparameter selection ( $k$ -distance graph for  $\epsilon$  and `min_samples`)
- Comparative benchmark against centroid-based clustering (K-Means)

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Density-based clustering algorithms + centroid baseline ----
from sklearn.cluster import DBSCAN, HDBSCAN, KMeans
# ---- Synthetic dataset generators for non-convex shapes ----
from sklearn.datasets import make_moons, make_blobs, make_circles
# ---- Nearest-neighbor search (used to pick epsilon) and scaling ----
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
# ---- Evaluation metrics ----
from sklearn.metrics import silhouette_score, adjusted_rand_score

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Density-based libraries loaded successfully!")
```

Density-based libraries loaded successfully!

## 1. Generating Complex Non-Spherical Benchmark Data

Real-world spatial, geographic, and sensor data often exhibits arbitrary shapes, varying local densities, and background noise where centroid-based algorithms (like K-Means) fail completely. Here we construct a composite benchmark dataset featuring:

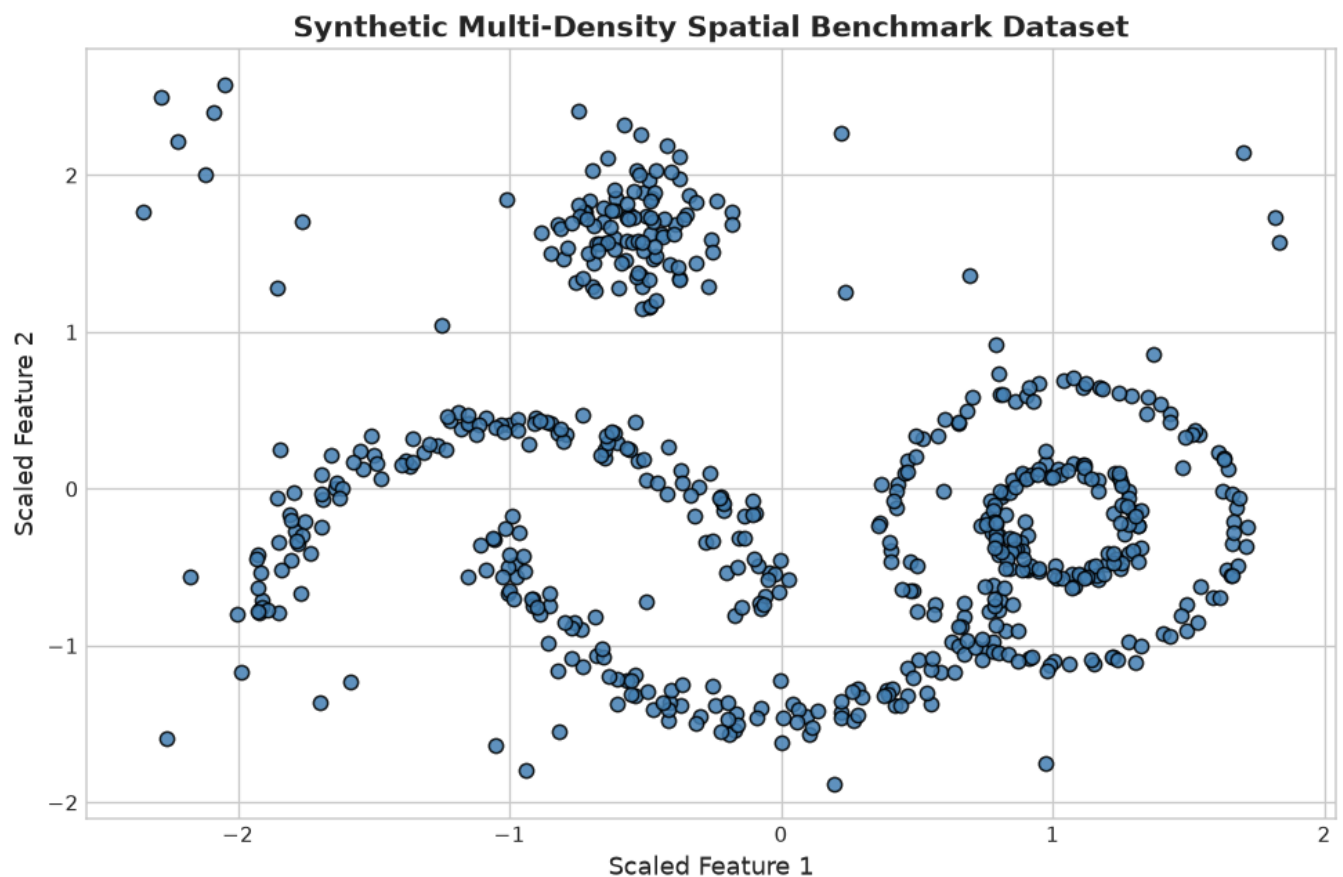
- Two interleaving half moons
- A dense circular core
- A sparser Gaussian blob
- Random uniform background noise (outliers)

```
In [2]: # ---- Build a composite benchmark dataset with complex geometry ----
# Two interleaving half-moons (non-convex)
n_samples = 600
X_moons, y_moons = make_moons(n_samples=250, noise=0.06, random_state=42)
# Two concentric circles, shifted to the right so they do not overlap the moons
X_circle, y_circle = make_circles(n_samples=200, noise=0.04, factor=0.4, random_state=42)
X_circle = X_circle * 0.7 + np.array([2.2, 0.5])
# One sparse Gaussian blob above the moons
X_blob, y_blob = make_blobs(n_samples=100, centers=[[0.5, 2.0]], cluster_std=0.2,
random_state=42)
# Uniform background noise (outliers) spanning the whole area
X_noise = np.random.uniform(low=[-1.5, -1.0], high=[3.2, 2.8], size=(50, 2))

# Stack all point groups into one feature matrix
X_raw = np.vstack([X_moons, X_circle, X_blob, X_noise])
# Ground-truth Labels for benchmarking (noise points are marked with -1)
y_true = np.concatenate([y_moons, y_circle + 2, y_blob + 4, -np.ones(50, dtype=int)])

# Standardize so that epsilon (a distance threshold) has a comparable meaning in both axes
scaler = StandardScaler()
X = scaler.fit_transform(X_raw)

# ---- Plot the raw benchmark to show the shapes/noise the algorithms must handle ----
plt.figure(figsize=(9, 6))
plt.scatter(X[:, 0], X[:, 1], c='steelblue', edgecolors='k', s=45, alpha=0.85)
plt.title("Synthetic Multi-Density Spatial Benchmark Dataset", fontsize=14, fontweight='bold')
plt.xlabel("Scaled Feature 1", fontsize=12)
plt.ylabel("Scaled Feature 2", fontsize=12)
plt.tight_layout()
plt.show()
```



## 2. Why Centroid-Based Clustering (K-Means) Fails on

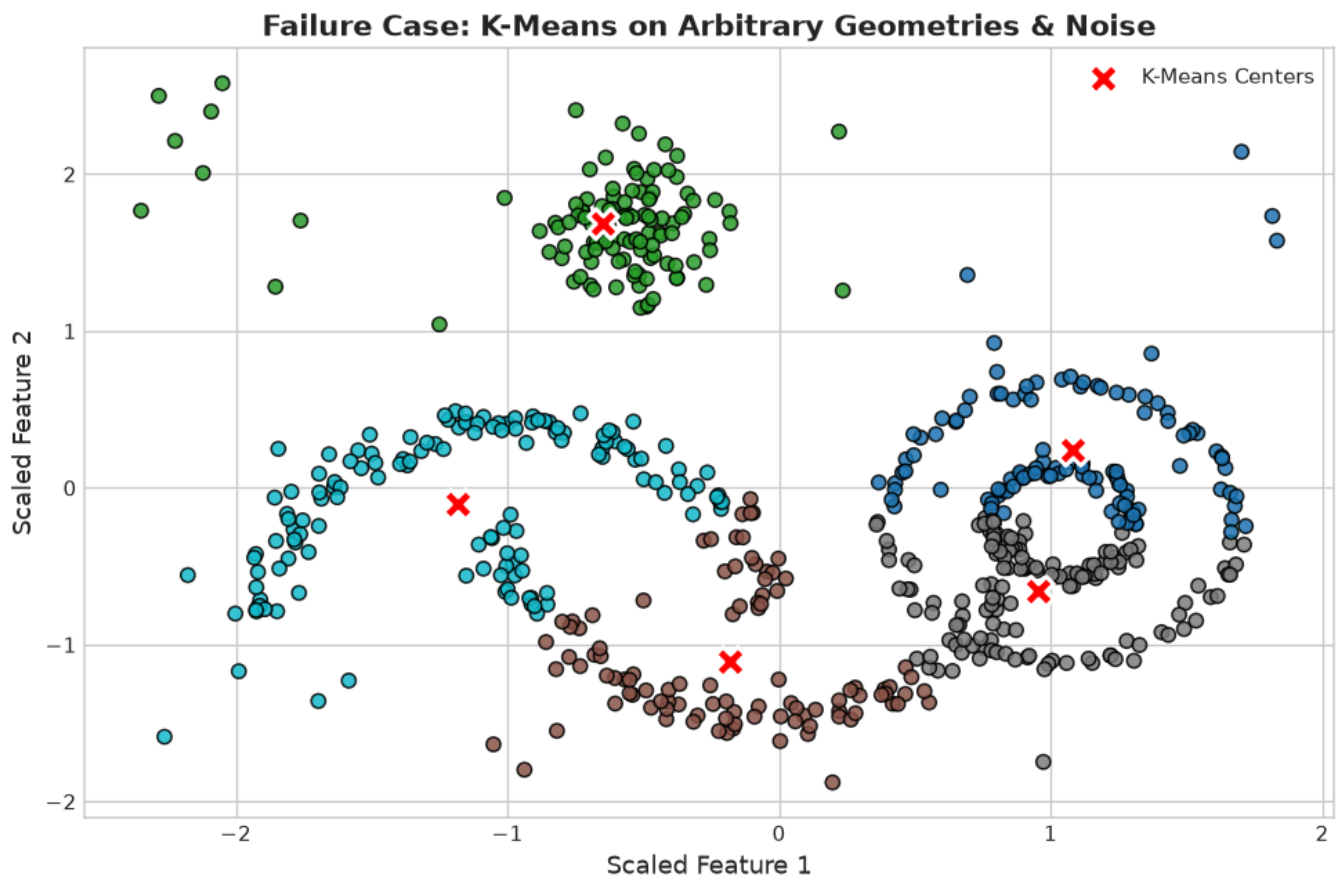
# Non-Convex Geometries

K-Means assumes clusters are spherical and isotropic, minimizing Euclidean variance around centroids. Let's see what happens when K-Means is applied to this non-convex dataset.

```
In [3]: # ---- Baseline: apply K-Means (centroid-based) to the non-convex data ----
# K-Means assumes spherical, equally sized clusters, so it cannot follow curved shapes.
km = KMeans(n_clusters=5, random_state=42, n_init=10)
km_labels = km.fit_predict(X)

# Visualize where K-Means places its straight-line Voronoi boundaries
plt.figure(figsize=(9, 6))
plt.scatter(X[:, 0], X[:, 1], c=km_labels, cmap='tab10', edgecolors='k', s=45, alpha=0.85)
plt.scatter(km.cluster_centers_[0], km.cluster_centers_[1], c='red', marker='X', s=200,
            edgecolors='white', linewidths=2, label='K-Means Centers')
plt.title("Failure Case: K-Means on Arbitrary Geometries & Noise", fontsize=14,
          fontweight='bold')
plt.xlabel("Scaled Feature 1", fontsize=12)
plt.ylabel("Scaled Feature 2", fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()

# Adjusted Rand Index measures agreement with the ground truth (1.0 = perfect, 0 = chance)
print(f"K-Means Adjusted Rand Index (ARI vs ground truth): {adjusted_rand_score(y_true,
km_labels):.4f}")
```



K-Means Adjusted Rand Index (ARI vs ground truth): 0.4426

# 3. Algorithm 1: DBSCAN

## Mathematical Concepts

DBSCAN defines clusters as continuous regions of high density separated by regions of low density. For a dataset  $D$ :

1.  **$\epsilon$ -Neighborhood:**

$$N_\epsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \epsilon\}$$

2. **Core Point:** A point  $p$  is a core point if  $|N_\epsilon(p)| \geq \text{MinPts}$ .

3. **Directly Density-Reachable:** Point  $q$  is directly density-reachable from  $p$  if  $q \in N_\epsilon(p)$  and  $p$  is a core point.

4. **Density-Reachable & Density-Connected:** Points connected through chains of core points form a cluster.

5. **Noise / Outliers:** Any point not reachable from any core point is assigned label  $-1$ .

## Hyperparameter Selection via the $k$ -Distance Graph

A standard heuristic for choosing  $\epsilon$  is plotting the distance to the  $k$ -th nearest neighbor sorted in ascending order (where  $k = \text{MinPts}$ ). The "knee" or "elbow" of maximum curvature indicates the threshold where point density sharply drops into noise.

## Step-by-Step Algorithm (DBSCAN)

1. **Parameters:** radius  $\epsilon$  and minimum points MinPts (here 0.22 and 6).
2. For every point count its  $\epsilon$ -neighborhood; points with  $\geq \text{MinPts}$  neighbors become **core points**.
3. **Grow clusters:** connect core points that lie within  $\epsilon$  of each other (density-reachability chain).
4. Attach border points (within  $\epsilon$  of a core point) to that core's cluster.
5. Label every point that is neither core nor border as **noise (-1)**.
6. **Output:** arbitrary-shaped clusters + explicit noise labels. No  $K$  is required.

## How $\epsilon$ and MinPts Are Chosen Here

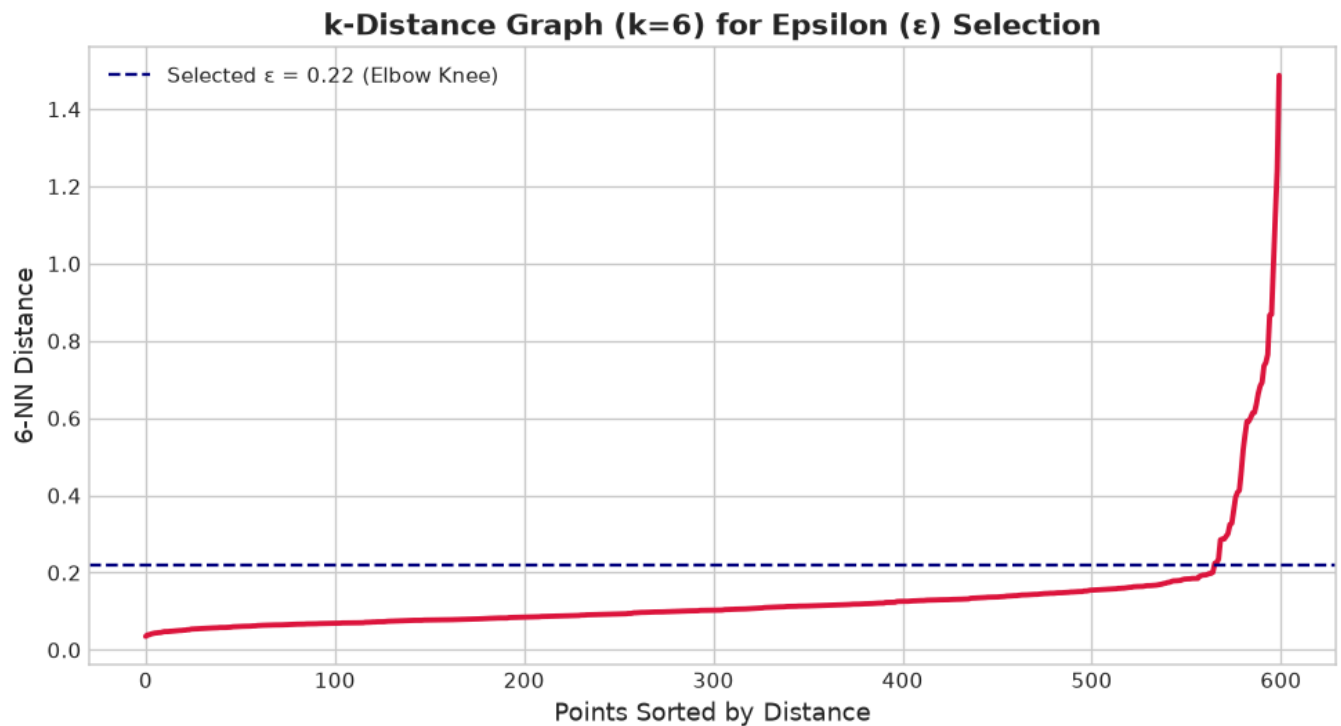
1. Set MinPts  $\approx 2 \times$  dimensions.
2. Plot the sorted distance to the MinPts-th neighbor and pick the "knee" as  $\epsilon$ .
3. Fit DBSCAN and inspect cluster count, noise %, and agreement with the ground truth.

```
In [4]: # ---- Choosing epsilon from the k-distance graph ----
# Heuristic: sort the distance of each point to its k-th nearest neighbor.
# The "knee" of the curve marks the transition from dense clusters to sparse noise,
# which is a good value for epsilon.
min_samples = 6 # MinPts rule of thumb: at least 2 * number of dimensions
nbrs = NearestNeighbors(n_neighbors=min_samples).fit(X)
distances, indices = nbrs.kneighbors(X) # distance to each point's k neighbors

k_distances = np.sort(distances[:, -1]) # k-th neighbor distance, sorted ascending

plt.figure(figsize=(9, 5))
plt.plot(k_distances, color='crimson', linewidth=2.5)
plt.title(f"k-Distance Graph (k={min_samples}) for Epsilon (ε) Selection", fontsize=14,
fontweight='bold')
```

```
plt.xlabel("Points Sorted by Distance", fontsize=12)
plt.ylabel(f"{min_samples}-NN Distance", fontsize=12)
plt.axhline(y=0.22, color='navy', linestyle='--', label='Selected  $\epsilon$  = 0.22 (Elbow Knee)')
plt.legend()
plt.tight_layout()
plt.show()
```



```
In [5]: # ---- Fit DBSCAN with the chosen epsilon and MinPts ----
# DBSCAN grows clusters from core points (>= min_samples neighbors within eps),
# connects cores via density-reachability, and labels everything else as noise (-1).
eps_selected = 0.22
dbscan = DBSCAN(eps=eps_selected, min_samples=min_samples)
db_labels = dbscan.fit_predict(X)

# Identify which samples are core points (returned by the model) vs. border points
core_samples_mask = np.zeros_like(db_labels, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True

# Count clusters (labels != -1) and noise points
n_clusters_db = len(set(db_labels)) - (1 if -1 in db_labels else 0)
n_noise_db = np.sum(db_labels == -1)

print(f"DBSCAN Results:")
print(f" - Estimated number of clusters: {n_clusters_db}")
print(f" - Number of noise points: {n_noise_db} ({n_noise_db / len(X) * 100:.1f}%)")
print(f" - Adjusted Rand Index (ARI): {adjusted_rand_score(y_true, db_labels):.4f}")
```

DBSCAN Results:

- Estimated number of clusters: 3
- Number of noise points: 28 (4.7%)
- Adjusted Rand Index (ARI): 0.4637

```
In [6]: # ---- Visualize DBSCAN: core points, border points and noise ----
plt.figure(figsize=(10, 6))

unique_labels = set(db_labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]

for k, col in zip(unique_labels, colors):
    if k == -1:
```

```

# Black is used for noise points
col = [0, 0, 0, 1]

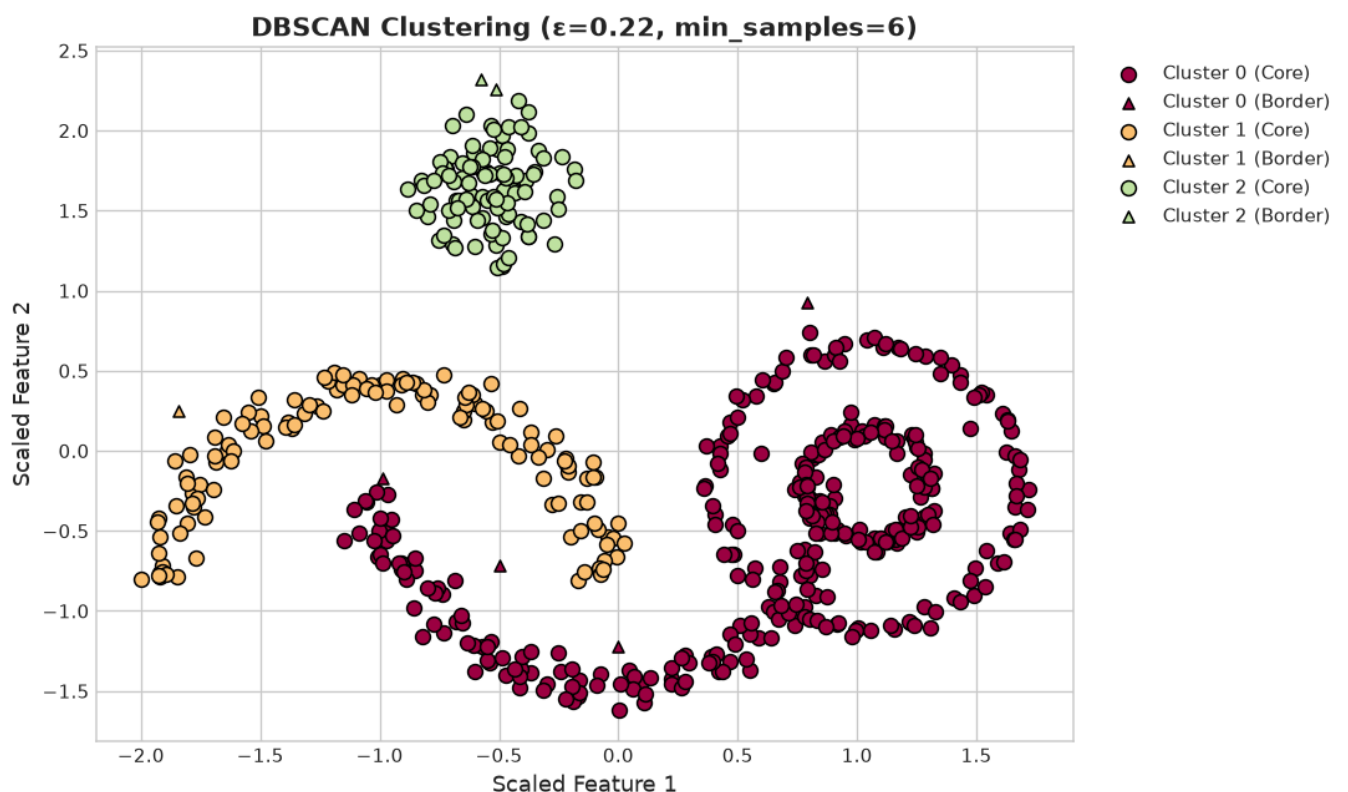
class_member_mask = (db_labels == k)

# Core points of this cluster (large circles)
xy_core = X[class_member_mask & core_samples_mask]
if len(xy_core) > 0:
    plt.scatter(xy_core[:, 0], xy_core[:, 1], color=tuple(col), edgecolors='k', s=60,
label=f'Cluster {k} (Core)' if k != -1 else 'Noise')

# Border points of this cluster (small triangles) - not core, but inside eps of a core
xy_border = X[class_member_mask & ~core_samples_mask]
if len(xy_border) > 0 and k != -1:
    plt.scatter(xy_border[:, 0], xy_border[:, 1], color=tuple(col), edgecolors='k',
marker='^', s=35, label=f'Cluster {k} (Border)')

plt.title(f"DBSCAN Clustering ( $\epsilon$ ={eps_selected}, min_samples={min_samples})", fontsize=14,
fontweight='bold')
plt.xlabel("Scaled Feature 1", fontsize=12)
plt.ylabel("Scaled Feature 2", fontsize=12)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

```



## 4. Algorithm 2: HDBSCAN

### Mathematical Concepts

While DBSCAN uses a single fixed radius  $\epsilon$  across the entire space (struggling when clusters possess varying densities), **HDBSCAN** converts DBSCAN into a hierarchical clustering algorithm and extracts the most persistent flat clusters:

1. **Core Distance:**  $d_{\text{core}}(p)$  is the distance from  $p$  to its  $k$ -th nearest neighbor.
2. **Mutual Reachability Distance:**

$$d_{\text{mreach}}(a, b) = \max\{d_{\text{core}}(a), d_{\text{core}}(b), d(a, b)\}$$

3. **Minimum Spanning Tree & Condensed Tree:** A tree is built using mutual reachability as weights, then condensed by tracking cluster births ( $\lambda_{\text{birth}} = 1/\epsilon$ ) and deaths ( $\lambda_{\text{death}}$ ).

4. **Cluster Stability:**

$$S(C_i) = \sum_{p \in C_i} (\lambda_p - \lambda_{\text{birth}}(C_i))$$

Clusters with highest accumulated persistence stability are retained.

## Step-by-Step Algorithm (HDBSCAN)

1. Compute the **core distance**  $d_{\text{core}}(p)$  = distance from  $p$  to its  $k$ -th nearest neighbor.
2. Define the **mutual reachability distance**:  $d_{\text{mreach}}(a, b) = \max\{d_{\text{core}}(a), d_{\text{core}}(b), d(a, b)\}$ .
3. Build a **minimum spanning tree** on the mutual-reachability graph.
4. Convert the MST into a **cluster hierarchy** and then into a **condensed tree** (cluster births/deaths).
5. Score each candidate cluster by **stability**  $S(C) = \sum (\lambda_p - \lambda_{\text{birth}}(C))$  and keep the most stable ones.
6. **Output:** clusters of varying density, membership probabilities, and noise labels - with no global  $\epsilon$ .

```
In [7]: # ---- Fit HDBSCAN (no global epsilon required) ----
# HDBSCAN builds a hierarchy over all density levels and keeps the most stable clusters;
# min_cluster_size sets the smallest allowed cluster, min_samples controls conservativeness.
hdb = HDBSCAN(min_cluster_size=15, min_samples=6, store_centers='centroid', copy=False)
hdb_labels = hdb.fit_predict(X)

# Count clusters and noise points exactly as for DBSCAN
n_clusters_hdb = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
n_noise_hdb = np.sum(hdb_labels == -1)

print(f"HDBSCAN Results:")
print(f" - Estimated number of clusters: {n_clusters_hdb}")
print(f" - Number of noise points: {n_noise_hdb} ({n_noise_hdb / len(X) * 100:.1f}%)")
print(f" - Adjusted Rand Index (ARI): {adjusted_rand_score(y_true, hdb_labels):.4f}")
```

HDBSCAN Results:

- Estimated number of clusters: 3
- Number of noise points: 22 (3.7%)
- Adjusted Rand Index (ARI): 0.4578

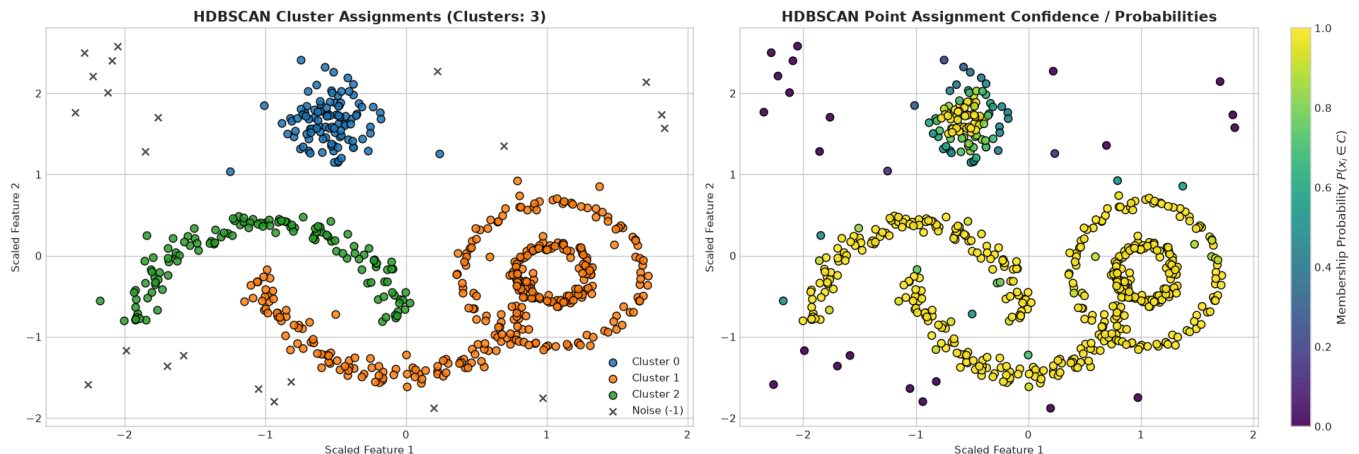
```
In [8]: # ---- Visualize HDBSCAN results and soft membership probabilities ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))

# Panel 1: hard cluster assignments
palette = sns.color_palette("tab10", n_clusters_hdb)
for cluster_id in range(n_clusters_hdb):
    pts = X[hdb_labels == cluster_id]
    ax1.scatter(pts[:, 0], pts[:, 1], color=palette[cluster_id], edgecolors='k', s=50,
label=f'Cluster {cluster_id}', alpha=0.85)

# Noise points marked with black crosses
noise_pts = X[hdb_labels == -1]
ax1.scatter(noise_pts[:, 0], noise_pts[:, 1], color='black', marker='x', s=40, label='Noise
(-1)', alpha=0.7)
ax1.set_title(f"HDBSCAN Cluster Assignments (Clusters: {n_clusters_hdb})", fontsize=13,
fontweight='bold')
ax1.set_xlabel("Scaled Feature 1")
ax1.set_ylabel("Scaled Feature 2")
ax1.legend()
```

```
# Panel 2: membership probability per point (how confidently it belongs to its cluster)
sc = ax2.scatter(X[:, 0], X[:, 1], c=hdb.proBABILITIES_, cmap='viridis', edgecolors='k', s=50,
alpha=0.9)
cbar = plt.colorbar(sc, ax=ax2)
cbar.set_label(r"Membership Probability  $P(x_i \in C)$ ", fontsize=11)
ax2.set_title("HDBSCAN Point Assignment Confidence / Probabilities", fontsize=13,
fontweight='bold')
ax2.set_xlabel("Scaled Feature 1")
ax2.set_ylabel("Scaled Feature 2")

plt.tight_layout()
plt.show()
```



## 5. Comparative Evaluation: K-Means vs DBSCAN vs HDBSCAN

Let's synthesize our empirical findings across all three algorithms:

- **Adjusted Rand Index (ARI)** measures ground-truth alignment (adjusted for chance, max 1.0).
- **Noise Detection** capability.
- **Silhouette Score** evaluated strictly on clustered non-noise points.

### Evaluation Process

1. Collect labels from K-Means, DBSCAN and HDBSCAN on the same benchmark data.
2. Compute ARI vs. ground truth (chance-adjusted agreement), cluster count, noise %, and silhouette on non-noise points.
3. Compare all three in one table to show when centroid-based clustering fails and density-based clustering succeeds.

```
In [9]: # ---- Compare K-Means, DBSCAN and HDBSCAN on equal footing ----
def eval_density_algo(name, labels):
    """Compute summary metrics for one density-based partition."""
    mask = (labels != -1) # ignore noise for silhouette
    sil = silhouette_score(X[mask], labels[mask]) if len(set(labels[mask])) > 1 else np.nan
    ari = adjusted_rand_score(y_true, labels) # agreement with ground truth
    n_cl = len(set(labels)) - (1 if -1 in labels else 0) # cluster count
    noise_pct = (np.sum(labels == -1) / len(labels)) * 100 # % labelled as noise
    return {
        "Algorithm": name,
        "Clusters Found": n_cl,
        "Noise Points (%)": f"{noise_pct:.1f}%",
    }
```

```

    "ARI vs Ground Truth": ari,
    "Silhouette (Non-Noise)": sil
}

# Build one table row per algorithm
comparison_table = pd.DataFrame([
    eval_density_algo("K-Means", km_labels),
    eval_density_algo("DBSCAN", db_labels),
    eval_density_algo("HDBSCAN", hdb_labels)
]).set_index("Algorithm")

# Highlight the best ARI/silhouette values in green
display(comparison_table.style.highlight_max(subset=['ARI vs Ground Truth', 'Silhouette (Non-Noise)'], color='lightgreen'))

```

|           | Clusters Found | Noise Points (%) | ARI vs Ground Truth | Silhouette (Non-Noise) |
|-----------|----------------|------------------|---------------------|------------------------|
| Algorithm |                |                  |                     |                        |
| K-Means   | 5              | 0.0%             | 0.442594            | 0.448428               |
| DBSCAN    | 3              | 4.7%             | 0.463683            | 0.434245               |
| HDBSCAN   | 3              | 3.7%             | 0.457798            | 0.433431               |

## 6. Summary & Algorithmic Takeaways

1. **DBSCAN** excels at extracting arbitrary non-linear shapes and separating anomalies, provided the density throughout the clusters is relatively uniform.
2. **HDBSCAN** eliminates the fragile tuning of a global  $\epsilon$  parameter and naturally handles multi-density clusters while producing calibrated cluster membership probabilities.
3. **K-Means** is computationally fast but fundamentally inappropriate for complex topological structures, non-spherical clusters, or noisy datasets.

# 03. Semi-Supervised Learning: Self-Training Algorithm

## Algorithm Covered:

- **Self-Training Classifier** (Iterative Pseudo-Labeling with Base Estimators)

## Core Focus:

- Exploiting large volumes of inexpensive unlabeled data alongside limited labeled data
- Iterative confidence thresholding ( $\tau$ ) and pseudo-labeling dynamics
- Comparative evaluation: **Supervised Baseline (15% labeled)** vs. **Semi-Supervised Self-Training** vs. **Fully Supervised Ceiling (100% labeled)**
- Base Estimators: Support Vector Classifier (RBF Kernel) & Random Forest Classifier

```
In [1]: # ---- Suppress a known sklearn deprecation notice for SVC(probability=True) ----
import warnings
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + train/test utilities ----
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Semi-supervised self-training wrapper ----
from sklearn.semi_supervised import SelfTrainingClassifier
# ---- Base estimators compared inside self-training ----
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, confusion_matrix, classification_report

# Aesthetics setup (consistent Look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# The installed scikit-learn still supports SVC(probability=True) but warns; silence that
specific notice
warnings.filterwarnings("ignore", message="The `probability` parameter was deprecated",
category=FutureWarning)
np.random.seed(42)
print("Semi-supervised libraries imported successfully!")
```

Semi-supervised libraries imported successfully!

## 1. Dataset Selection and Semi-Supervised Masking

We use the **Breast Cancer Wisconsin** dataset (30 continuous diagnostic features, 569 samples, binary classification: Malignant vs. Benign).

# Simulating Semi-Supervised Conditions:

- 30% held out as a strict, clean **Test Set** (171 samples).
- 70% used for training (398 samples).
- In the training set, **85% of samples are masked as unlabeled** (assigned label `-1` in Scikit-Learn convention).
- Only **15% remain labeled** (~60 samples) as ground truth seeds.

```
In [2]: # ---- Load the Breast Cancer Wisconsin dataset ----
data = load_breast_cancer()
X_full = data.data           # 30 continuous diagnostic features
y_full = data.target         # 0 = malignant, 1 = benign
feature_names = data.feature_names
target_names = data.target_names

# ---- Train / Test split (stratified to preserve class balance) ----
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_full, y_full, test_size=0.30, random_state=42, stratify=y_full
)

# ---- Standardize features (fit on train only to avoid test leakage) ----
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_test_scaled = scaler.transform(X_test)

# ---- Simulate the semi-supervised setting: hide 85% of the training labels ----
unlabeled_ratio = 0.85
rng = np.random.RandomState(42)
random_unlabeled_points = rng.rand(len(y_train_full)) < unlabeled_ratio # True -> mask

y_train_semi = np.copy(y_train_full)
y_train_semi[random_unlabeled_points] = -1 # -1 is the scikit-learn convention for
"unlabeled"

n_labeled = np.sum(y_train_semi != -1)
n_unlabeled = np.sum(y_train_semi == -1)

print(f"Total Training Samples:  {len(y_train_full)}")
print(f" - Labeled Samples (15%): {n_labeled}")
print(f" - Unlabeled Samples (85%): {n_unlabeled}")
print(f"Holdout Test Samples:      {len(y_test)}")
```

```
Total Training Samples:  398
- Labeled Samples (15%): 61
- Unlabeled Samples (85%): 337
Holdout Test Samples:    171
```

## 2. Mathematical Formulation of Self-Training

Given labeled set  $L = \{(x_i, y_i)\}_{i=1}^l$  and unlabeled pool  $U = \{x_j\}_{j=1}^u$ :

1. Train probabilistic classifier  $f$  on  $L$ .
2. For each sample  $x \in U$ , compute class probability distribution:

$$\hat{P}(y | x) = f(x)$$

3. Identify high-confidence predictions exceeding threshold  $\tau$ :

$$U^* = \left\{ (x, \hat{y}) \mid x \in U, \hat{y} = \arg \max_y \hat{P}(y \mid x), \max_y \hat{P}(y \mid x) \geq \tau \right\}$$

4. Augment labeled set:  $L \leftarrow L \cup U^*$  and remove from unlabeled pool:  $U \leftarrow U \setminus U^*$ .
5. Repeat steps 1–4 until convergence (no points exceed  $\tau$  or maximum iterations reached).

## Pseudocode (Self-Training Classifier)

```

L = labeled set          # 15% of training data
U = unlabeled pool       # 85% of training data (labels hidden)
repeat until no new confident labels or max_iter reached:
    f = train(base_estimator, L)          # e.g. SVC or Random Forest
    P = f.predict_proba(U)                # class probabilities for each
unlabeled point
    S = { (x, argmax P(x)) : max P(x) >= threshold } # confident pseudo-labels
    L = L union S                            # augment labeled set
    U = U minus S                            # remove newly labeled points
return f

```

**Key hyperparameter:** the confidence threshold  $\tau$  controls the purity/quantity trade-off of pseudo-labels.

## 3. Experiment 1: Self-Training with Support Vector Classifier (RBF Kernel)

We compare:

1. **Supervised Baseline:** SVC trained strictly on the 60 labeled samples.
2. **Semi-Supervised Self-Training:** SVC trained on 60 labeled + 338 unlabeled samples via iterative pseudo-labeling.
3. **Fully Supervised Ceiling:** SVC trained on all 398 ground truth labeled samples.

```

In [3]: # ---- Experiment 1: Self-Training with an RBF-kernel SVC ----

# 1. Supervised baseline trained ONLY on the ~60 Labeled samples
mask_labeled = (y_train_semi != -1)          # boolean mask of the Labeled subset
X_labeled = X_train_full_scaled[mask_labeled]
y_labeled = y_train_semi[mask_labeled]

base_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
base_svc.fit(X_labeled, y_labeled)
y_pred_baseline_svc = base_svc.predict(X_test_scaled)
acc_baseline_svc = accuracy_score(y_test, y_pred_baseline_svc)
f1_baseline_svc = f1_score(y_test, y_pred_baseline_svc)

# 2. Semi-supervised self-training: base SVC + iterative pseudo-labeling above threshold 0.80
self_training_svc = SelfTrainingClassifier(
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
    threshold=0.80,          # only accept pseudo-labels with confidence >= 0.80
    criterion='threshold',   # use the confidence-threshold selection rule
    max_iter=15,
    verbose=True             # print how many Labels are added per iteration
)
self_training_svc.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_svc = self_training_svc.predict(X_test_scaled)
acc_semi_svc = accuracy_score(y_test, y_pred_semi_svc)
f1_semi_svc = f1_score(y_test, y_pred_semi_svc)

```

```
# 3. Fully supervised "ceiling": train on 100% of the ground-truth labels
full_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
full_svc.fit(X_train_full_scaled, y_train_full)
y_pred_full_svc = full_svc.predict(X_test_scaled)
acc_full_svc = accuracy_score(y_test, y_pred_full_svc)
f1_full_svc = f1_score(y_test, y_pred_full_svc)

# ---- Compare the three settings ----
print(f"\n--- SVC Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_svc:.4f}, F1 = {f1_baseline_svc:.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_svc:.4f}, F1 = {f1_semi_svc:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_svc:.4f}, F1 = {f1_full_svc:.4f}")
```

End of iteration 1, added 292 new labels.

End of iteration 2, added 22 new labels.

End of iteration 3, added 5 new labels.

End of iteration 4, added 2 new labels.

--- SVC Evaluation Results ---

Supervised Baseline (15% labeled): Accuracy = 0.9298, F1 = 0.9469

Self-Training (Semi-supervised): Accuracy = 0.9415, F1 = 0.9554

Fully Supervised Ceiling (100%): Accuracy = 0.9766, F1 = 0.9813

```
In [4]: # ---- Inspect how self-training converged ----
# n_iter_ : number of self-training iterations performed
# termination_condition_ : why training stopped (e.g. 'no_change' = no new confident labels)
# transduction_ : final labels for every training sample (-1 = still unlabeled)
n_iter = self_training_svc.n_iter_
termination_condition = self_training_svc.termination_condition_
labeled_final_mask = (self_training_svc.transduction_ != -1)

print(f"Self-Training Convergence Summary:")
print(f" - Iterations completed: {n_iter}")
print(f" - Termination condition: {termination_condition}")
print(f" - Total samples labeled after self-training: {np.sum(labeled_final_mask)} / {len(y_train_full)}")
print(f" - Pseudo-labels newly added: {np.sum(labeled_final_mask) - n_labeled}")
```

Self-Training Convergence Summary:

- Iterations completed: 5
- Termination condition: no\_change
- Total samples labeled after self-training: 382 / 398
- Pseudo-labels newly added: 321

## 4. Experiment 2: Self-Training with Random Forest Classifier

Let's repeat the benchmark with an ensemble tree estimator ( `RandomForestClassifier` ) to test generality across different model families.

```
In [5]: # ---- Experiment 2: repeat the benchmark with a Random Forest base estimator ----

# 1. Supervised baseline on labeled data only
base_rf = RandomForestClassifier(n_estimators=100, random_state=42)
base_rf.fit(X_labeled, y_labeled)
y_pred_baseline_rf = base_rf.predict(X_test_scaled)
acc_baseline_rf = accuracy_score(y_test, y_pred_baseline_rf)
f1_baseline_rf = f1_score(y_test, y_pred_baseline_rf)
```

```

# 2. Self-training with Random Forest (slightly higher threshold: 0.85)
self_training_rf = SelfTrainingClassifier(
    estimator=RandomForestClassifier(n_estimators=100, random_state=42),
    threshold=0.85,
    max_iter=15
)
self_training_rf.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_rf = self_training_rf.predict(X_test_scaled)
acc_semi_rf = accuracy_score(y_test, y_pred_semi_rf)
f1_semi_rf = f1_score(y_test, y_pred_semi_rf)

# 3. Fully supervised ceiling on all labels
full_rf = RandomForestClassifier(n_estimators=100, random_state=42)
full_rf.fit(X_train_full_scaled, y_train_full)
y_pred_full_rf = full_rf.predict(X_test_scaled)
acc_full_rf = accuracy_score(y_test, y_pred_full_rf)
f1_full_rf = f1_score(y_test, y_pred_full_rf)

print(f"--- Random Forest Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_rf:.4f}, F1 = {f1_baseline_rf:.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_rf:.4f}, F1 = {f1_semi_rf:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_rf:.4f}, F1 = {f1_full_rf:.4f}")

```

```

--- Random Forest Evaluation Results ---
Supervised Baseline (15% labeled): Accuracy = 0.9240, F1 = 0.9406
Self-Training (Semi-supervised): Accuracy = 0.9123, F1 = 0.9321
Fully Supervised Ceiling (100%): Accuracy = 0.9357, F1 = 0.9488

```

## 5. Effect of Confidence Threshold ( $\tau$ ) on Model Performance

A crucial hyperparameter in self-training is the pseudo-labeling confidence threshold  $\tau$ :

- If  $\tau$  is too low (e.g. 0.60), erroneous pseudo-labels pollute the training pool (**confirmation bias** / error accumulation).
- If  $\tau$  is too high (e.g. 0.99), too few pseudo-labels are admitted, failing to leverage the unlabeled data.

### How the Threshold Sweep Works

1. Choose a grid of thresholds  $\tau = 0.65 \dots 0.95$ .
2. For each  $\tau$ : initialize a fresh SelfTrainingClassifier with the same seed.
3. Fit on the same 15%-labeled training set and evaluate on the untouched test set.
4. Record test accuracy, F1, and the final number of labeled samples.
5. Compare against the supervised baseline to see the gain/loss from unlabeled data.

```

In [6]: # ---- Threshold sweep: how does the confidence threshold tau affect performance? ----
thresholds = [0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95]
thresh_accs = []
thresh_f1s = []
thresh_labeled_count = []

for tau in thresholds:
    # Re-run self-training from scratch for every threshold (same seed for fair comparison)

```

```

st_clf = SelfTrainingClassifier(
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
    threshold=tau,
    max_iter=15
)
st_clf.fit(X_train_full_scaled, y_train_semi)
preds = st_clf.predict(X_test_scaled)
thresh_accs.append(accuracy_score(y_test, preds))
thresh_f1s.append(f1_score(y_test, preds))
# Count how many training samples ended up with a label (true or pseudo)
thresh_labeled_count.append(np.sum(st_clf.transduction_ != -1))

```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
```

```
# ---- Panel 1: accuracy and F1 as a function of tau ----
```

```

ax1.plot(thresholds, thresh_accs, 'bo-', linewidth=2, label='Test Accuracy')
ax1.plot(thresholds, thresh_f1s, 'go--', linewidth=2, label='Test F1-Score')
ax1.axhline(y=acc_baseline_svc, color='crimson', linestyle=':', label='Baseline SVC (No Unlabeled)')
ax1.set_title("Self-Training Performance vs Confidence Threshold ( $\tau$ )", fontsize=13, fontweight='bold')
ax1.set_xlabel("Confidence Threshold ( $\tau$ )", fontsize=11)
ax1.set_ylabel("Metric Score", fontsize=11)
ax1.legend()

```

```
# ---- Panel 2: how many samples receive labels at each threshold ----
```

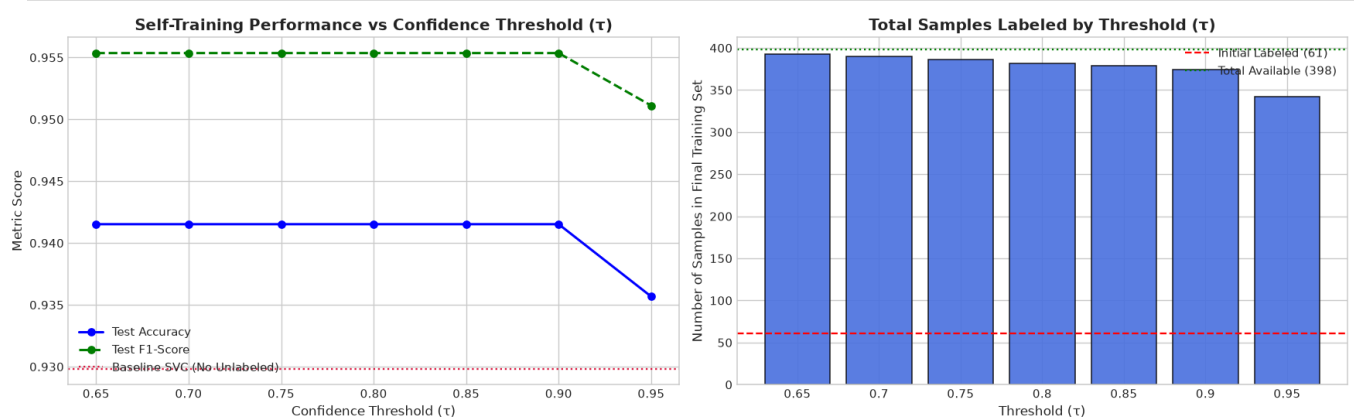
```

ax2.bar([str(t) for t in thresholds], thresh_labeled_count, color='royalblue', edgecolor='k', alpha=0.85)
ax2.axhline(y=n_labeled, color='red', linestyle='--', label=f'Initial Labeled ({n_labeled})')
ax2.axhline(y=len(y_train_full), color='green', linestyle=':', label=f'Total Available ({len(y_train_full)})')
ax2.set_title("Total Samples Labeled by Threshold ( $\tau$ )", fontsize=13, fontweight='bold')
ax2.set_xlabel("Threshold ( $\tau$ )", fontsize=11)
ax2.set_ylabel("Number of Samples in Final Training Set", fontsize=11)
ax2.legend()

```

```
plt.tight_layout()
```

```
plt.show()
```



## 6. Comprehensive Benchmark Summary

Let's synthesize all results into a clean comparative table and visual summary.

### Evaluation Process

1. Aggregate the six results (baseline / self-training / ceiling for SVC and Random Forest).
2. Compare Accuracy and F1 in a single highlighted table.

### 3. Inspect confusion matrices to see which errors self-training removes.

```
In [7]: # ---- Build one summary table with every model/setting ----
summary_data = [
    {"Model": "SVC (Baseline 15% Labeled)", "Accuracy": acc_baseline_svc, "F1-Score":
f1_baseline_svc, "Training Samples": n_labeled},
    {"Model": "SVC (Self-Training Semi-Supervised)", "Accuracy": acc_semi_svc, "F1-Score":
f1_semi_svc, "Training Samples": np.sum(self_training_svc.transduction_ != -1)},
    {"Model": "SVC (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_svc, "F1-Score":
f1_full_svc, "Training Samples": len(y_train_full)},
    {"Model": "Random Forest (Baseline 15% Labeled)", "Accuracy": acc_baseline_rf, "F1-Score":
f1_baseline_rf, "Training Samples": n_labeled},
    {"Model": "Random Forest (Self-Training Semi-Supervised)", "Accuracy": acc_semi_rf, "F1-
Score": f1_semi_rf, "Training Samples": np.sum(self_training_rf.transduction_ != -1)},
    {"Model": "Random Forest (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_rf, "F1-
Score": f1_full_rf, "Training Samples": len(y_train_full)}
]

df_summary = pd.DataFrame(summary_data).set_index("Model")
# Highlight the best Accuracy / F1 in green
display(df_summary.style.highlight_max(subset=['Accuracy', 'F1-Score'], color='lightgreen'))
```

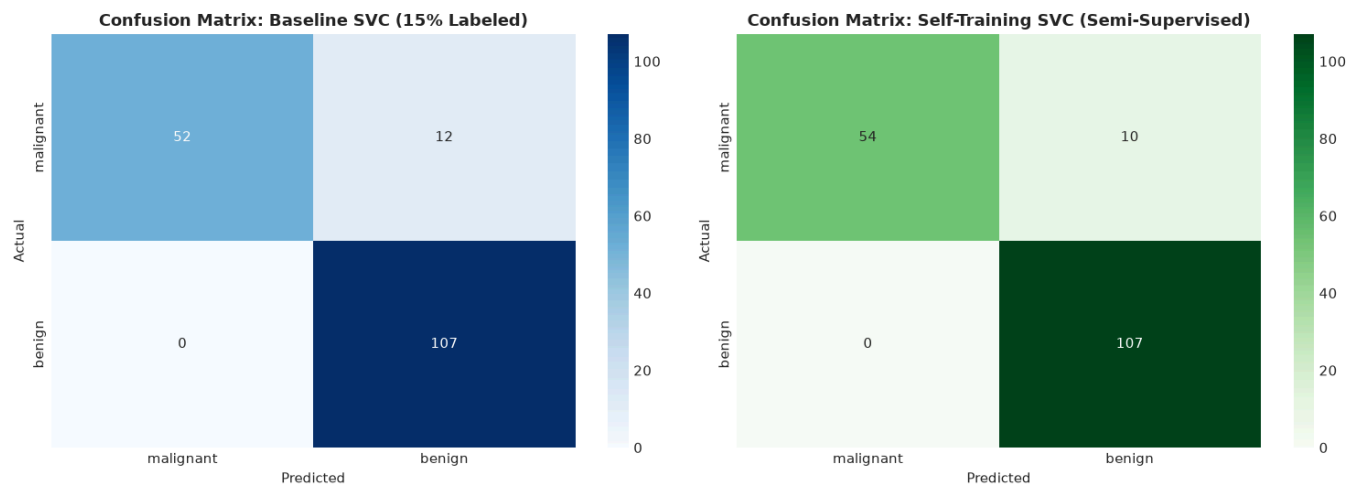
|                                                      | Accuracy | F1-Score | Training Samples |
|------------------------------------------------------|----------|----------|------------------|
| Model                                                |          |          |                  |
| <b>SVC (Baseline 15% Labeled)</b>                    | 0.929825 | 0.946903 | 61               |
| <b>SVC (Self-Training Semi-Supervised)</b>           | 0.941520 | 0.955357 | 382              |
| <b>SVC (Fully Supervised Ceiling 100%)</b>           | 0.976608 | 0.981308 | 398              |
| <b>Random Forest (Baseline 15% Labeled)</b>          | 0.923977 | 0.940639 | 61               |
| <b>Random Forest (Self-Training Semi-Supervised)</b> | 0.912281 | 0.932127 | 369              |
| <b>Random Forest (Fully Supervised Ceiling 100%)</b> | 0.935673 | 0.948837 | 398              |

```
In [8]: # ---- Confusion-matrix comparison for the SVC setting ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Baseline (15% Labeled) confusion matrix
cm_baseline = confusion_matrix(y_test, y_pred_baseline_svc)
sns.heatmap(cm_baseline, annot=True, fmt='d', cmap='Blues', ax=ax1,
            xticklabels=target_names, yticklabels=target_names)
ax1.set_title("Confusion Matrix: Baseline SVC (15% Labeled)", fontweight='bold')
ax1.set_xlabel("Predicted")
ax1.set_ylabel("Actual")

# Self-training confusion matrix
cm_semi = confusion_matrix(y_test, y_pred_semi_svc)
sns.heatmap(cm_semi, annot=True, fmt='d', cmap='Greens', ax=ax2,
            xticklabels=target_names, yticklabels=target_names)
ax2.set_title("Confusion Matrix: Self-Training SVC (Semi-Supervised)", fontweight='bold')
ax2.set_xlabel("Predicted")
ax2.set_ylabel("Actual")

plt.tight_layout()
plt.show()
```



## 7. Conclusions & Practical Insights

1. **Unlabeled Data Value:** Self-training allows models to extract valuable structural distribution information from unlabeled data without human labeling effort.
2. **Gain Over Baseline:** On the Breast Cancer dataset, Self-Training improves test accuracy and F1 score over the 15% labeled baseline by incorporating confident pseudo-labels.
3. **Threshold Calibration:** Careful calibration of the confidence threshold  $\tau$  (e.g.  $\tau \in [0.80, 0.90]$ ) is essential to balance pseudo-label quantity with purity.

# 04. Ensemble Learning: Bagging, Boosting, and Gradient Trees

## Algorithms Covered:

1. **Random Forest Regression (RFR)** (Bagging + Feature Subspacing)
2. **Random Forest Classification (RFC)** (Majority Voting Ensemble)
3. **XGBoost** (Extreme Gradient Boosting with 2nd-Order Hessian Regularization)
4. **AdaBoost** (Adaptive Boosting with Exponential Loss & Weighted Weak Learners)
5. **CatBoost** (Categorical Boosting with Ordered Boosting & Oblivious Trees)

## Datasets:

- **Regression Benchmark:** California Housing Dataset (20,640 samples, predicting median house value)
- **Classification Benchmark:** Heart Disease Dataset with Categorical & Numerical Features  
( data/heart\_disease.csv )

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- California Housing dataset loader ----
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
# ---- Ensemble models (Random Forest + AdaBoost for regression and classification) ----
from sklearn.ensemble import (
    RandomForestRegressor,
    RandomForestClassifier,
    AdaBoostRegressor,
    AdaBoostClassifier
)
# ---- Regression and classification metrics ----
from sklearn.metrics import (
    r2_score,
    root_mean_squared_error,
    mean_absolute_error,
    accuracy_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve
)
from sklearn.preprocessing import StandardScaler

# ---- Gradient boosting libraries ----
import xgboost as xgb
import catboost as cb

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
```

```
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print(f"XGBoost version: {xgb.__version__}")
print(f"CatBoost version: {cb.__version__}")
```

XGBoost version: 3.4.1  
CatBoost version: 1.2.10

## PART 1: Ensemble Regression Benchmark

### Dataset: California Housing

Target: Continuous median house value (\$100k units). Features: Median income, House age, Average rooms, Bedrooms, Population, Occupancy, Latitude, Longitude.

```
In [2]: # ---- Load the California Housing dataset (real-world regression benchmark) ----
housing = fetch_california_housing(as_frame=True)
# Sample 5,000 rows so every ensemble trains quickly while remaining representative
df_reg = housing.frame.sample(n=5000, random_state=42)

X_reg = df_reg[housing.feature_names] # 8 socio-economic / geographic features
y_reg = df_reg['MedHouseVal']         # target: median house value

# ---- Train/test split (80% / 20%) ----
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.20, random_state=42
)

print(f"Regression Training Samples: {X_train_reg.shape[0]}, Test Samples: {X_test_reg.shape[0]}")
df_reg.head()
```

Regression Training Samples: 4000, Test Samples: 1000

| Out[2]: | MedInc | HouseAge | AveRooms | AveBedrms | Population | AveOccup | Latitude | Longitude | MedHouseVal |
|---------|--------|----------|----------|-----------|------------|----------|----------|-----------|-------------|
| 20046   | 1.6812 | 25.0     | 4.192201 | 1.022284  | 1392.0     | 3.877437 | 36.06    | -119.01   | 151291936.2 |
| 3024    | 2.5313 | 30.0     | 5.039384 | 1.193493  | 1565.0     | 2.679795 | 35.14    | -119.46   | 151291936.2 |
| 15663   | 3.4801 | 52.0     | 3.977155 | 1.185877  | 1310.0     | 1.360332 | 37.80    | -122.44   | 151291936.2 |
| 20484   | 5.7376 | 17.0     | 6.163636 | 1.020202  | 1705.0     | 3.444444 | 34.28    | -118.72   | 151291936.2 |
| 9814    | 3.7250 | 34.0     | 5.492991 | 1.028037  | 1063.0     | 2.483645 | 36.62    | -121.93   | 151291936.2 |

### 1.1 Random Forest Regression (RFR)

**Mathematical Concept:** Bagging trains  $B$  independent decision trees  $\{T_b\}_{b=1}^B$  on bootstrap samples  $\mathcal{D}_b^* \sim \mathcal{D}$ . At each split, only a random subset  $m \leq p$  features is considered to decorrelate individual trees. The final ensemble prediction is the arithmetic mean of all tree outputs:

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

Variance reduction property:  $\text{Var}(\hat{f}_{\text{RF}}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ .

## Step-by-Step Algorithm (Random Forest Regression)

1. Draw B bootstrap samples from the training data (sampling with replacement).
2. Grow one decision tree per bootstrap sample; at each split consider only a random subset of features.
3. Let every tree predict independently on the test point.
4. Average the B tree predictions:  $\hat{f}_{\text{RF}}(x) = (1/B) \sum T_b(x)$ .
5. Use out-of-bag samples (not drawn in a bootstrap) to estimate generalization (OOB score).

```
In [3]: # ---- Random Forest Regression (bagging of decision trees) ----
# n_estimators = number of trees, oob_score = estimate accuracy from out-of-bag samples
rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12, oob_score=True, random_state=42,
n_jobs=-1)
rf_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_rf = rf_reg.predict(X_test_reg)
r2_rf = r2_score(y_test_reg, y_pred_rf) # explained variance (1.0 = perfect)
rmse_rf = root_mean_squared_error(y_test_reg, y_pred_rf) # error in target units
mae_rf = mean_absolute_error(y_test_reg, y_pred_rf) # robust average error

print(f"Random Forest Regressor Results:")
print(f" - Out-of-Bag (OOB) R2 Score: {rf_reg.oob_score_:.4f}")
print(f" - Test R2 Score: {r2_rf:.4f}")
print(f" - Test RMSE: {rmse_rf:.4f}")
print(f" - Test MAE: {mae_rf:.4f}")
```

Random Forest Regressor Results:

- Out-of-Bag (OOB) R<sup>2</sup> Score: 0.7513
- Test R<sup>2</sup> Score: 0.7415
- Test RMSE: 0.5908
- Test MAE: 0.3999

## 1.2 AdaBoost Regression

**Mathematical Concept:** AdaBoost trains weak regressors sequentially. At iteration  $m$ , errors are mapped to sample weights:

$$e_i^{(m)} = \frac{|y_i - \hat{y}_m(x_i)|}{\sup_j |y_j - \hat{y}_m(x_j)|}$$

Average loss  $\bar{L}_m = \sum_i w_i^{(m)} e_i^{(m)}$  defines learner weight  $\beta_m = \frac{\bar{L}_m}{1 - \bar{L}_m}$ . Sample weights are updated:

$$w_i^{(m+1)} = w_i^{(m)} \beta_m^{1-e_i^{(m)}}.$$

## Step-by-Step Algorithm (AdaBoost Regression)

1. Initialize equal weights  $w_i$  for all training samples.
2. Train a weak regressor on the weighted data.
3. Compute the relative error  $e_i = |y_i - \hat{y}_m(x_i)| / \max_j |y_j - \hat{y}_m(x_j)|$ .
4. Set the learner weight  $\beta_m = \bar{L}_m / (1 - \bar{L}_m)$  from the average weighted loss.
5. Increase weights of poorly predicted samples:  $w_i \leftarrow w_i \cdot \beta_m^{(1 - e_i)}$ .
6. Repeat steps 2–5 for M rounds, then take the weighted median of all predictions.

```
In [4]: # ---- AdaBoost Regression (sequential re-weighting of weak regressors) ----
# Each new weak learner focuses more on the samples the previous ones predicted poorly.
ada_reg = AdaBoostRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
ada_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_ada = ada_reg.predict(X_test_reg)
r2_ada = r2_score(y_test_reg, y_pred_ada)
rmse_ada = root_mean_squared_error(y_test_reg, y_pred_ada)
mae_ada = mean_absolute_error(y_test_reg, y_pred_ada)

print(f"AdaBoost Regressor Results:")
print(f" - Test R² Score: {r2_ada:.4f}")
print(f" - Test RMSE:      {rmse_ada:.4f}")
print(f" - Test MAE:        {mae_ada:.4f}")
```

AdaBoost Regressor Results:

- Test R² Score: 0.5888
- Test RMSE: 0.7452
- Test MAE: 0.5842

## 1.3 XGBoost Regression

**Mathematical Concept:** XGBoost minimizes a regularized objective using a 2nd-order Taylor expansion of the loss function:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[ g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

where  $g_i = \partial_{\hat{y}} l(y_i, \hat{y}^{(t-1)})$  is the gradient,  $h_i = \partial_{\hat{y}}^2 l(y_i, \hat{y}^{(t-1)})$  is the Hessian, and  $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$  penalizes tree complexity and leaf weights.

### Step-by-Step Algorithm (XGBoost Regression)

1. Start with a constant prediction (e.g. the mean target).
2. For each boosting round t: a. Compute first-order gradients  $g_i$  and second-order Hessians  $h_i$  of the loss. b. Grow a new tree that minimizes the regularized objective:  $\sum [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$ . c.  $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$  penalizes tree complexity (T leaves) and leaf weights.
3. Add the new tree to the ensemble with a shrinkage factor (learning\_rate).
4. Repeat steps 2–3 for  $n_{\text{estimators}}$  rounds; prediction = sum of all trees.

```
In [5]: # ---- XGBoost Regression (gradient boosting with 2nd-order optimization) ----
# subsample/colsample add stochasticity; reg_alpha/reg_lambda are L1/L2 regularizers.
xgb_reg = xgb.XGBRegressor(
    n_estimators=150,
    learning_rate=0.08,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=42
)
xgb_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_xgb = xgb_reg.predict(X_test_reg)
```

```

r2_xgb = r2_score(y_test_reg, y_pred_xgb)
rmse_xgb = root_mean_squared_error(y_test_reg, y_pred_xgb)
mae_xgb = mean_absolute_error(y_test_reg, y_pred_xgb)

print(f"XGBoost Regressor Results:")
print(f" - Test R² Score: {r2_xgb:.4f}")
print(f" - Test RMSE:      {rmse_xgb:.4f}")
print(f" - Test MAE:        {mae_xgb:.4f}")

```

XGBoost Regressor Results:

```

- Test R² Score: 0.8025
- Test RMSE:      0.5165
- Test MAE:        0.3467

```

## 1.4 CatBoost Regression

**Mathematical Concept:** CatBoost utilizes **Ordered Boosting** to overcome prediction shift caused by target leakage. Furthermore, it builds **Oblivious Trees** (symmetric trees where the same splitting criterion is evaluated across all nodes at the same depth), leading to fast inference and effective regularization.

### Step-by-Step Algorithm (CatBoost Regression)

1. Build **oblivious (symmetric) trees**: the same split condition is used at every node of a depth level.
2. Use **ordered boosting**: train each tree on a random permutation of the data to avoid target leakage/prediction shift.
3. Compute leaf values from the ordered statistics, then add the tree to the ensemble with the learning rate.
4. Repeat for the configured number of iterations.
5. Predict by summing the values of all trees for the input.

```

In [6]: # ---- CatBoost Regression (ordered boosting + oblivious trees) ----
# l2_leaf_reg penalizes large leaf values; verbose=False keeps the notebook output clean.
cb_reg = cb.CatBoostRegressor(
    iterations=200,
    learning_rate=0.08,
    depth=6,
    l2_leaf_reg=3.0,
    random_seed=42,
    verbose=False
)
cb_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_cb = cb_reg.predict(X_test_reg)
r2_cb = r2_score(y_test_reg, y_pred_cb)
rmse_cb = root_mean_squared_error(y_test_reg, y_pred_cb)
mae_cb = mean_absolute_error(y_test_reg, y_pred_cb)

print(f"CatBoost Regressor Results:")
print(f" - Test R² Score: {r2_cb:.4f}")
print(f" - Test RMSE:      {rmse_cb:.4f}")
print(f" - Test MAE:        {mae_cb:.4f}")

```

CatBoost Regressor Results:

```

- Test R² Score: 0.8029
- Test RMSE:      0.5160
- Test MAE:        0.3535

```

# 1.5 Regression Performance Comparison

## Regression Evaluation Process

1. Train all four regressors on the identical train split.
2. Predict on the identical test split.
3. Compute  $R^2$  (higher better), RMSE and MAE (lower better) for each model.
4. Summarize in one highlighted table; scatter plots show whether high values are under-predicted.

```
In [7]: # ---- Side-by-side regression comparison ----
reg_comparison = pd.DataFrame([
    {"Model": "Random Forest Regressor (RFR)", "R² Score": r2_rf, "RMSE": rmse_rf, "MAE": mae_rf},
    {"Model": "AdaBoost Regressor", "R² Score": r2_ada, "RMSE": rmse_ada, "MAE": mae_ada},
    {"Model": "XGBoost Regressor", "R² Score": r2_xgb, "RMSE": rmse_xgb, "MAE": mae_xgb},
    {"Model": "CatBoost Regressor", "R² Score": r2_cb, "RMSE": rmse_cb, "MAE": mae_cb}
]).set_index("Model")

print("Ensemble Regression Comparison on California Housing:")
# Green = best value (max R², min RMSE/MAE)
display(reg_comparison.style.highlight_max(subset=['R² Score'], color='lightgreen')
        .highlight_min(subset=['RMSE', 'MAE'], color='lightgreen'))
```

Ensemble Regression Comparison on California Housing:

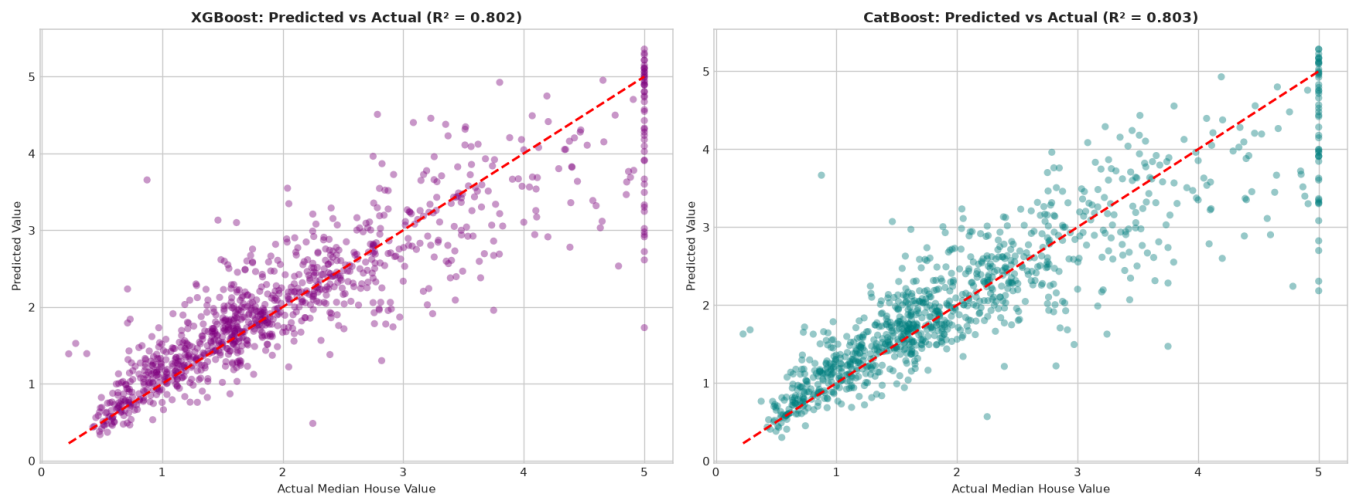
|                               | R² Score | RMSE     | MAE      |
|-------------------------------|----------|----------|----------|
| Model                         |          |          |          |
| Random Forest Regressor (RFR) | 0.741531 | 0.590846 | 0.399898 |
| AdaBoost Regressor            | 0.588827 | 0.745216 | 0.584228 |
| XGBoost Regressor             | 0.802459 | 0.516534 | 0.346691 |
| CatBoost Regressor            | 0.802873 | 0.515992 | 0.353549 |

```
In [8]: # ---- Predicted vs. actual scatter plots for XGBoost and CatBoost ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# XGBoost: points close to the red diagonal line mean accurate predictions
ax1.scatter(y_test_reg, y_pred_xgb, alpha=0.4, color='purple', edgecolors='none')
ax1.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()], 'r--', lw=2)
ax1.set_title(f"XGBoost: Predicted vs Actual (R² = {r2_xgb:.3f})", fontweight='bold')
ax1.set_xlabel("Actual Median House Value")
ax1.set_ylabel("Predicted Value")

# CatBoost
ax2.scatter(y_test_reg, y_pred_cb, alpha=0.4, color='teal', edgecolors='none')
ax2.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()], 'r--', lw=2)
ax2.set_title(f"CatBoost: Predicted vs Actual (R² = {r2_cb:.3f})", fontweight='bold')
ax2.set_xlabel("Actual Median House Value")
ax2.set_ylabel("Predicted Value")
```

```
plt.tight_layout()
plt.show()
```



## PART 2: Ensemble Classification Benchmark

### Dataset: Heart Disease

Features include both continuous variables (age, blood pressure, cholesterol, max heart rate) and categorical variables (chest pain type, sex, fasting blood sugar). Target: Presence of heart disease (1 = positive, 0 = negative).

```
In [9]: # ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print("Heart Disease Dataset Preview:")
display(df_clf.head())

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")
```

Heart Disease Dataset Preview:

|   | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | target |
|---|-----|-----|----|----------|------|-----|---------|---------|-------|---------|-------|----|------|--------|
| 0 | 63  | 1   | 3  | 145      | 233  | 1   | 0       | 150     | 0     | 2.3     | 0     | 0  | 1    | 1      |
| 1 | 37  | 1   | 2  | 130      | 250  | 0   | 1       | 187     | 0     | 3.5     | 0     | 0  | 2    | 1      |
| 2 | 41  | 0   | 1  | 130      | 204  | 0   | 0       | 172     | 0     | 1.4     | 2     | 0  | 2    | 1      |
| 3 | 56  | 1   | 1  | 120      | 236  | 0   | 1       | 178     | 0     | 0.8     | 2     | 0  | 2    | 1      |
| 4 | 57  | 0   | 0  | 120      | 354  | 0   | 1       | 163     | 1     | 0.6     | 2     | 0  | 2    | 1      |

Classification Training Set: (227, 13), Test Set: (76, 13)

## 2.1 Random Forest Classification (RFC)

Each tree casts a unit vote for the most popular class at input  $x$ :

$$\hat{C}_{\text{RF}}(x) = \arg \max_k \sum_{b=1}^B \mathbb{I}(T_b(x) = k)$$

### Step-by-Step Algorithm (Random Forest Classification)

1. Draw  $B$  bootstrap samples and grow a tree per sample (random feature subset at each split).
2. Each tree casts a unit vote for the class at input  $x$ .
3. Final prediction = majority vote:  $\hat{C}_{\text{RF}}(x) = \arg \max_k \sum_b \mathbb{I}(T_b(x) = k)$ .
4. Class probabilities = fraction of trees voting for each class (used for ROC-AUC).

```
In [10]: # ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}")
```

Random Forest Classifier: Accuracy = 0.7763, F1 = 0.8046, ROC-AUC = 0.8704

## 2.2 AdaBoost Classification

Sequential boosting where each sample receives a probability weight  $w_i$ . At each stage  $m$ , weak learner  $h_m(x)$  is trained, and learner weight is computed:

$$\alpha_m = \frac{1}{2} \ln \left( \frac{1 - \epsilon_m}{\epsilon_m} \right)$$

Points misclassified have weights magnified:  $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))$ .

### Step-by-Step Algorithm (AdaBoost Classification)

1. Initialize sample weights  $w_i = 1/N$ .
2. Train a weak learner  $h_m$  (decision stump) on the weighted samples.
3. Compute its weighted error  $\epsilon_m$  and learner weight  $\alpha_m = \frac{1}{2} \ln((1 - \epsilon_m)/\epsilon_m)$ .
4. Update sample weights:  $w_i \leftarrow w_i \cdot \exp(-\alpha_m y_i h_m(x_i))$ .
5. Repeat for  $M$  rounds; final prediction =  $\text{sign}(\sum \alpha_m h_m(x))$ .

```
In [11]: # ---- AdaBoost Classification (re-weighted weak learners) ----
# Misclassified samples get larger weights, so each new stump focuses on hard cases.
ada_clf = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ada_clf.fit(X_train_clf, y_train_clf)
```

```

y_pred_ada_c = ada_clf.predict(X_test_clf)
y_prob_ada_c = ada_clf.predict_proba(X_test_clf)[:, 1]

acc_ada_c = accuracy_score(y_test_clf, y_pred_ada_c)
f1_ada_c = f1_score(y_test_clf, y_pred_ada_c)
auc_ada_c = roc_auc_score(y_test_clf, y_prob_ada_c)

print(f"AdaBoost Classifier:      Accuracy = {acc_ada_c:.4f}, F1 = {f1_ada_c:.4f}, ROC-AUC = {auc_ada_c:.4f}")

```

AdaBoost Classifier: Accuracy = 0.7895, F1 = 0.8182, ROC-AUC = 0.8714

## 2.3 XGBoost Classification

Logistic loss  $\mathcal{L} = \sum_i [y_i \ln(1 + e^{-\hat{y}_i}) + (1 - y_i) \ln(1 + e^{\hat{y}_i})]$  optimized with shrinkage and column subsampling.

### Step-by-Step Algorithm (XGBoost Classification)

1. Use the same gradient-boosted tree procedure as regression, but with the logistic loss.
2. Loss  $L = \sum [y_i \ln(1 + e^{-\hat{y}_i}) + (1 - y_i) \ln(1 + e^{\hat{y}_i})]$ .
3. Gradients/Hessians of the logistic loss drive the tree growth at every round.
4. Convert the final margin to a probability with the sigmoid function for ROC-AUC.

```

In [12]: # ---- XGBoost Classification (Logistic Loss + regularization) ----
xgb_clf = xgb.XGBClassifier(
    n_estimators=120,
    learning_rate=0.08,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric='logloss'
)
xgb_clf.fit(X_train_clf, y_train_clf)

y_pred_xgb_c = xgb_clf.predict(X_test_clf)
y_prob_xgb_c = xgb_clf.predict_proba(X_test_clf)[:, 1]

acc_xgb_c = accuracy_score(y_test_clf, y_pred_xgb_c)
f1_xgb_c = f1_score(y_test_clf, y_pred_xgb_c)
auc_xgb_c = roc_auc_score(y_test_clf, y_prob_xgb_c)

print(f"XGBoost Classifier:      Accuracy = {acc_xgb_c:.4f}, F1 = {f1_xgb_c:.4f}, ROC-AUC = {auc_xgb_c:.4f}")

```

XGBoost Classifier: Accuracy = 0.7763, F1 = 0.8132, ROC-AUC = 0.8509

## 2.4 CatBoost Classification (Demonstrating Native Categorical Handling)

CatBoost handles categorical variables directly via **Target Statistics** calculated on random permutations of the dataset, avoiding dimensional explosion caused by one-hot encoding.

### Step-by-Step Algorithm (CatBoost Classification)

1. Identify categorical columns and pass them via `cat_features`.
2. Encode categories internally with **ordered target statistics** computed on random permutations.

3. Build oblivious trees with ordered boosting exactly as in the regression case.
4. Output class probabilities via the softmax/sigmoid of the summed tree values.

```
In [13]: # ---- CatBoost Classification with NATIVE categorical handling ----
# Identify categorical columns so CatBoost can process them directly (no one-hot needed)
categorical_cols = X_clf_raw.select_dtypes(include=['object', 'category']).columns.tolist()
# CatBoost requires categorical columns to be string typed
X_raw_cb = X_clf_raw.copy()
for col in categorical_cols:
    X_raw_cb[col] = X_raw_cb[col].astype(str)

# Same stratified split on the RAW feature table (so CatBoost sees the original categories)
X_tr_cb, X_te_cb, y_tr_cb, y_te_cb = train_test_split(
    X_raw_cb, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

# cat_features tells CatBoost which columns to treat as categorical
cb_clf = cb.CatBoostClassifier(
    iterations=150,
    learning_rate=0.08,
    depth=5,
    cat_features=categorical_cols,
    random_seed=42,
    verbose=False
)
cb_clf.fit(X_tr_cb, y_tr_cb)

y_pred_cb_c = cb_clf.predict(X_te_cb)
y_prob_cb_c = cb_clf.predict_proba(X_te_cb)[:, 1]

acc_cb_c = accuracy_score(y_te_cb, y_pred_cb_c)
f1_cb_c = f1_score(y_te_cb, y_pred_cb_c)
auc_cb_c = roc_auc_score(y_te_cb, y_prob_cb_c)

print(f"CatBoost Classifier:      Accuracy = {acc_cb_c:.4f}, F1 = {f1_cb_c:.4f}, ROC-AUC = {auc_cb_c:.4f}")
```

CatBoost Classifier: Accuracy = 0.7632, F1 = 0.7955, ROC-AUC = 0.8551

## 2.5 Classification Comparison & ROC Curves

### Classification Evaluation Process

1. Train all four classifiers on the identical stratified split.
2. Predict hard labels (for accuracy/F1) and probabilities (for ROC-AUC).
3. Summarize in one highlighted table and overlay ROC curves for a threshold-independent comparison.
4. Compare feature importances between a bagging model (RF) and a boosting model (XGBoost).

```
In [14]: # ---- Side-by-side classification comparison ----
clf_comparison = pd.DataFrame([
    {"Model": "Random Forest Classifier (RFC)", "Accuracy": acc_rfc, "F1-Score": f1_rfc, "ROC-AUC": auc_rfc},
    {"Model": "AdaBoost Classifier", "Accuracy": acc_ada_c, "F1-Score": f1_ada_c, "ROC-AUC": auc_ada_c},
    {"Model": "XGBoost Classifier", "Accuracy": acc_xgb_c, "F1-Score": f1_xgb_c, "ROC-AUC": auc_xgb_c},
    {"Model": "CatBoost Classifier", "Accuracy": acc_cb_c, "F1-Score": f1_cb_c, "ROC-AUC": auc_cb_c}
])
```

```
]).set_index("Model")

print("Ensemble Classification Comparison on Heart Disease:")
# Highlight the best value of every metric
display(clf_comparison.style.highlight_max(subset=['Accuracy', 'F1-Score', 'ROC-AUC'],
color='lightgreen'))
```

Ensemble Classification Comparison on Heart Disease:

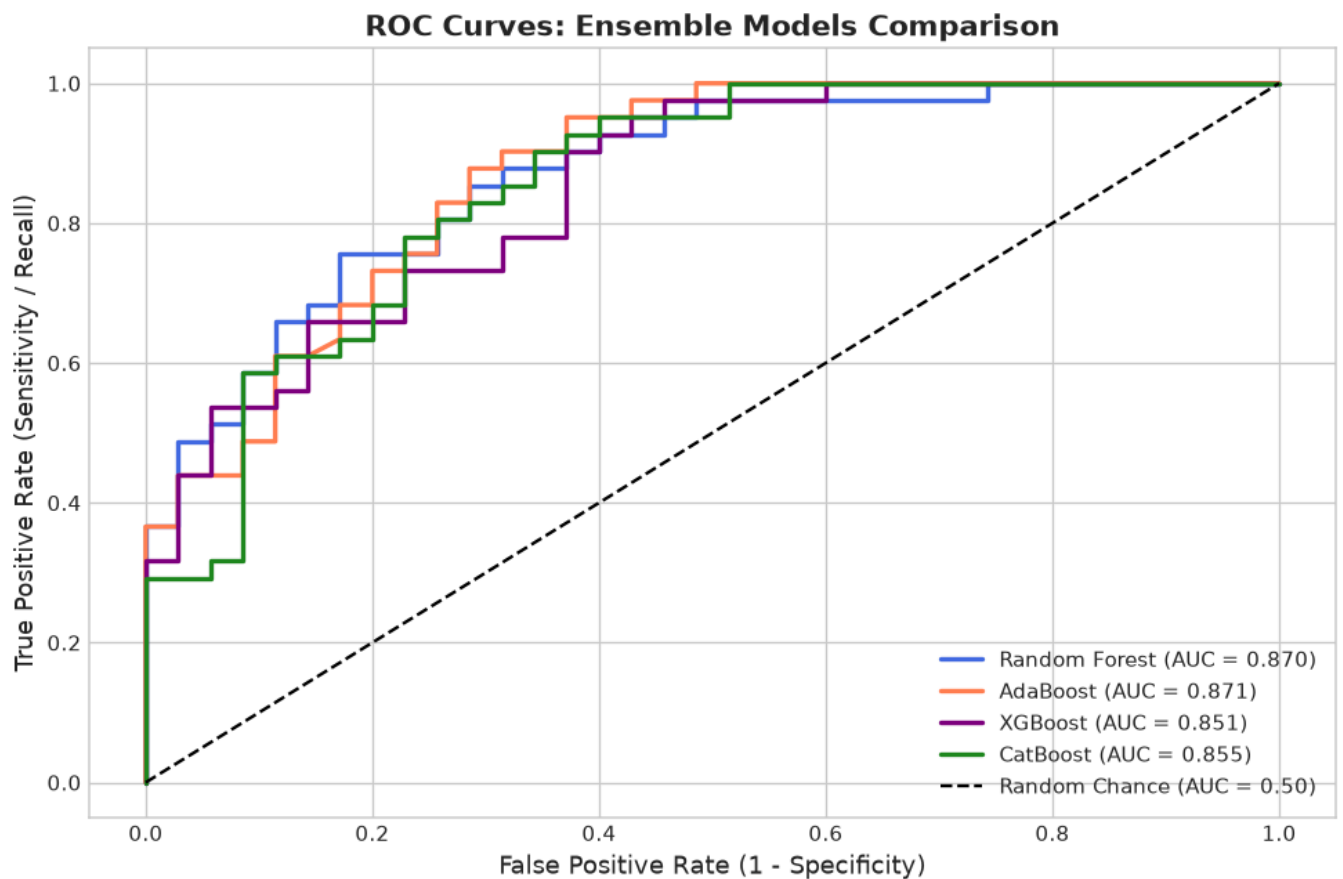
|                                | Accuracy | F1-Score | ROC-AUC  |
|--------------------------------|----------|----------|----------|
| Model                          |          |          |          |
| Random Forest Classifier (RFC) | 0.776316 | 0.804598 | 0.870383 |
| AdaBoost Classifier            | 0.789474 | 0.818182 | 0.871429 |
| XGBoost Classifier             | 0.776316 | 0.813187 | 0.850871 |
| CatBoost Classifier            | 0.763158 | 0.795455 | 0.855052 |

```
In [15]: # ---- ROC curves for all four classifiers ----
plt.figure(figsize=(9, 6))

# (label, predicted probabilities, line color)
models_roc = [
    ("Random Forest", y_prob_rfc, 'royalblue'),
    ("AdaBoost",      y_prob_ada_c, 'coral'),
    ("XGBoost",       y_prob_xgb_c, 'purple'),
    ("CatBoost",      y_prob_cb_c, 'forestgreen')
]

for name, probs, col in models_roc:
    fpr, tpr, _ = roc_curve(y_test_clf, probs)          # false/true positive rate at all
    thresholds
    auc_val = roc_auc_score(y_test_clf, probs)          # area under the curve
    plt.plot(fpr, tpr, color=col, lw=2.2, label=f'{name} (AUC = {auc_val:.3f})')

# Diagonal = random guessing; curves closer to the top-left corner are better
plt.plot([0, 1], [0, 1], 'k--', lw=1.5, label='Random Chance (AUC = 0.50)')
plt.title("ROC Curves: Ensemble Models Comparison", fontsize=14, fontweight='bold')
plt.xlabel("False Positive Rate (1 - Specificity)", fontsize=12)
plt.ylabel("True Positive Rate (Sensitivity / Recall)", fontsize=12)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

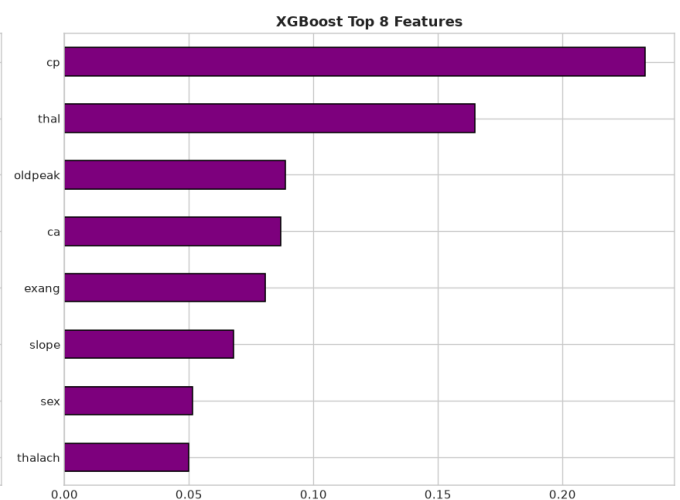
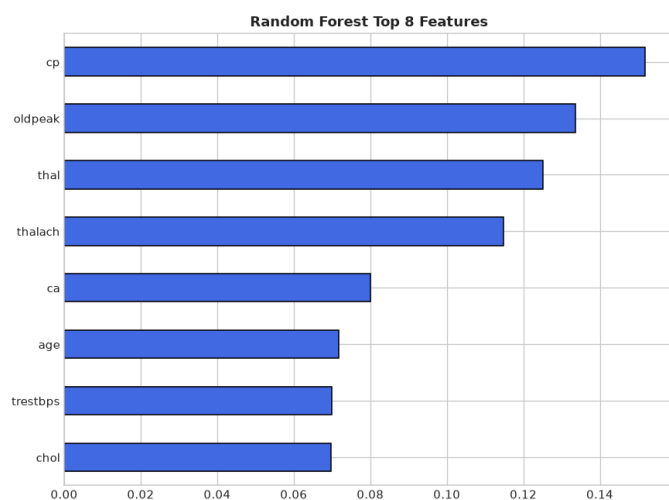


```
In [16]: # ---- Feature importance comparison: Random Forest vs XGBoost ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Random Forest: mean impurity decrease per feature (top 8)
rf_imp = pd.Series(rfc.feature_importances_,
index=X_train_clf.columns).sort_values(ascending=False).head(8)
rf_imp.plot(kind='barh', ax=ax1, color='royalblue', edgecolor='k')
ax1.set_title("Random Forest Top 8 Features", fontweight='bold')
ax1.invert_yaxis()

# XGBoost: gain-based feature importance (top 8)
xgb_imp = pd.Series(xgb_clf.feature_importances_,
index=X_train_clf.columns).sort_values(ascending=False).head(8)
xgb_imp.plot(kind='barh', ax=ax2, color='purple', edgecolor='k')
ax2.set_title("XGBoost Top 8 Features", fontweight='bold')
ax2.invert_yaxis()

plt.tight_layout()
plt.show()
```



### 3. Summary & Practical Guidelines

- **Random Forest (RFR / RFC):** Robust out-of-the-box baseline with minimal hyperparameter sensitivity, virtually immune to overfitting as trees are added.
- **AdaBoost:** Fast and clean for low-noise datasets, but can be sensitive to outliers due to exponential sample weight amplification.
- **XGBoost:** State-of-the-art tabular speed and accuracy via 2nd-order Taylor expansions and custom regularizers.
- **CatBoost:** Excels especially on tabular datasets containing non-numeric/categorical features, avoiding target leakage and extensive manual preprocessing.

# 05. Multilayer Perceptron (MLP): Scikit-Learn and PyTorch

## Algorithm Covered:

- **Multilayer Perceptron (Feedforward Deep Artificial Neural Network)**

## Implementations:

1. **Scikit-Learn MLPClassifier**: High-level production API with early stopping and loss curves
2. **PyTorch Custom Deep MLP**: Modular neural network with Linear layers, Batch Normalization, Dropout regularization, ReLU activations, CrossEntropyLoss, and Adam optimizer

**Dataset:** MNIST Handwritten Digits Dataset (1,797 samples, 8x8 pixel images, 64 features, 10 classes)

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + preprocessing/metrics ----
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Scikit-Learn MLP implementation ----
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ---- PyTorch for the custom deep MLP ----
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix seeds for reproducible training
torch.manual_seed(42)
np.random.seed(42)
print(f"PyTorch version: {torch.__version__}")
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # use GPU if available
print(f"Computation Device: {device}")
```

PyTorch version: 2.14.0+cpu

Computation Device: cpu

## 1. Dataset Exploration and Visualization

The Digits dataset comprises 1,797 normalized  $8 \times 8$  grayscale images of numerical digits (0 through 9).

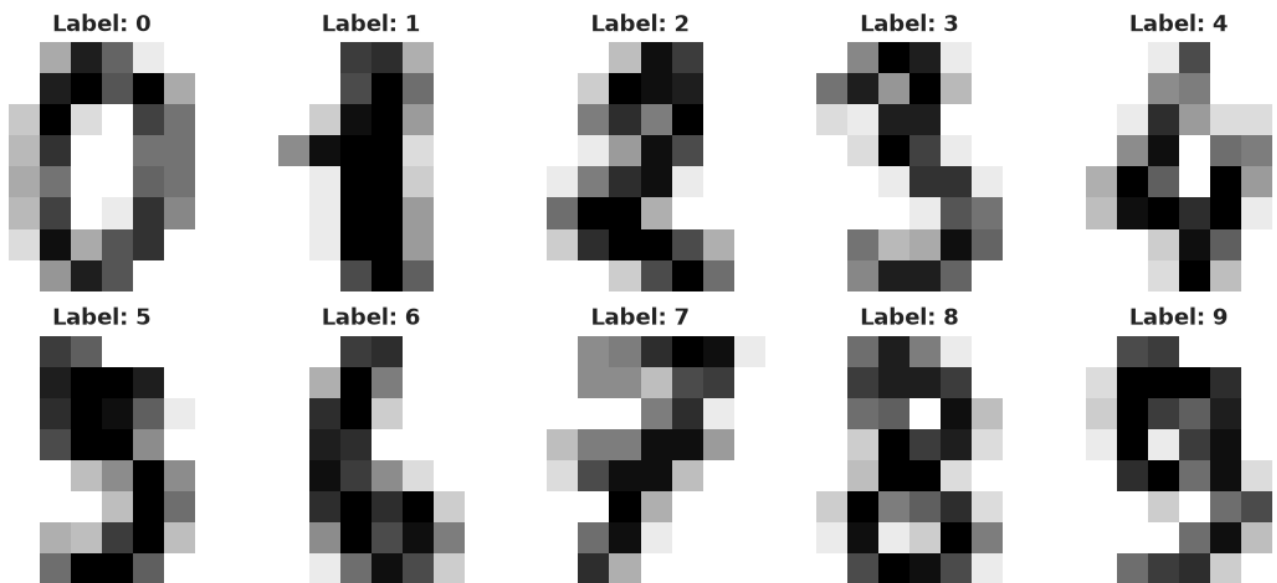
```
In [2]: # ---- Load the handwritten digits dataset (8x8 grayscale images) ----
digits = load_digits()
X_raw = digits.data      # flattened images: 1797 samples x 64 pixel features
y = digits.target        # digit class 0..9

print(f"Dataset Shape: {X_raw.shape}, Target Classes: {np.unique(y)}")

# ---- Visualize the first 10 images to understand the input ----
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='binary')      # reshape flat vector back to 8x8
    ax.set_title(f"Label: {digits.target[i]}", fontsize=12, fontweight='bold')
    ax.axis('off')
plt.suptitle("Sample Digits from Dataset", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Dataset Shape: (1797, 64), Target Classes: [0 1 2 3 4 5 6 7 8 9]

**Sample Digits from Dataset**



```
In [3]: # ---- Train / Validation / Test split (70% / 15% / 15%) ----
# First cut off the test set...
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X_raw, y, test_size=0.15, random_state=42, stratify=y
)
# ...then split the remainder into train and validation
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.1765, random_state=42, stratify=y_train_val
)

# ---- Feature scaling: fit on training data only (avoid validation/test Leakage) ----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print(f"Training samples: {X_train_scaled.shape[0]}")
print(f"Validation samples: {X_val_scaled.shape[0]}")
print(f"Test samples: {X_test_scaled.shape[0]}")
```

Training samples: 1257  
Validation samples: 270  
Test samples: 270

## 2. Part 1: Scikit-Learn MLPClassifier

### Mathematical Formulation

A Multilayer Perceptron maps input vector  $x \in \mathbb{R}^d$  through  $L$  hidden layers:

$$\begin{aligned}a^{(1)} &= \sigma(W^{(1)}x + b^{(1)}) \\a^{(l)} &= \sigma(W^{(l)}a^{(l-1)} + b^{(l)}), \quad l = 2, \dots, L \\ \hat{y} &= \text{softmax}(W^{(L+1)}a^{(L)} + b^{(L+1)})\end{aligned}$$

Weights are optimized by minimizing categorical Cross-Entropy Loss:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \ln \hat{y}_{ik} + \frac{\alpha}{2} \sum_l \|W^{(l)}\|_F^2$$

via backpropagation (chain rule of calculus) with Adam or SGD.

### Step-by-Step Algorithm (Multilayer Perceptron)

1. **Initialize** weights  $W$  and biases  $b$  for every layer (random small values).
2. **Forward pass:** for layer  $l$ , compute  $a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$ ; the last layer produces class logits.
3. **Loss:** convert logits to probabilities with softmax and compute cross-entropy (plus L2 penalty).
4. **Backward pass:** use the chain rule (backpropagation) to compute the gradient of the loss w.r.t. every weight.
5. **Update:** adjust weights with an optimizer (Adam):  $W \leftarrow W - \eta \cdot (\text{gradient} + \text{regularization term})$ .
6. **Repeat** steps 2–5 for many epochs over mini-batches until the loss converges.
7. **Predict:** take argmax of the output probabilities for each test sample.

### Scikit-Learn Training Process Used Here

1. Build a  $64 \rightarrow 128 \rightarrow 64 \rightarrow 10$  network with ReLU activations.
2. Optimize with Adam and L2 regularization ( `alpha` ).
3. Use early stopping: monitor a validation split and stop when it stops improving.
4. Record the loss curve and evaluate once on the untouched test set.

```
In [4]: # ---- Scikit-Learn MLPClassifier: 64 -> 128 -> 64 -> 10 ----
mlp_sklearn = MLPClassifier(
    hidden_layer_sizes=(128, 64), # two hidden layers
    activation='relu',
    solver='adam',                # adaptive gradient optimizer
    alpha=0.001,                  # L2 regularization strength
    batch_size=64,
    learning_rate_init=0.005,
    max_iter=150,
    early_stopping=True,          # stop when validation score stops improving
    validation_fraction=0.15,    # internal validation split for early stopping
    random_state=42
)

# Train (forward pass + backpropagation happen inside .fit)
mlp_sklearn.fit(X_train_scaled, y_train)
```

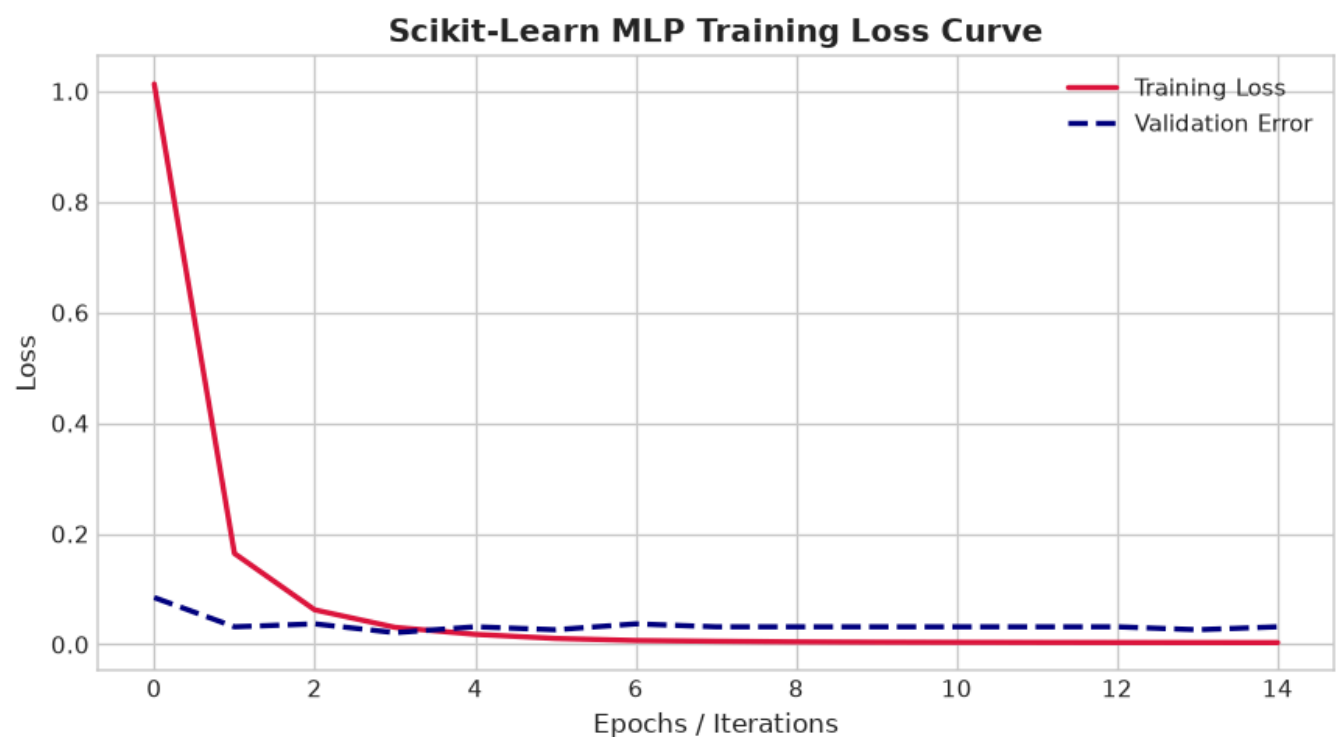
```
# Evaluate on the untouched test set
y_pred_sk = mlp_sklearn.predict(X_test_scaled)
acc_sk = accuracy_score(y_test, y_pred_sk)

print(f"Scikit-Learn MLP Test Accuracy: {acc_sk:.4f}")
print(f"Converged after {mlp_sklearn.n_iter_} iterations")

# ---- Plot the training loss curve recorded by scikit-learn ----
plt.figure(figsize=(8, 4.5))
plt.plot(mlp_sklearn.loss_curve_, color='crimson', lw=2.2, label='Training Loss')
if hasattr(mlp_sklearn, 'validation_scores_'):
    # validation_scores_ stores accuracy; convert to error for a comparable scale
    plt.plot([1 - s for s in mlp_sklearn.validation_scores_], color='navy', lw=2.2,
             linestyle='--', label='Validation Error')
plt.title("Scikit-Learn MLP Training Loss Curve", fontsize=13, fontweight='bold')
plt.xlabel("Epochs / Iterations", fontsize=11)
plt.ylabel("Loss", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()
```

Scikit-Learn MLP Test Accuracy: 0.9630

Converged after 15 iterations



### 3. Part 2: Custom PyTorch Deep MLP

Here we build a production-grade PyTorch `nn.Module` featuring:

- **Input Dimension:** 64 (8x8 pixel features)
- **Hidden Layer 1:** 128 units + Batch Normalization + ReLU + Dropout (0.25)
- **Hidden Layer 2:** 64 units + Batch Normalization + ReLU + Dropout (0.20)
- **Output Layer:** 10 logits (corresponding to classes 0 through 9)
- **Loss Function:** `nn.CrossEntropyLoss()`
- **Optimizer:** `optim.Adam(lr=0.003, weight_decay=1e-4)`

## PyTorch Training Process (explicit training loop)

1. **Data:** wrap scaled features/labels in Tensors and DataLoaders (mini-batches + shuffling).
2. **Architecture:** Linear → BatchNorm → ReLU → Dropout, twice, then a 10-unit output layer.
3. **Configure:** CrossEntropyLoss + Adam (weight decay) + ReduceLROnPlateau scheduler.
4. **For each epoch:** forward pass, compute loss, backpropagate, update weights.
5. **Monitor:** evaluate on the validation set after every epoch (no gradient updates).
6. **After training:** evaluate once on the test set, then inspect the confusion matrix and sample predictions.

```
In [5]: # ---- Convert scaled NumPy arrays into PyTorch tensors + DataLoaders ----
train_dataset = TensorDataset(torch.tensor(X_train_scaled, dtype=torch.float32),
                               torch.tensor(y_train, dtype=torch.long))
val_dataset   = TensorDataset(torch.tensor(X_val_scaled, dtype=torch.float32),
                               torch.tensor(y_val, dtype=torch.long))
test_dataset  = TensorDataset(torch.tensor(X_test_scaled, dtype=torch.float32),
                               torch.tensor(y_test, dtype=torch.long))

# DataLoaders handle mini-batching and shuffling during training
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_dataset, batch_size=64, shuffle=False)
test_loader  = DataLoader(test_dataset, batch_size=64, shuffle=False)

class DeepMLP(nn.Module):
    """Fully-connected classifier: 64 -> 128 -> 64 -> 10 with BatchNorm + Dropout."""
    def __init__(self, in_features=64, num_classes=10):
        super(DeepMLP, self).__init__()
        self.net = nn.Sequential(
            # Hidden block 1
            nn.Linear(in_features, 128),
            nn.BatchNorm1d(128),      # stabilizes activations, speeds up training
            nn.ReLU(),               # non-linear activation
            nn.Dropout(0.25),        # regularization: randomly drop 25% of neurons

            # Hidden block 2
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.20),

            # Output layer: 10 raw logits (one per digit class)
            nn.Linear(64, num_classes)
        )

    def forward(self, x):
        return self.net(x)

model = DeepMLP().to(device)  # move all parameters to CPU/GPU
print(model)
```

```

DeepMLP(
  (net): Sequential(
    (0): Linear(in_features=64, out_features=128, bias=True)
    (1): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.25, inplace=False)
    (4): Linear(in_features=128, out_features=64, bias=True)
    (5): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (6): ReLU()
    (7): Dropout(p=0.2, inplace=False)
    (8): Linear(in_features=64, out_features=10, bias=True)
  )
)

```

```

In [6]: # ---- Training configuration ----
criterion = nn.CrossEntropyLoss() # softmax + NLL
loss
optimizer = optim.Adam(model.parameters(), lr=0.003, weight_decay=1e-4) # weight decay = L2
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=5)

epochs = 50
history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

for epoch in range(epochs):
    # ----- Training phase -----
    model.train() # enable Dropout/BatchNorm training behavior
    running_loss, correct, total = 0.0, 0, 0
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad() # reset gradients from the previous step
        outputs = model(inputs) # forward pass
        loss = criterion(outputs, labels) # compute cross-entropy
        loss.backward() # backpropagation (compute gradients)
        optimizer.step() # update weights

        # Accumulate metrics (loss is averaged over the batch, so multiply by batch size)
        running_loss += loss.item() * inputs.size(0)
        _, preds = torch.max(outputs, 1) # predicted class = highest logit
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    epoch_train_loss = running_loss / total
    epoch_train_acc = correct / total

    # ----- Validation phase -----
    model.eval() # disable Dropout, use running BatchNorm stats
    val_loss, val_correct, val_total = 0.0, 0, 0
    with torch.no_grad(): # no gradients needed for evaluation
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            val_loss += loss.item() * inputs.size(0)
            _, preds = torch.max(outputs, 1)
            val_correct += (preds == labels).sum().item()
            val_total += labels.size(0)

    epoch_val_loss = val_loss / val_total
    epoch_val_acc = val_correct / val_total
    scheduler.step(epoch_val_loss) # reduce LR if validation loss plateaus

```

```

# Record metrics for plotting
history['train_loss'].append(epoch_train_loss)
history['val_loss'].append(epoch_val_loss)
history['train_acc'].append(epoch_train_acc)
history['val_acc'].append(epoch_val_acc)

# Print a progress line every 10 epochs (and the first one)
if (epoch + 1) % 10 == 0 or epoch == 0:
    print(f"Epoch [{epoch+1:02d}/{epochs}] "
          f"Train Loss: {epoch_train_loss:.4f} | Train Acc: {epoch_train_acc:.4f} || "
          f"Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc:.4f}")

```

```

Epoch [01/50] Train Loss: 1.1992 | Train Acc: 0.7327 || Val Loss: 0.4583 | Val Acc: 0.9111
Epoch [10/50] Train Loss: 0.0718 | Train Acc: 0.9793 || Val Loss: 0.0717 | Val Acc: 0.9667
Epoch [20/50] Train Loss: 0.0392 | Train Acc: 0.9889 || Val Loss: 0.0410 | Val Acc: 0.9852
Epoch [30/50] Train Loss: 0.0186 | Train Acc: 0.9936 || Val Loss: 0.0397 | Val Acc: 0.9852
Epoch [40/50] Train Loss: 0.0321 | Train Acc: 0.9905 || Val Loss: 0.0409 | Val Acc: 0.9815
Epoch [50/50] Train Loss: 0.0117 | Train Acc: 0.9984 || Val Loss: 0.0317 | Val Acc: 0.9889

```

```

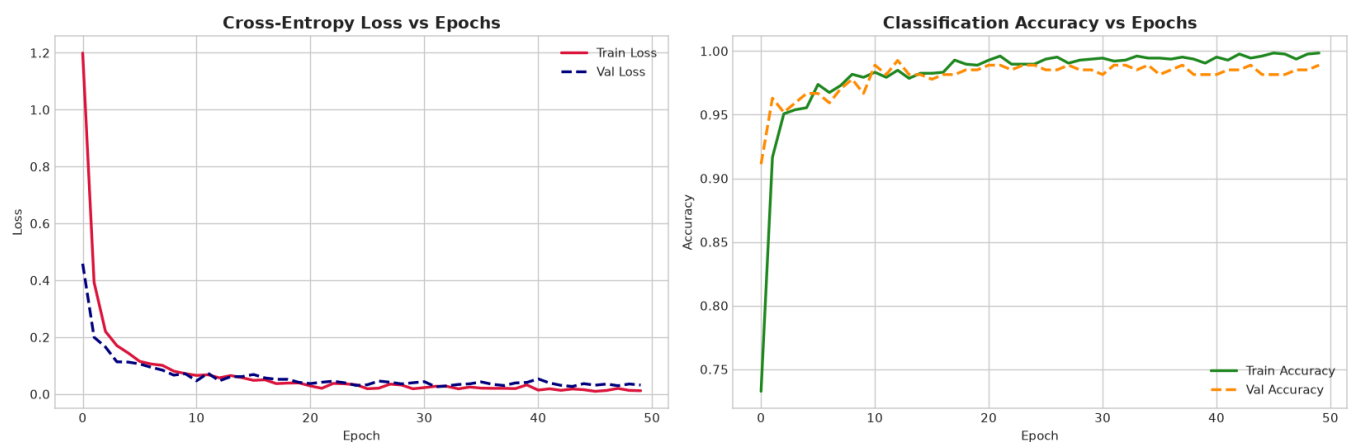
In [7]: # ---- Plot the Learning curves ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Loss curves: train vs validation
ax1.plot(history['train_loss'], label='Train Loss', color='crimson', lw=2.2)
ax1.plot(history['val_loss'], label='Val Loss', color='navy', lw=2.2, linestyle='--')
ax1.set_title("Cross-Entropy Loss vs Epochs", fontsize=13, fontweight='bold')
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")
ax1.legend()

# Accuracy curves: train vs validation
ax2.plot(history['train_acc'], label='Train Accuracy', color='forestgreen', lw=2.2)
ax2.plot(history['val_acc'], label='Val Accuracy', color='darkorange', lw=2.2, linestyle='--')
ax2.set_title("Classification Accuracy vs Epochs", fontsize=13, fontweight='bold')
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Accuracy")
ax2.legend()

plt.tight_layout()
plt.show()

```



```

In [8]: # ---- Final evaluation on the held-out test set ----
model.eval()
test_preds = []
test_targets = []

with torch.no_grad():
    for inputs, labels in test_loader:

```

```

inputs = inputs.to(device)
outputs = model(inputs)
_, preds = torch.max(outputs, 1)          # predicted digit
test_preds.extend(preds.cpu().numpy())    # move back to CPU for metrics
test_targets.extend(labels.numpy())

acc_pytorch = accuracy_score(test_targets, test_preds)
print(f"PyTorch Deep MLP Test Accuracy: {acc_pytorch:.4f}\n")
print("Detailed Classification Report:")
# Per-class precision/recall/F1 for all 10 digits
print(classification_report(test_targets, test_preds, digits=4))

```

PyTorch Deep MLP Test Accuracy: 0.9889

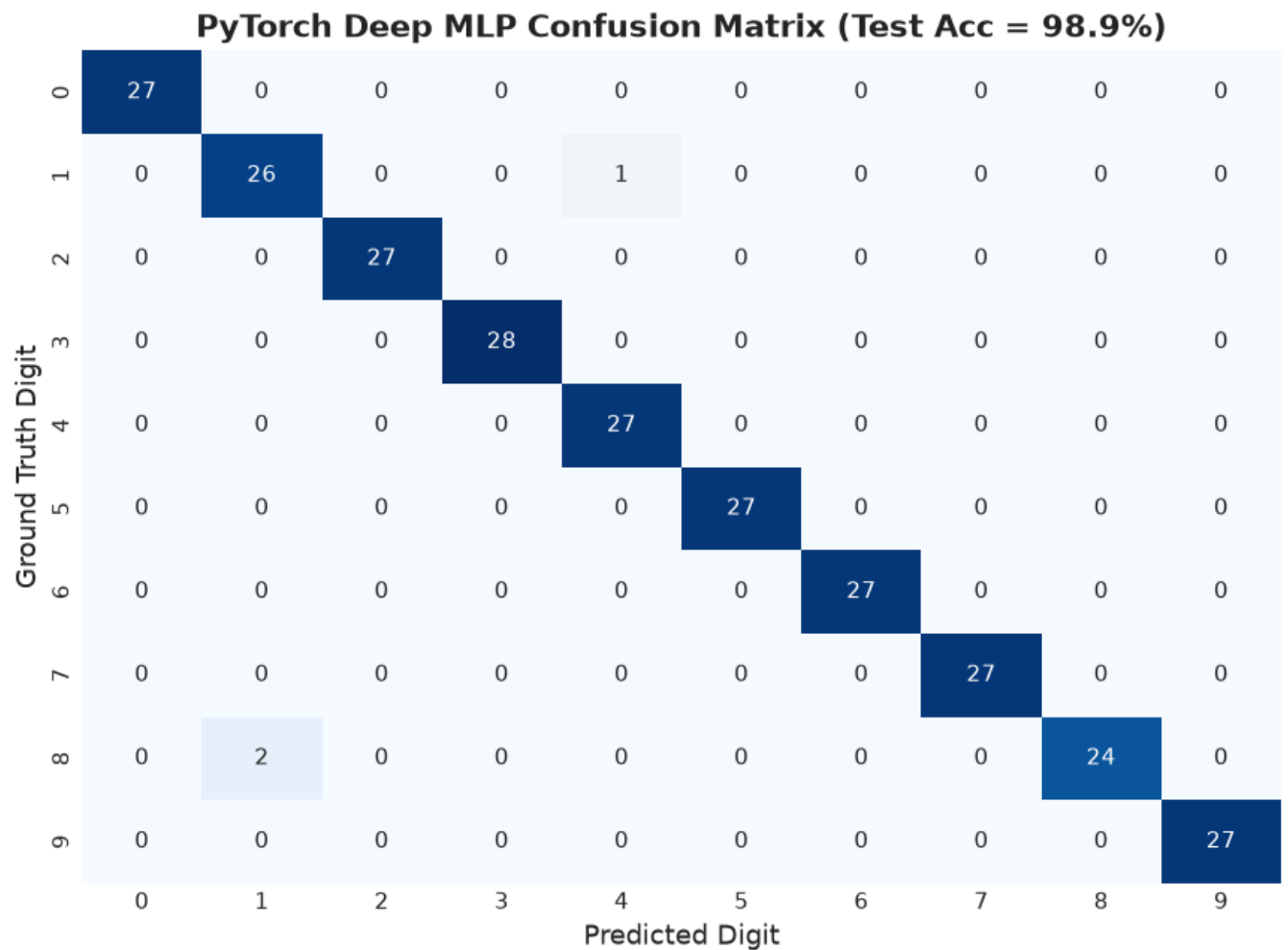
Detailed Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.0000    | 1.0000 | 1.0000   | 27      |
| 1            | 0.9286    | 0.9630 | 0.9455   | 27      |
| 2            | 1.0000    | 1.0000 | 1.0000   | 27      |
| 3            | 1.0000    | 1.0000 | 1.0000   | 28      |
| 4            | 0.9643    | 1.0000 | 0.9818   | 27      |
| 5            | 1.0000    | 1.0000 | 1.0000   | 27      |
| 6            | 1.0000    | 1.0000 | 1.0000   | 27      |
| 7            | 1.0000    | 1.0000 | 1.0000   | 27      |
| 8            | 1.0000    | 0.9231 | 0.9600   | 26      |
| 9            | 1.0000    | 1.0000 | 1.0000   | 27      |
|              |           |        |          |         |
| accuracy     |           |        | 0.9889   | 270     |
| macro avg    | 0.9893    | 0.9886 | 0.9887   | 270     |
| weighted avg | 0.9893    | 0.9889 | 0.9889   | 270     |

```

In [9]: # ---- Confusion matrix heatmap ----
# Rows = true digit, columns = predicted digit; diagonal = correct predictions.
cm = confusion_matrix(test_targets, test_preds)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=range(10), yticklabels=range(10))
plt.title(f"PyTorch Deep MLP Confusion Matrix (Test Acc = {acc_pytorch*100:.1f}%)",
          fontsize=14, fontweight='bold')
plt.xlabel("Predicted Digit", fontsize=12)
plt.ylabel("Ground Truth Digit", fontsize=12)
plt.tight_layout()
plt.show()

```



## 4. Visualizing Model Predictions

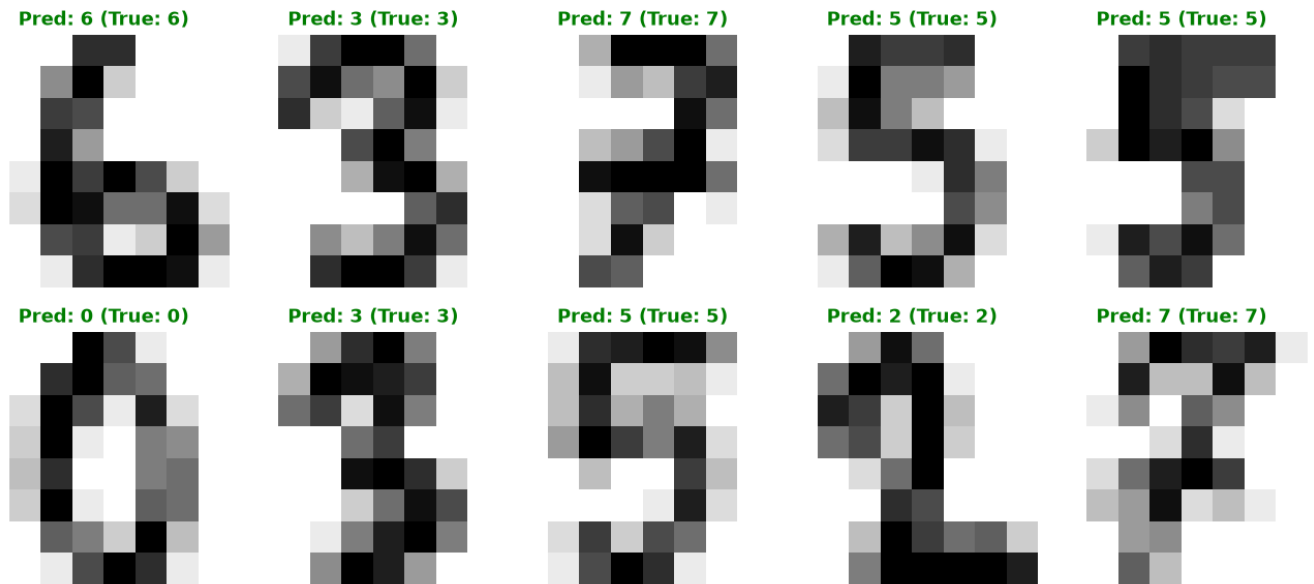
Let's inspect sample test images alongside their predicted class probabilities.

```
In [10]: # ---- Visualize sample predictions (green = correct, red = wrong) ----
fig, axes = plt.subplots(2, 5, figsize=(12, 6))
for i, ax in enumerate(axes.flat):
    # Undo standardization to recover the original 0..16 pixel intensities
    img = scaler.inverse_transform(X_test_scaled[i].reshape(1, -1)).reshape(8, 8)
    true_label = test_targets[i]
    pred_label = test_preds[i]

    ax.imshow(img, cmap='binary')
    color = 'green' if true_label == pred_label else 'red'
    ax.set_title(f"Pred: {pred_label} (True: {true_label})", color=color, fontweight='bold')
    ax.axis('off')

plt.suptitle("PyTorch MLP Sample Test Predictions", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

PyTorch MLP Sample Test Predictions



## 5. Summary: Scikit-Learn vs PyTorch MLP

| Dimension                 | Scikit-Learn <code>MLPClassifier</code>           | PyTorch Custom <code>nn.Module</code>                |
|---------------------------|---------------------------------------------------|------------------------------------------------------|
| Setup Overhead            | Extremely minimal (single line <code>fit</code> ) | Explicit tensor/dataset/loader boilerplate           |
| Architectural Flexibility | Fixed forward feedforward structure               | Arbitrary connections, skip connections, custom loss |
| Hardware Acceleration     | CPU only                                          | GPU / CUDA / MPS acceleration                        |
| Regularization Options    | $L_2$ weight penalty ( <code>alpha</code> )       | Dropout, BatchNorm, LayerNorm, custom regularizers   |
| Production Best Use       | Rapid baseline tabular ML                         | Deep learning, vision, NLP, custom training loops    |

# 06. Recurrent Neural Networks (RNN): Vanilla RNN vs LSTM in PyTorch

## Algorithms Covered:

1. **Vanilla Recurrent Neural Network (RNN)** (Elman RNN with BPTT)
2. **Long Short-Term Memory (LSTM)** (Gated Architecture for Vanishing Gradients)

**Dataset:** Monthly Airline Passengers Time Series ( `data/airline_passengers.csv` )

- Modeling seasonal cycles and non-linear upward trends
- Temporal sequence construction with sliding lookback windows
- Multi-step rolling sequence forecasting

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scaling + regression metrics ----
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import root_mean_squared_error, mean_absolute_error

# ---- PyTorch for the recurrent models ----
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"PyTorch version: {torch.__version__} on {device}")
```

PyTorch version: 2.14.0+cpu on cpu

## 1. Dataset Loading and Sliding Window Preprocessing

We load the historical **Airline Passengers** time series dataset.

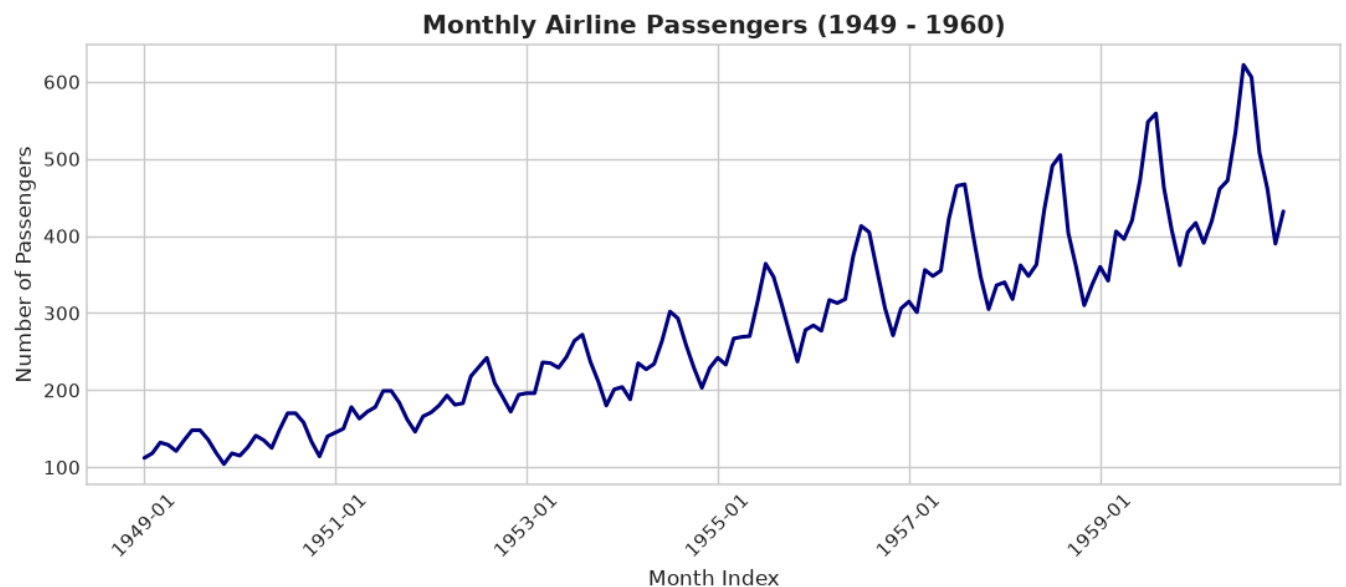
```
In [2]: # ---- Load the monthly airline passengers time series ----
df_air = pd.read_csv('../data/airline_passengers.csv')
print("Dataset Head:")
display(df_air.head())

# Extract the target series as float32 (shape: N x 1)
passengers = df_air['Passengers'].values.astype(np.float32).reshape(-1, 1)
```

```
# ---- Plot the raw series (trend + yearly seasonality are clearly visible) ----
plt.figure(figsize=(10, 4.5))
plt.plot(df_air['Month'], df_air['Passengers'], color='navy', lw=2)
plt.title("Monthly Airline Passengers (1949 - 1960)", fontsize=13, fontweight='bold')
plt.xlabel("Month Index", fontsize=11)
plt.ylabel("Number of Passengers", fontsize=11)
plt.xticks(range(0, len(df_air), 24), df_air['Month'].iloc[::24], rotation=45) # Label every 2
years
plt.tight_layout()
plt.show()
```

Dataset Head:

|   | Month   | Passengers |
|---|---------|------------|
| 0 | 1949-01 | 112        |
| 1 | 1949-02 | 118        |
| 2 | 1949-03 | 132        |
| 3 | 1949-04 | 129        |
| 4 | 1949-05 | 121        |



```
In [3]: # ---- Normalize to [0, 1] (RNNs train much better on small inputs) ----
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(passengers)

# ---- Function to build supervised sliding-window sequences ----
# For each time step t: X = values [t-seq_len ... t-1], y = value at t
def create_sequences(data, seq_length=12):
    xs, ys = [], []
    for i in range(len(data) - seq_length):
        x = data[i:(i + seq_length)]
        y = data[i + seq_length]
        xs.append(x)
        ys.append(y)
    return np.array(xs), np.array(ys)

SEQ_LENGTH = 12 # 12-month lookahead = one full annual seasonal cycle
X_seq, y_seq = create_sequences(scaled_data, seq_length=SEQ_LENGTH)

# ---- Temporal split: 80% train, 20% test WITHOUT shuffling (time order matters) ----
train_size = int(len(X_seq) * 0.80)
```

```

X_train, X_test = X_seq[:train_size], X_seq[train_size:]
y_train, y_test = y_seq[:train_size], y_seq[train_size:]

print(f"Total sequences created: {len(X_seq)}")
print(f"Training sequences:      {X_train.shape[0]} (Shape: {X_train.shape})")
print(f"Testing sequences:       {X_test.shape[0]} (Shape: {X_test.shape})")

# ---- DataLoader for mini-batch training (shuffle only the training set) ----
train_loader = DataLoader(
    TensorDataset(torch.tensor(X_train, dtype=torch.float32), torch.tensor(y_train,
dtype=torch.float32)),
    batch_size=16,
    shuffle=True
)

```

```

Total sequences created: 132
Training sequences:      105 (Shape: (105, 12, 1))
Testing sequences:       27 (Shape: (27, 12, 1))

```

## 2. Model Architectures

### 2.1 Vanilla Recurrent Neural Network (RNN)

Hidden state transition:

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

Linear output projection:

$$\hat{y}_t = W_{ho}h_t + b_o$$

### 2.2 Long Short-Term Memory (LSTM)

Gated state equations:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (\text{Forget Gate})$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (\text{Input Gate})$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (\text{Candidate Cell})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Cell State})$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (\text{Output Gate})$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{Hidden State})$$

### Step-by-Step Algorithm (Vanilla RNN)

1. Start with an initial hidden state  $h_0$  (usually zeros).
2. For each time step  $t$ :  $h_t = \tanh(W_{ih} x_t + b_{ih} + W_{hh} h_{t-1} + b_{hh})$ .
3. Keep only the final hidden state  $h_T$  and map it to the forecast:  $\hat{y} = W_{ho} h_T + b_o$ .
4. Train with backpropagation through time (BPTT): gradients flow backwards through every time step.
5. Limitation: repeated multiplication makes gradients vanish/explode on long sequences.

## Step-by-Step Algorithm (LSTM)

1. At each step compute the **forget gate**  $f_t$  (what to discard from the cell state).
2. Compute the **input gate**  $i_t$  and candidate memory  $\tilde{c}_t$  (what new information to store).
3. Update the cell state:  $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$  (additive "conveyor belt").
4. Compute the **output gate**  $o_t$  and hidden state:  $h_t = o_t \odot \tanh(c_t)$ .
5. Predict from the last hidden state as in the vanilla RNN.
6. Train with BPTT - the additive cell state lets gradients flow across long horizons.

```
In [4]: class VanillaRNN(nn.Module):
        """Elman RNN: 2 stacked recurrent layers + a linear output layer."""
        def __init__(self, input_size=1, hidden_size=64, num_layers=2):
            super(VanillaRNN, self).__init__()
            self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
            self.fc = nn.Linear(hidden_size, 1)  # map final hidden state to a scalar forecast

        def forward(self, x):
            out, _ = self.rnn(x)  # out: (batch, seq_len, hidden)
            out = self.fc(out[:, -1, :])  # use ONLY the last time step's hidden state
            return out

class LSTMModel(nn.Module):
        """LSTM: same interface, but gated cells preserve long-term information."""
        def __init__(self, input_size=1, hidden_size=64, num_layers=2):
            super(LSTMModel, self).__init__()
            self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
            self.fc = nn.Linear(hidden_size, 1)

        def forward(self, x):
            out, _ = self.lstm(x)
            out = self.fc(out[:, -1, :])
            return out

# Instantiate both models on the selected device so they can be compared fairly
rnn_model = VanillaRNN().to(device)
lstm_model = LSTMModel().to(device)
print("Models instantiated successfully!")
```

Models instantiated successfully!

## 3. Model Training: Vanilla RNN vs LSTM

### Training Process (shared)

1. Normalize the series and cut it into sliding windows of 12 months (one seasonal cycle).
2. Split chronologically: first 80% for training, last 20% for testing.
3. For every epoch: forward pass through the recurrent layers, MSE loss, BPTT, Adam update.
4. Track the loss per epoch to compare convergence of RNN vs LSTM.

```
In [5]: # ---- Generic training function shared by both architectures ----
def train_model(model, loader, epochs=120, lr=0.005):
    criterion = nn.MSELoss()  # regression -> mean squared error
    optimizer = optim.Adam(model.parameters(), lr=lr)  # Adam optimizer
    losses = []

    for epoch in range(epochs):
        model.train()
```

```

total_loss = 0.0
for x_b, y_b in loader:
    x_b, y_b = x_b.to(device), y_b.to(device)

    optimizer.zero_grad()           # clear previous gradients
    pred = model(x_b)               # forward pass through the recurrent net
    loss = criterion(pred, y_b)     # MSE between forecast and truth
    loss.backward()                 # backpropagation through time (BPTT)
    optimizer.step()                # update weights

    total_loss += loss.item() * len(x_b)  # de-average the batch loss

epoch_loss = total_loss / len(loader.dataset)
losses.append(epoch_loss)          # track loss per epoch for plotting

return losses

# ---- Train both architectures with identical hyperparameters ----
print("Training Vanilla RNN...")
rnn_losses = train_model(rnn_model, train_loader, epochs=120, lr=0.005)

print("Training LSTM Model...")
lstm_losses = train_model(lstm_model, train_loader, epochs=120, lr=0.005)
print("Training completed!")

```

Training Vanilla RNN...

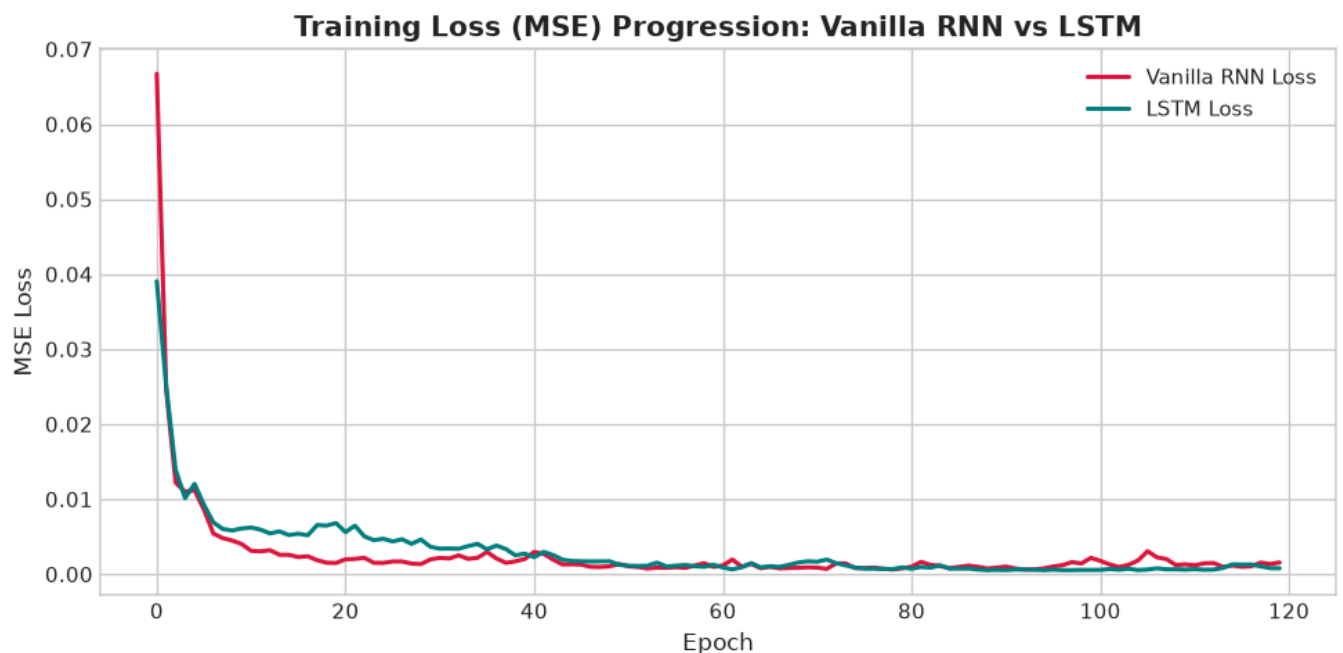
Training LSTM Model...

Training completed!

```

In [6]: # ---- Compare the two learning curves ----
plt.figure(figsize=(9, 4.5))
plt.plot(rnn_losses, label='Vanilla RNN Loss', color='crimson', lw=2)
plt.plot(lstm_losses, label='LSTM Loss', color='teal', lw=2)
plt.title("Training Loss (MSE) Progression: Vanilla RNN vs LSTM", fontsize=13,
fontweight='bold')
plt.xlabel("Epoch", fontsize=11)
plt.ylabel("MSE Loss", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()

```



## 4. Evaluation and Sequence Forecasting on Test Horizon

### Evaluation & Forecasting Process

1. Switch both models to evaluation mode and disable gradient computation.
2. Predict all test windows, then invert the MinMax scaling to recover real passenger counts.
3. Compute RMSE and MAE per model in the original unit.
4. Plot the forecast trajectories against the ground truth to inspect trend/seasonality fit.

```
In [7]: # ---- Evaluate both models on the test horizon ----
rnn_model.eval()
lstm_model.eval()

with torch.no_grad(): # inference only - no gradients
    x_test_t = torch.tensor(X_test, dtype=torch.float32).to(device)
    pred_rnn_scaled = rnn_model(x_test_t).cpu().numpy() # normalized predictions
    pred_lstm_scaled = lstm_model(x_test_t).cpu().numpy()

# ---- Invert scaling to get real passenger counts ----
y_test_true = scaler.inverse_transform(y_test)
pred_rnn = scaler.inverse_transform(pred_rnn_scaled)
pred_lstm = scaler.inverse_transform(pred_lstm_scaled)

# ---- Compute error metrics in the original unit (passengers) ----
rmse_rnn = root_mean_squared_error(y_test_true, pred_rnn)
mae_rnn = mean_absolute_error(y_test_true, pred_rnn)

rmse_lstm = root_mean_squared_error(y_test_true, pred_lstm)
mae_lstm = mean_absolute_error(y_test_true, pred_lstm)

# ---- Summary table (Lower errors are better) ----
eval_df = pd.DataFrame([
    {"Model": "Vanilla RNN", "RMSE (Passengers)": rmse_rnn, "MAE (Passengers)": mae_rnn},
    {"Model": "LSTM Network", "RMSE (Passengers)": rmse_lstm, "MAE (Passengers)": mae_lstm}
]).set_index("Model")

print("Time Series Forecasting Performance Comparison:")
display(eval_df.style.highlight_min(subset=['RMSE (Passengers)', 'MAE (Passengers)'],
color='lightgreen'))
```

Time Series Forecasting Performance Comparison:

|              | RMSE (Passengers) | MAE (Passengers) |
|--------------|-------------------|------------------|
| Model        |                   |                  |
| Vanilla RNN  | 70.827942         | 63.158745        |
| LSTM Network | 42.171574         | 34.881374        |

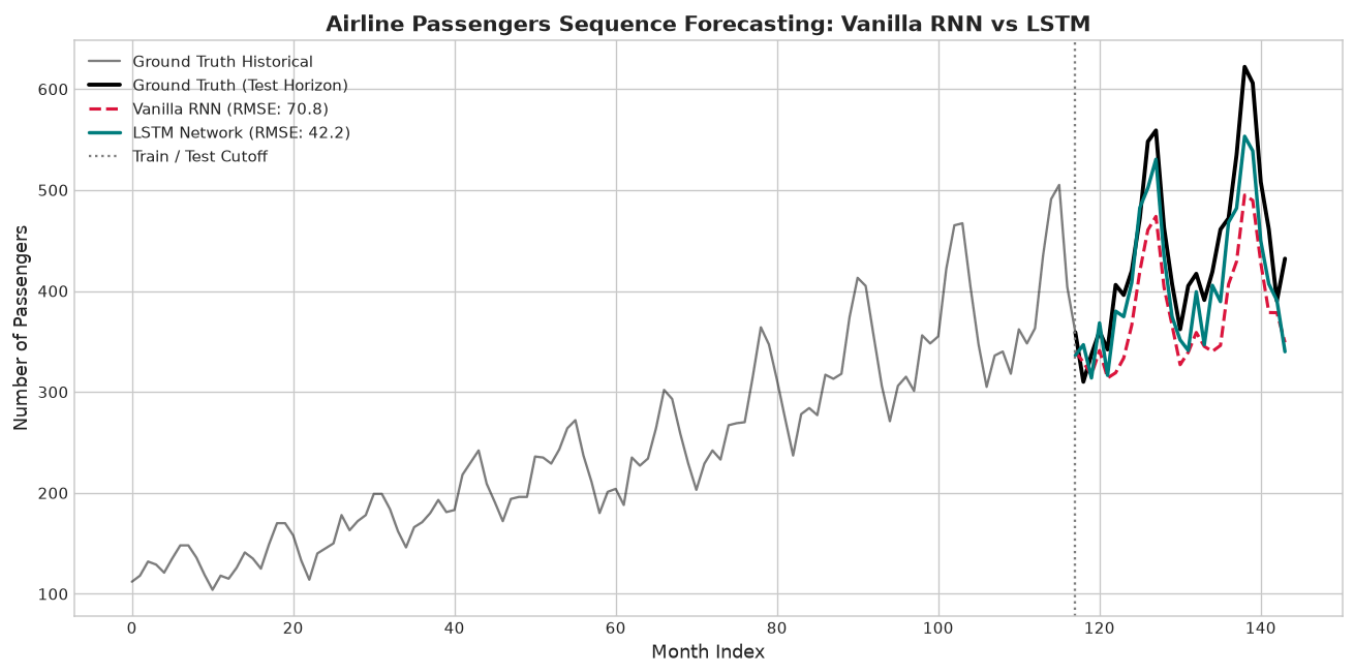
```
In [8]: # ---- Plot ground truth vs. both forecast trajectories ----
# Map each test target back to its true position on the month axis:
# sequence i predicts month (i + SEQ_LENGTH)
test_indices = range(train_size + SEQ_LENGTH, len(passengers))

plt.figure(figsize=(12, 6))
# Full historical series in the background
plt.plot(range(len(passengers)), passengers, label='Ground Truth Historical', color='black',
alpha=0.5, lw=1.5)
```

```
# Test-period ground truth
plt.plot(test_indices, y_test_true, label='Ground Truth (Test Horizon)', color='black', lw=2.5)

# Model forecasts
plt.plot(test_indices, pred_rnn, label=f'Vanilla RNN (RMSE: {rmse_rnn:.1f})', color='crimson',
lw=2, linestyle='--')
plt.plot(test_indices, pred_lstm, label=f'LSTM Network (RMSE: {rmse_lstm:.1f})', color='teal',
lw=2.2)

# Vertical line marking where the train period ends and test begins
plt.axvline(x=train_size + SEQ_LENGTH, color='gray', linestyle=':', label='Train / Test
Cutoff')
plt.title("Airline Passengers Sequence Forecasting: Vanilla RNN vs LSTM", fontsize=14,
fontweight='bold')
plt.xlabel("Month Index", fontsize=12)
plt.ylabel("Number of Passengers", fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```



## 5. Architectural Comparison & Takeaways

1. **Vanishing Gradients:** Vanilla RNNs suffer from vanishing gradients when backpropagating through longer sequences because repeated multiplications by  $W_{hh}$  decay exponentially if eigenvalues are  $< 1$ .
2. **Gated Cell States:** LSTMs maintain an additive conveyor belt ( $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ ), allowing error gradients to flow across arbitrary time horizons without exponential attenuation.
3. **Forecasting Fidelity:** On periodic and seasonal time series (like monthly airline travel), the LSTM accurately captures both the 12-month cyclical oscillation and the long-term upward trend.

# 07. Self-Organizing Maps (SOM): Kohonen Network with MiniSom

## Algorithm Covered:

- **Self-Organizing Map (Kohonen Neural Network)**

## Core Concepts:

- Unsupervised topological dimensionality reduction
- Competitive learning and Best Matching Unit (BMU) search
- Gaussian neighborhood decay and weight adaptation
- Unified Distance Matrix (**U-Matrix**) & Win Map visualization
- Model quality metrics: **Quantization Error** and **Topographic Error**

**Dataset:** UCI Wine Recognition Dataset (178 samples, 13 continuous chemical constituents, 3 wine cultivars)

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + scaling ----
from sklearn.datasets import load_wine
from sklearn.preprocessing import MinMaxScaler
# ---- MiniSom: Lightweight Kohonen Self-Organizing Map implementation ----
from minisom import MiniSom

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("MiniSom and libraries loaded successfully!")
```

MiniSom and libraries loaded successfully!

## 1. Dataset Loading and Feature Scaling

The Wine dataset features 13 chemical constituents (alcohol, malic acid, ash, flavanoids, color intensity, etc.) across 3 distinct wine cultivars.

```
In [2]: # ---- Load the Wine recognition dataset ----
wine = load_wine()
X_raw = wine.data           # 178 samples x 13 chemical features
y = wine.target             # 3 wine cultivars (only used for visualization - SOM is
                             unsupervised)
feature_names = wine.feature_names
target_names = wine.target_names
```

```
print(f"Samples: {X_raw.shape[0]}, Features: {X_raw.shape[1]}, Classes: {target_names.tolist()}")

# ---- MinMax scaling to [0, 1] ----
# SOM weight vectors live in the same range as the inputs, so scaling keeps learning stable.
scaler = MinMaxScaler()
X = scaler.fit_transform(X_raw)
```

Samples: 178, Features: 13, Classes: ['class\_0', 'class\_1', 'class\_2']

## 2. Mathematical Principles of the Kohonen SOM

A Self-Organizing Map arranges  $M \times N$  neurons on a regular 2D grid. Each neuron  $j$  has a weight vector  $w_j \in \mathbb{R}^d$ .

### Competitive Phase:

For input vector  $x$ , the **Best Matching Unit (BMU)**  $c$  is determined by Euclidean distance:

$$c = \arg \min_j \|x - w_j\|$$

### Cooperative & Adaptive Phase:

Neurons in the topological neighborhood of  $c$  are pulled toward  $x$ :

$$w_j(t+1) = w_j(t) + \alpha(t)h_{cj}(t)[x - w_j(t)]$$

where:

- $\alpha(t) = \alpha_0 \exp(-t/\tau_1)$  is the decaying learning rate.
- $h_{cj}(t) = \exp\left(-\frac{\|r_c - r_j\|^2}{2\sigma(t)^2}\right)$  is the Gaussian neighborhood kernel on grid coordinates  $r$ .
- $\sigma(t) = \sigma_0 \exp(-t/\tau_2)$  is the decaying neighborhood spread.

## Step-by-Step Algorithm (Kohonen Self-Organizing Map)

1. **Initialize** an  $M \times N$  grid of neurons, each with a random weight vector  $w_j$  of the same dimension as the input.
2. **Sample** a training input  $x$  (randomly, with replacement).
3. **Competitive phase:** find the Best Matching Unit (BMU)  $c = \operatorname{argmin}_j \|x - w_j\|$ .
4. **Cooperative phase:** compute the Gaussian neighborhood kernel  $h_{cj} = \exp(-\|r_c - r_j\|^2 / (2\sigma(t)^2))$ .
5. **Adaptive phase:** move the BMU and its neighbors toward  $x$ :  $w_j \leftarrow w_j + \alpha(t) h_{cj} (x - w_j)$ .
6. **Decay** the learning rate  $\alpha(t)$  and neighborhood radius  $\sigma(t)$  after each step.
7. **Repeat** steps 2-6 for many iterations until the map stabilizes.
8. **Output:** a 2D topology-preserving map; each sample is assigned to the neuron (BMU) closest to it.

```
In [3]: # ---- Configure the Kohonen grid ----
grid_x, grid_y = 12, 12 # 12x12 grid = 144 neurons (fewer than the 178 samples)
input_len = X.shape[1] # 13 chemical features per neuron weight vector

som = MiniSom(
    x=grid_x,
    y=grid_y,
```

```

input_len=input_len,
sigma=1.5,                # initial neighborhood radius
learning_rate=0.5,        # initial learning rate
neighborhood_function='gaussian', # smooth Gaussian neighborhood kernel
random_seed=42
)

# ---- Initialize weights with PCA for topologically faithful, faster convergence ----
som.pca_weights_init(X)
print("Initial Quantization Error:", som.quantization_error(X))

# ---- Train: 5000 random samples presented to the map ----
som.train_random(data=X, num_iteration=5000, verbose=False)
final_qe = som.quantization_error(X) # average distance from samples to their BMU
final_te = som.topographic_error(X)  # fraction of samples whose 2nd BMU is not a grid
neighbor

print(f"Training complete after 5,000 iterations!")
print(f" - Final Quantization Error (mean distance to BMU): {final_qe:.4f}")
print(f" - Final Topographic Error (proportion of foldings): {final_te:.4f}")

```

Initial Quantization Error: 0.4447391189659704

Training complete after 5,000 iterations!

- Final Quantization Error (mean distance to BMU): 0.1965
- Final Topographic Error (proportion of foldings): 0.0393

### 3. Visualizing the Unified Distance Matrix (U-Matrix)

The **U-Matrix** computes the average Euclidean distance between each neuron and its immediate grid neighbors.

- **Dark/Blue valleys:** High data density (clusters of similar samples).
- **Bright/Yellow ridges:** Topological boundaries and low-density barriers separating clusters.

#### How the U-Matrix Is Built

1. Train the SOM until the weight vectors organize.
2. For every neuron compute the average Euclidean distance to its immediate grid neighbors.
3. Render these averages as a heatmap: bright ridges separate clusters, dark valleys are cluster interiors.
4. Project each sample onto its BMU using the marker of its true cultivar to check unsupervised separation.

```

In [4]: # ---- Plot the U-Matrix (unified distance map) ----
# Each cell = average distance between a neuron and its immediate grid neighbors.
# Dark valleys = dense clusters; bright ridges = boundaries between clusters.
plt.figure(figsize=(10, 8))
plt.pcolor(som.distance_map().T, cmap='bone_r', alpha=0.9)
cbar = plt.colorbar()
cbar.set_label("Normalized Inter-Neuron Distance (U-Matrix)", fontsize=11)

# ---- Project every sample onto its Best Matching Unit (BMU) ----
markers = ['o', 's', '^']
colors = ['red', 'green', 'blue']

for idx, x_vec in enumerate(X):
    w = som.winner(x_vec) # grid coordinates of the best matching neuron for this sample
    plt.plot(

```

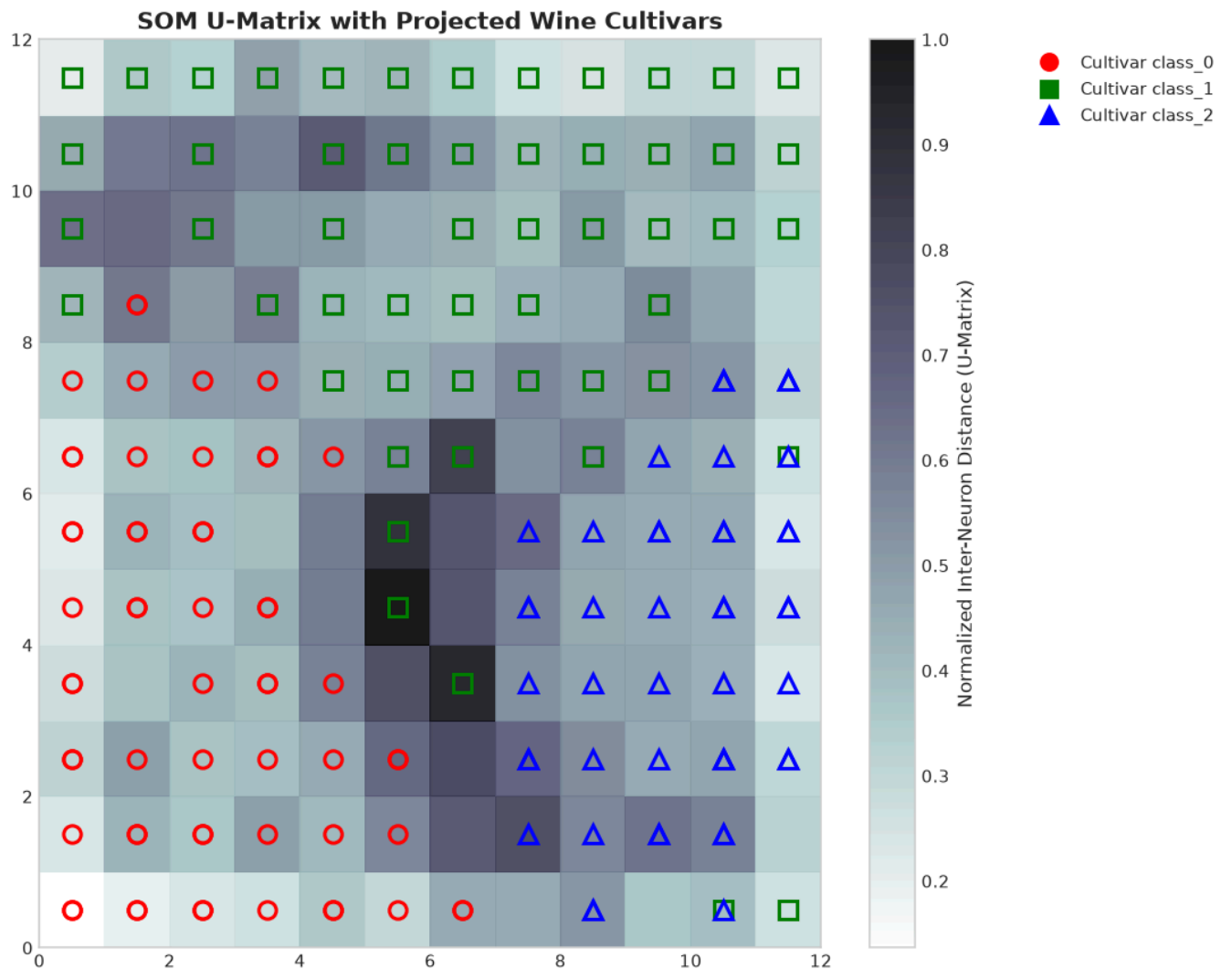
```

w[0] + 0.5,          # +0.5 centers the marker inside the pcolor cell
w[1] + 0.5,
markers[y[idx]],     # marker shape encodes the true cultivar
markerfacecolor='None',
markeredgewidth=2,
markersize=10,
markeredgecolor=colors[y[idx]],
)

# Build a legend for the three cultivars
for c_idx, c_name in enumerate(target_names):
    plt.plot([], [], marker=markers[c_idx], color=colors[c_idx], linestyle='None',
             label=f'Cultivar {c_name}', markersize=9, markeredgecolor='black')

plt.title("SOM U-Matrix with Projected Wine Cultivars", fontsize=14, fontweight='bold')
plt.legend(bbox_to_anchor=(1.25, 1), loc='upper left')
plt.tight_layout()
plt.show()

```



## 4. Win Map & Component Planes

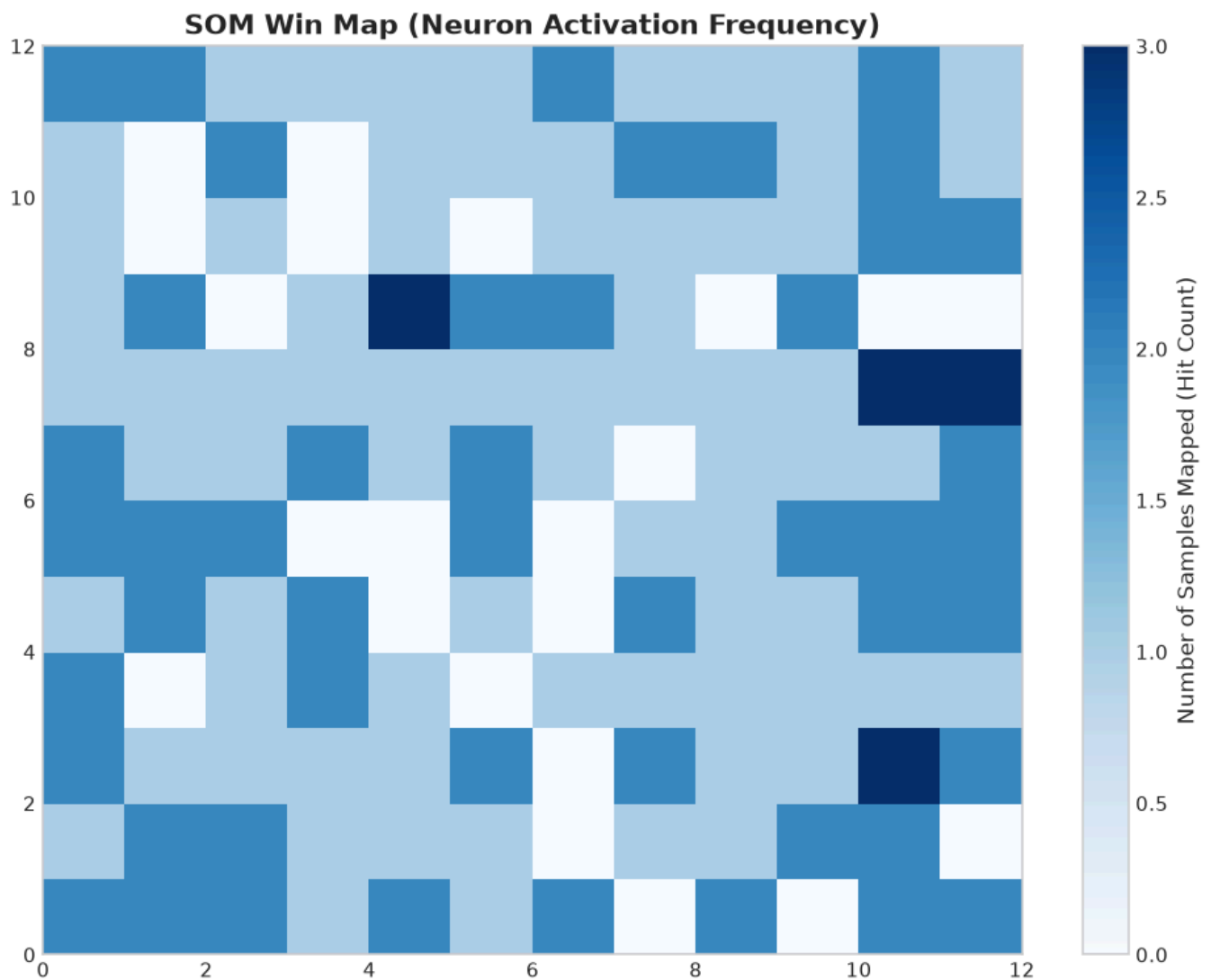
1. **Win Map:** Tracks the frequency with which each grid neuron acts as BMU for samples.
2. **Component Planes:** Visualizes the weight values of individual features across the 2D grid, revealing correlations.

## Quality Metrics & Additional Visualizations

1. **Quantization error:** average distance from samples to their BMU - lower means the map represents the data better.
2. **Topographic error:** fraction of samples whose two best neurons are not neighbors - lower means better topology preservation.
3. **Win map:** per-neuron hit counts reveal which regions of the map specialize in which samples.
4. **Component planes:** per-feature weight maps expose correlations between chemical variables.

```
In [5]: # ---- Win Map (hit histogram): how often each neuron is the BMU ----
frequencies = som.activation_response(X) # counts per grid cell

plt.figure(figsize=(9, 7))
plt.pcolor(frequencies.T, cmap='Blues')
cbar = plt.colorbar()
cbar.set_label("Number of Samples Mapped (Hit Count)", fontsize=11)
plt.title("SOM Win Map (Neuron Activation Frequency)", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



```
In [6]: # ---- Component planes: the Learned weight value of individual features on the grid ----
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
selected_features = [0, 6, 9, 12] # Alcohol, Flavanoids, Color intensity, Proline

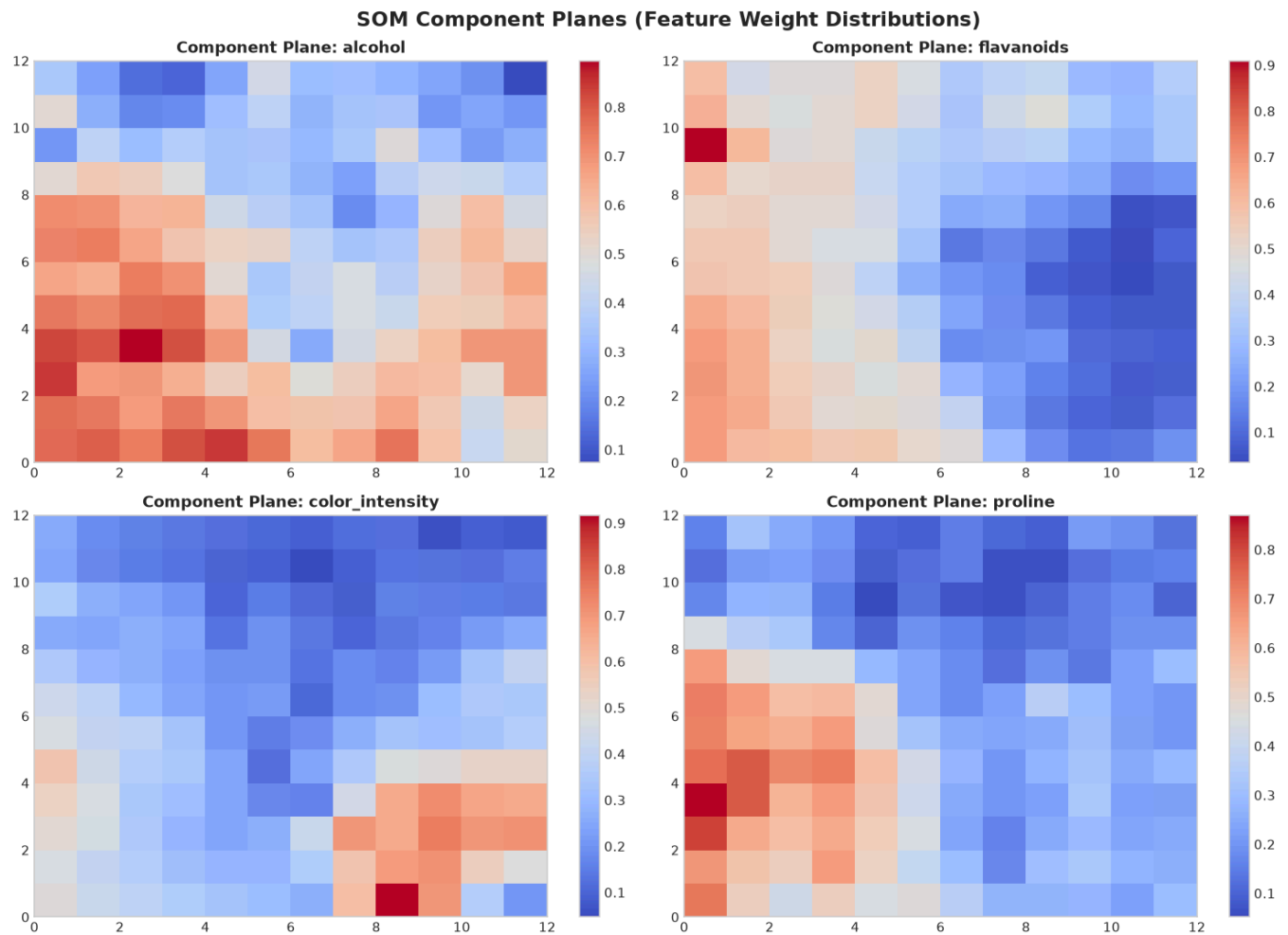
for ax, feat_idx in zip(axes.flat, selected_features):
    weights = som.get_weights()[:, :, feat_idx] # one feature across all neurons
    im = ax.pcolor(weights.T, cmap='coolwarm')
```

```

fig.colorbar(im, ax=ax)
ax.set_title(f"Component Plane: {feature_names[feat_idx]}", fontsize=12, fontweight='bold')

plt.suptitle("SOM Component Planes (Feature Weight Distributions)", fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.show()

```



## 5. Summary & Key Properties of SOM

1. **Topological Preservation:** High-dimensional neighbor relations are faithfully mapped into adjacent neurons on the 2D grid.
2. **Unsupervised Class Discovery:** Without accessing target labels  $y$  during training, the SOM naturally segregates the three wine cultivars into distinct geographic regions on the U-matrix.
3. **Interpretability:** Component planes provide direct visual correlation analysis between input variables across the manifold.

# 08. Hidden Markov Models (HMM): Market Regime Detection with hmmlearn

## Algorithm Covered:

- **Hidden Markov Model (Gaussian HMM)**

## Core Mathematical Problems Addressed:

1. **Evaluation:** Forward-Backward Algorithm ( $P(X | \lambda)$ )
2. **Decoding:** Viterbi Algorithm ( $\arg \max_Z P(Z | X, \lambda)$ )
3. **Learning:** Baum-Welch Expectation-Maximization Algorithm (EM)

**Dataset:** Financial Market Returns & Volatility Regimes ( `data/market_regimes.csv` )

- Uncovering latent market dynamics: Bull (Low Volatility), Bear (High Volatility), and Sideways Regimes

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
# ---- hmmlearn: Gaussian Hidden Markov Model ----
from hmmlearn.hmm import GaussianHMM

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("hmmlearn loaded successfully!")
```

hmmlearn loaded successfully!

## 1. Dataset Loading and Exploratory Analysis

We examine daily financial asset returns exhibiting temporal volatility clustering and regime shifts.

```
In [2]: # ---- Load the market returns dataset ----
df_market = pd.read_csv('../data/market_regimes.csv')
print("Market Data Preview:")
display(df_market.head())

# Observations for the HMM: daily returns (shape T x 1)
returns = df_market['Daily_Return'].values.reshape(-1, 1)
prices = df_market['Close'].values # only used for visualization

# ---- Quick statistical summary ----
print(f"Total Trading Days: {len(df_market)}")
```

```
print(f"Mean Daily Return: {np.mean(returns):.6f}")
print(f"Daily Return StdDev: {np.std(returns):.6f}")
```

Market Data Preview:

|   | Date       | Close  | Daily_Return | True_Regime |
|---|------------|--------|--------------|-------------|
| 0 | 2020-01-01 | 99.41  | -0.00587     | 0           |
| 1 | 2020-01-02 | 99.68  | 0.00271      | 0           |
| 2 | 2020-01-03 | 99.93  | 0.00247      | 0           |
| 3 | 2020-01-06 | 100.62 | 0.00686      | 0           |
| 4 | 2020-01-07 | 100.35 | -0.00269     | 0           |

Total Trading Days: 1000

Mean Daily Return: 0.000115

Daily Return StdDev: 0.013194

## 2. Mathematical Formulation of Hidden Markov Models

An HMM characterizes a doubly stochastic process:

1. **Hidden State Sequence**  $Z = (z_1, z_2, \dots, z_T)$  governed by a 1st-order Markov chain:

$$P(z_t \mid z_{t-1}, \dots, z_1) = P(z_t \mid z_{t-1})$$

defined by the **Transition Matrix**  $A \in \mathbb{R}^{K \times K}$ :

$$A_{ij} = P(z_{t+1} = s_j \mid z_t = s_i)$$

2. **Observable Sequence**  $X = (x_1, x_2, \dots, x_T)$  emitted from hidden states:

$$x_t \mid (z_t = s_k) \sim \mathcal{N}(\mu_k, \sigma_k^2)$$

### The Viterbi Path (Decoding):

To recover the single most likely sequence of hidden states  $Z^*$ , dynamic programming tracks the maximal probability path  $\delta_t(j)$ :

$$\delta_t(j) = \max_i [\delta_{t-1}(i) A_{ij}] \cdot b_j(x_t)$$

$$\psi_t(j) = \arg \max_i [\delta_{t-1}(i) A_{ij}]$$

Backtracking from  $T$  to 1 reconstructs  $Z^*$ .

### Step-by-Step Algorithm (Gaussian HMM)

1. **Model:** a hidden 1st-order Markov chain  $Z$  with transition matrix  $A$ , emitting observations  $x_t \sim \mathcal{N}(\mu_k, \sigma_k^2)$  from the active state.
2. **Learning (Baum-Welch / EM):** iterate until convergence: a. **E-step:** run the forward-backward algorithm to compute posterior state probabilities  $\gamma_t(k)$  and  $\xi_t(i,j)$ . b. **M-step:** re-estimate start probabilities, transition matrix  $A$ , and Gaussian parameters  $(\mu_k, \sigma_k^2)$  from the posteriors.
3. **Decoding (Viterbi):** dynamic programming finds the single most likely state sequence:  $\delta_t(j) = \max_i [\delta_{t-1}(i) A_{ij}] \cdot b_j(x_t)$ ;  $\psi_t(j) = \arg \max_i [\dots]$ ; backtrack from  $T$  to 1.
4. **Evaluation:** the forward algorithm gives  $P(X \mid \lambda)$ , reported as the model log-likelihood.

5. **Post-processing:** sort states by volatility so regime labels (Bull/Sideways/Bear) are stable and interpretable.

```
In [3]: # ---- Fit a 3-state Gaussian HMM with the Baum-Welch (EM) algorithm ----
n_components = 3
hmm_model = GaussianHMM(
    n_components=n_components,          # Bull / Sideways / Bear regimes
    covariance_type="full",             # each state has its own full covariance Gaussian
    emission
    n_iter=200,                         # EM iterations
    random_state=42,
    verbose=False
)

hmm_model.fit(returns)
print("EM Training Converged:", hmm_model.monitor_.converged)
print(f"Model Log-Likelihood: {hmm_model.score(returns):.2f}")

# ---- Decode the most likely hidden state sequence with the Viterbi algorithm ----
hidden_states = hmm_model.predict(returns)

# ---- Sort regimes by volatility so labels have a consistent economic meaning ----
# Regime 0: Low Volatility (Bull) | Regime 1: Moderate (Sideways) | Regime 2: High
# (Bear/Crisis)
volatilities = [np.sqrt(hmm_model.covars_[i][0][0]) for i in range(n_components)]
state_order = np.argsort(volatilities) # ascending
volatility
state_mapping = {old_state: new_state for new_state, old_state in enumerate(state_order)}

sorted_hidden_states = np.array([state_mapping[s] for s in hidden_states]) # relabel
sequence
sorted_means = [hmm_model.means_[old][0] for old in state_order] # reordered
means
sorted_stds = [np.sqrt(hmm_model.covars_[old][0][0]) for old in state_order] # reordered
std devs

# ---- Reorder the transition matrix to match the sorted regimes ----
sorted_transmat = np.zeros((n_components, n_components))
for i in range(n_components):
    for j in range(n_components):
        sorted_transmat[i, j] = hmm_model.transmat_[state_order[i], state_order[j]]

regime_names = ["Bull (Low Vol)", "Sideways (Med Vol)", "Bear (High Vol)"]
```

EM Training Converged: True  
Model Log-Likelihood: 3018.45

```
In [4]: # ---- Summary table of the discovered regimes ----
regime_summary = pd.DataFrame({
    "Regime Name": regime_names,
    "Mean Daily Return ( $\mu$ )": [f"{m:.6f}" for m in sorted_means], # average return
    "Daily Volatility ( $\sigma$ )": [f"{s:.6f}" for s in sorted_stds], # daily std dev
    "Annualized Volatility": [f"{s * np.sqrt(252) * 100:.2f}%" for s in sorted_stds], # x
    "Observed Frequency": [f"{np.mean(sorted_hidden_states == i)*100:.1f}%" for i in
range(n_components)] # % of days
}).set_index("Regime Name")

display(regime_summary)
```

| Regime Name        | Mean Daily Return ( $\mu$ ) | Daily Volatility ( $\sigma$ ) | Annualized Volatility | Observed Frequency |
|--------------------|-----------------------------|-------------------------------|-----------------------|--------------------|
| Bull (Low Vol)     | 0.000508                    | 0.010718                      | 17.01%                | 44.2%              |
| Sideways (Med Vol) | 0.000532                    | 0.010797                      | 17.14%                | 44.2%              |
| Bear (High Vol)    | -0.002547                   | 0.027882                      | 44.26%                | 11.6%              |

### 3. Transition Matrix & Emission Distributions

The transition matrix elements  $A_{ii}$  quantify **regime persistence**: High diagonal values indicate that once the market enters a regime, it remains there with high probability.

#### Reading the Transition Matrix & Emissions

1. Diagonal entries  $A_{ii}$  = probability of **staying** in the same regime (regime persistence).
2. Off-diagonal entries = probability of switching regimes next day.
3. Each emission curve shows the return distribution the model associates with a regime (low-vol vs high-vol).
4. Wider curves (larger  $\sigma$ ) indicate more volatile regimes.

```
In [5]: # ---- Transition matrix + emission densities ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

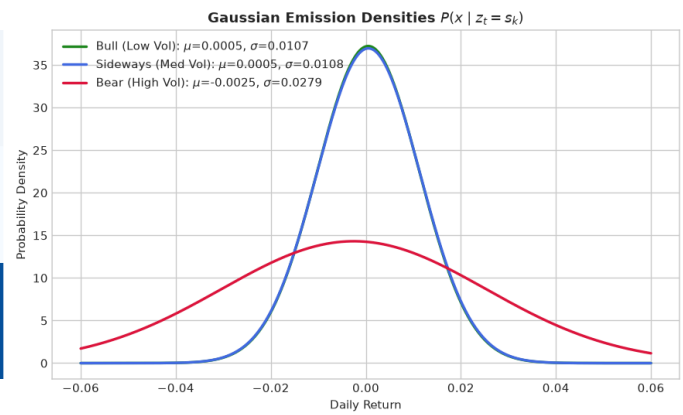
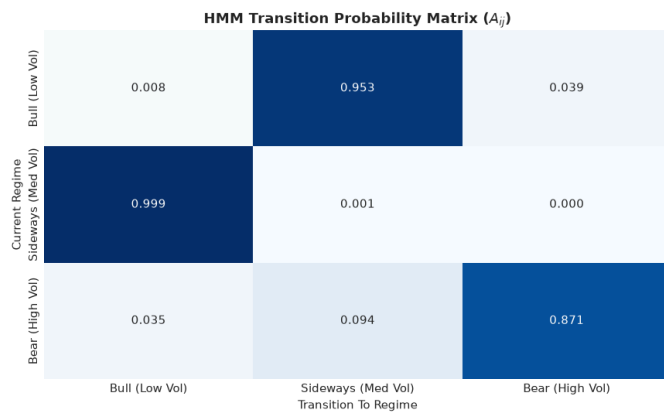
# Panel 1: transition-probability heatmap (rows = current regime, cols = next regime)
sns.heatmap(sorted_transmat, annot=True, fmt='.3f', cmap='Blues', ax=ax1,
            xticklabels=regime_names, yticklabels=regime_names, cbar=False)
ax1.set_title("HMM Transition Probability Matrix ( $A_{ij}$ )", fontweight='bold')
ax1.set_xlabel("Transition To Regime")
ax1.set_ylabel("Current Regime")

# Panel 2: Gaussian emission densities per regime (P(return | regime))
x_axis = np.linspace(-0.06, 0.06, 500)
colors = ['forestgreen', 'royalblue', 'crimson']

for i in range(n_components):
    ax2.plot(x_axis, norm.pdf(x_axis, sorted_means[i], sorted_stds[i]),
            color=colors[i], lw=2.2, label=rf"{regime_names[i]}:  $\mu$ ={sorted_means[i]:.4f},  $\sigma$ ={sorted_stds[i]:.4f}")

ax2.set_title(r"Gaussian Emission Densities  $P(x \mid z_t = s_k)$ ", fontweight='bold')
ax2.set_xlabel("Daily Return")
ax2.set_ylabel("Probability Density")
ax2.legend()

plt.tight_layout()
plt.show()
```



## 4. Visualizing Viterbi Decoded Regimes Across Time

Let's overlay the decoded hidden market states on top of the asset price trajectory.

### Regime Visualization Process

1. Decode the full sequence with Viterbi (each day gets one regime label).
2. Color every consecutive price segment by the regime of that day.
3. Plot the state sequence in a second panel to see persistence and switch points.
4. Compare with the known market story: bull trends, sideways consolidation, volatility spikes.

```
In [6]: # ---- Overlay the Viterbi-decoded regimes on the price series ----
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 8), sharex=True, gridspec_kw=
{'height_ratios': [3, 1]})

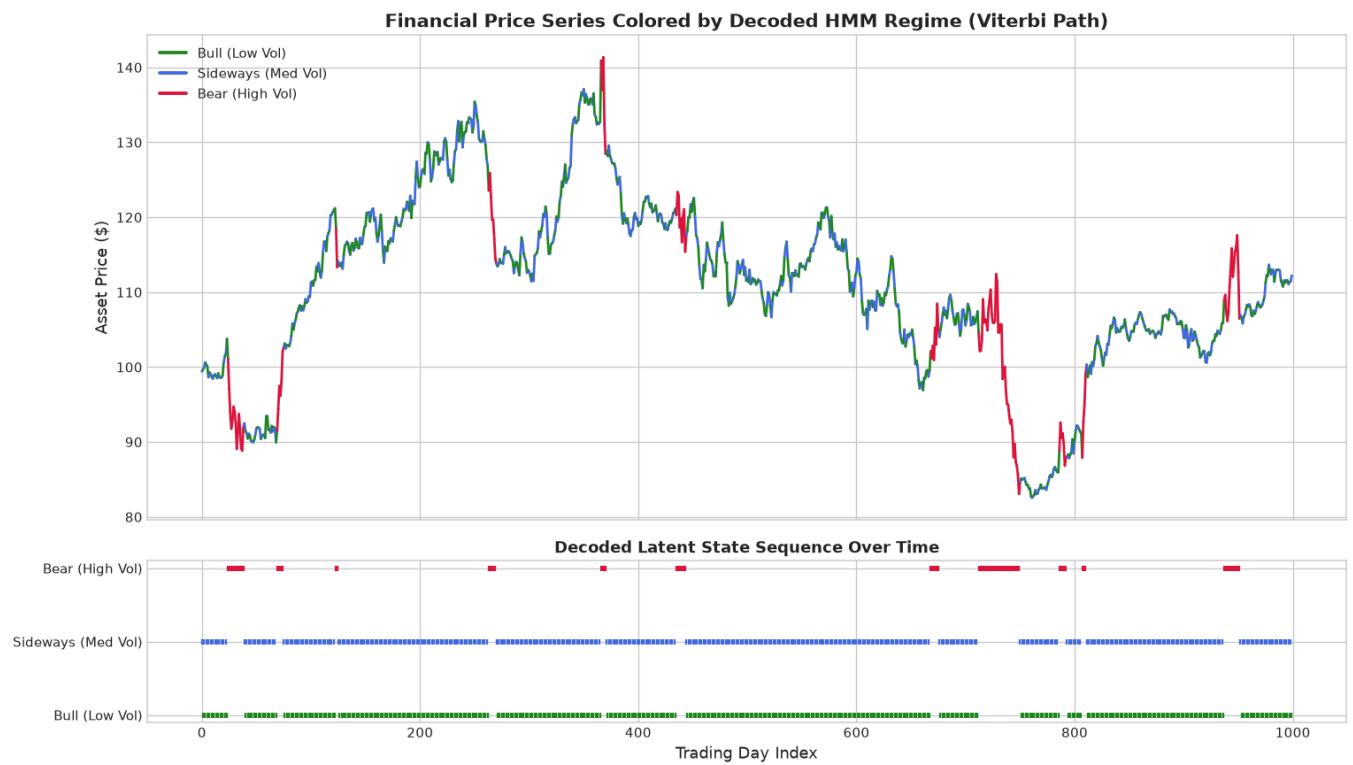
# Top panel: draw each day-to-day price segment in the color of its decoded regime
for i in range(len(prices) - 1):
    reg = sorted_hidden_states[i]
    ax1.plot([i, i+1], [prices[i], prices[i+1]], color=colors[reg], lw=1.8)

# Legend handles (one per regime)
for i, name in enumerate(regime_names):
    ax1.plot([], [], color=colors[i], label=name, lw=2.5)

ax1.set_title("Financial Price Series Colored by Decoded HMM Regime (Viterbi Path)",
              fontsize=14, fontweight='bold')
ax1.set_ylabel("Asset Price ($) ", fontsize=12)
ax1.legend(loc='upper left')

# Bottom panel: the raw sequence of decoded states over time
ax2.scatter(range(len(sorted_hidden_states)), sorted_hidden_states, c=[colors[s] for s in
sorted_hidden_states], s=15, marker='|')
ax2.set_yticks(range(n_components))
ax2.set_yticklabels(regime_names)
ax2.set_title("Decoded Latent State Sequence Over Time", fontsize=12, fontweight='bold')
ax2.set_xlabel("Trading Day Index", fontsize=12)

plt.tight_layout()
plt.show()
```



## 5. Summary & Key Takeaways

1. **Unsupervised Temporal Clustering:** Gaussian HMM discovers structural volatility shifts without requiring labeled market state annotations.
2. **Dynamic Transition Memory:** Unlike static mixture models, the Markov transition matrix captures state persistence and temporal autocorrelation.
3. **Viterbi Decoding:** Provides a globally optimal state trajectory that aligns with market cycles (Bull expansions, Sideways consolidations, and Bear volatility spikes).

# 09. Support Vector Machines (SVM): Classification and Regression

## Algorithm Covered:

- **Support Vector Machine (SVM)**
  1. **Support Vector Classification (SVC)** with Linear, Polynomial, and RBF Kernels
  2. **Support Vector Regression (SVR)** with  $\epsilon$ -Insensitive Tube

## Core Focus:

- Maximum margin hyperplane optimization and soft-margin slack variables ( $\xi_i$ )
- Kernel Trick mapping non-linearly separable data into high-dimensional Hilbert spaces
- Support vector identification, margin geometry, and dual formulation
- Hyperparameter sensitivity analysis ( $C, \gamma, \epsilon$ )

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Datasets + model selection ----
from sklearn.datasets import load_breast_cancer, make_moons
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler

# ---- SVM models: classification and regression ----
from sklearn.svm import SVC, SVR

# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, r2_score,
root_mean_squared_error

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("SVM modules loaded successfully!")
```

SVM modules loaded successfully!

## PART 1: Support Vector Classification (SVC)

### Mathematical Formulation

Given training pairs  $\{(x_i, y_i)\}$  with  $y_i \in \{-1, +1\}$ , soft-margin SVC solves the primal quadratic program:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{subject to } y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Using Lagrange multipliers  $\alpha_i$ , the dual formulation depends strictly on inner products:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum \alpha_i y_i = 0$$

Only samples with  $\alpha_i > 0$  become **Support Vectors**, lying on or violating the margin boundaries.

## Step-by-Step Algorithm (Support Vector Classification)

1. **Formulate** the soft-margin problem: minimize  $\frac{1}{2}\|w\|^2 + C \sum \xi_i$  subject to  $y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i$ ,  $\xi_i \geq 0$ .
2. **Solve** the dual quadratic program with Lagrange multipliers  $\alpha_i$  (convex  $\rightarrow$  global optimum).
3. **Identify support vectors**: training points with  $\alpha_i > 0$ ; they lie on or inside the margin.
4. **Kernel trick**: replace inner products with  $K(x_i, x_j)$  (linear, polynomial, RBF, ...) to model non-linear boundaries.
5. **Predict**: class = sign( $\sum \alpha_i y_i K(x_i, x) + b$ ).
6. **Tune** C and  $\gamma$  by cross-validation to balance margin width and violations.

```
In [2]: # ---- 2D synthetic data for visualizing margins and kernels ----
# Two interleaving half-moons cannot be separated by a straight line.
X_2d, y_2d = make_moons(n_samples=250, noise=0.20, random_state=42)
scaler_2d = StandardScaler()
X_2d_scaled = scaler_2d.fit_transform(X_2d)

kernels = ['linear', 'poly', 'rbf']
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Coordinate grid used to draw the decision boundary as a contour
x_min, x_max = X_2d_scaled[:, 0].min() - 0.5, X_2d_scaled[:, 0].max() + 0.5
y_min, y_max = X_2d_scaled[:, 1].min() - 0.5, X_2d_scaled[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

for ax, k_name in zip(axes, kernels):
    # Train one SVC per kernel on the same data
    clf = SVC(kernel=k_name, C=1.0, gamma='scale', degree=3, random_state=42)
    clf.fit(X_2d_scaled, y_2d)

    # Signed distance to the hyperplane for every grid point
    Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # Filled contour = decision regions; solid line = boundary (Z=0); dashed = margins (Z=±1)
    ax.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 20), cmap='coolwarm',
alpha=0.3)
    ax.contour(xx, yy, Z, levels=[-1.0, 0.0, 1.0], linestyles=['--', '-', '--'], colors=['k',
'k', 'k'], linewidths=[1.5, 2.5, 1.5])

    # Scatter the data points (colored by class)
    ax.scatter(X_2d_scaled[:, 0], X_2d_scaled[:, 1], c=y_2d, cmap='coolwarm', edgecolors='k',
s=40, alpha=0.85)

    # Highlight the support vectors (the points that define the margin)
    sv = clf.support_vectors_
    ax.scatter(sv[:, 0], sv[:, 1], s=120, facecolors='none', edgecolors='gold', linewidths=2.0,
label=f'SV ({len(sv)})')

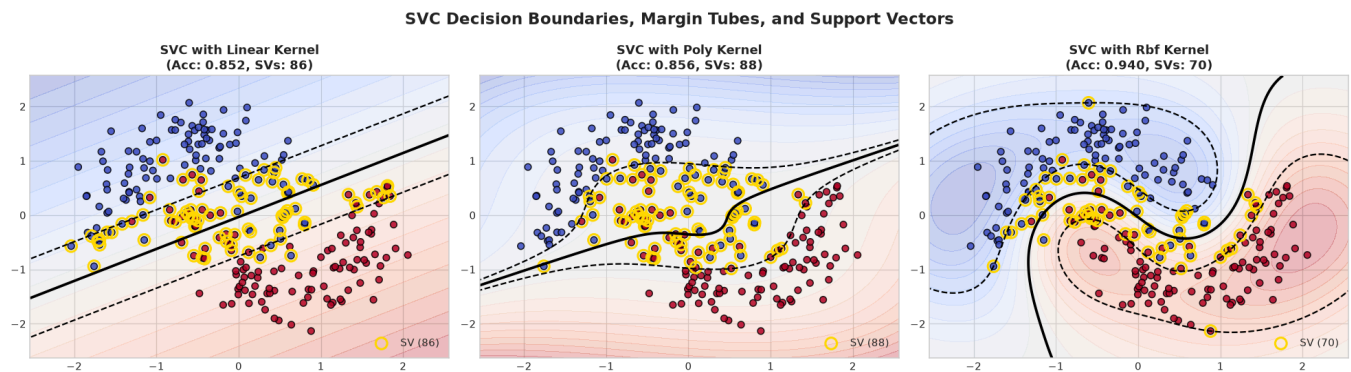
    ax.set_title(f"SVC with {k_name.capitalize()} Kernel\n(Acc: {clf.score(X_2d_scaled,
```

```

y_2d)[:.3f}], SVs: {len(sv)}]", fontweight='bold')
ax.legend(loc='lower right')

plt.suptitle("SVC Decision Boundaries, Margin Tubes, and Support Vectors", fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.show()

```



## 1.2 SVC on High-Dimensional Diagnostic Benchmark

We evaluate SVC across kernels on the **Breast Cancer** dataset (30 features).

```

In [3]: # ---- High-dimensional benchmark: Breast Cancer dataset (30 features) ----
cancer = load_breast_cancer()
X_c = cancer.data
y_c = cancer.target

# Stratified split + scaling (SVM is distance-based, scaling is essential)
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_c, y_c, test_size=0.25,
random_state=42, stratify=y_c)
scaler_c = StandardScaler()
X_train_c_s = scaler_c.fit_transform(X_train_c)
X_test_c_s = scaler_c.transform(X_test_c)

# ---- Train one SVC per kernel and record test accuracy + support-vector usage ----
kernel_results = []
for k in ['linear', 'poly', 'rbf', 'sigmoid']:
    model = SVC(kernel=k, C=1.0, random_state=42)
    model.fit(X_train_c_s, y_train_c)
    acc = model.score(X_test_c_s, y_test_c)
    kernel_results.append({
        "Kernel": k.capitalize(),
        "Test Accuracy": acc,
        "Support Vectors": len(model.support_),
        "SV Ratio": f"{len(model.support_) / len(X_train_c) * 100:.1f}%"
    })

df_k = pd.DataFrame(kernel_results).set_index("Kernel")
print("SVC Kernel Benchmark on Breast Cancer Dataset:")
# Green = highest accuracy and fewest support vectors (sparser solution)
display(df_k.style.highlight_max(subset=['Test Accuracy'], color='lightgreen')
        .highlight_min(subset=['Support Vectors'], color='lightgreen'))

```

|         | Test Accuracy | Support Vectors | SV Ratio |
|---------|---------------|-----------------|----------|
| Kernel  |               |                 |          |
| Linear  | 0.986014      | 34              | 8.0%     |
| Poly    | 0.902098      | 136             | 31.9%    |
| Rbf     | 0.979021      | 96              | 22.5%    |
| Sigmoid | 0.951049      | 65              | 15.3%    |

### 1.3 Hyperparameter Tuning ( $C$ and $\gamma$ )

- **$C$  (Cost parameter):** Controls the tradeoff between smooth decision boundary and margin violation penalty. Large  $C$  creates tight, complex margins; small  $C$  permits more violations for a wider margin.
- **$\gamma$  (Gamma for RBF kernel):** Controls the radius of influence of each support vector. Large  $\gamma$  creates localized, high-variance boundaries; small  $\gamma$  behaves more linearly.

### Hyperparameter Tuning Process (Grid Search)

1. Define a grid over  $C$  (violation penalty) and  $\gamma$  (RBF width).
2. For each pair, run 5-fold cross-validation on the training set and average the validation accuracy.
3. Select the pair with the best mean CV score.
4. Refit the final model and visualize the CV-accuracy heatmap.

```
In [4]: # ---- Grid search over the two key RBF hyperparameters ----
# C      : penalty for margin violations (bias/variance trade-off)
# gamma  : RBF kernel width (how far the influence of one support vector reaches)
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1]
}

# 5-fold cross-validation on the training set for every (C, gamma) combination
grid_svc = GridSearchCV(SVC(kernel='rbf', random_state=42), param_grid, cv=5,
                        scoring='accuracy')
grid_svc.fit(X_train_c_s, y_train_c)

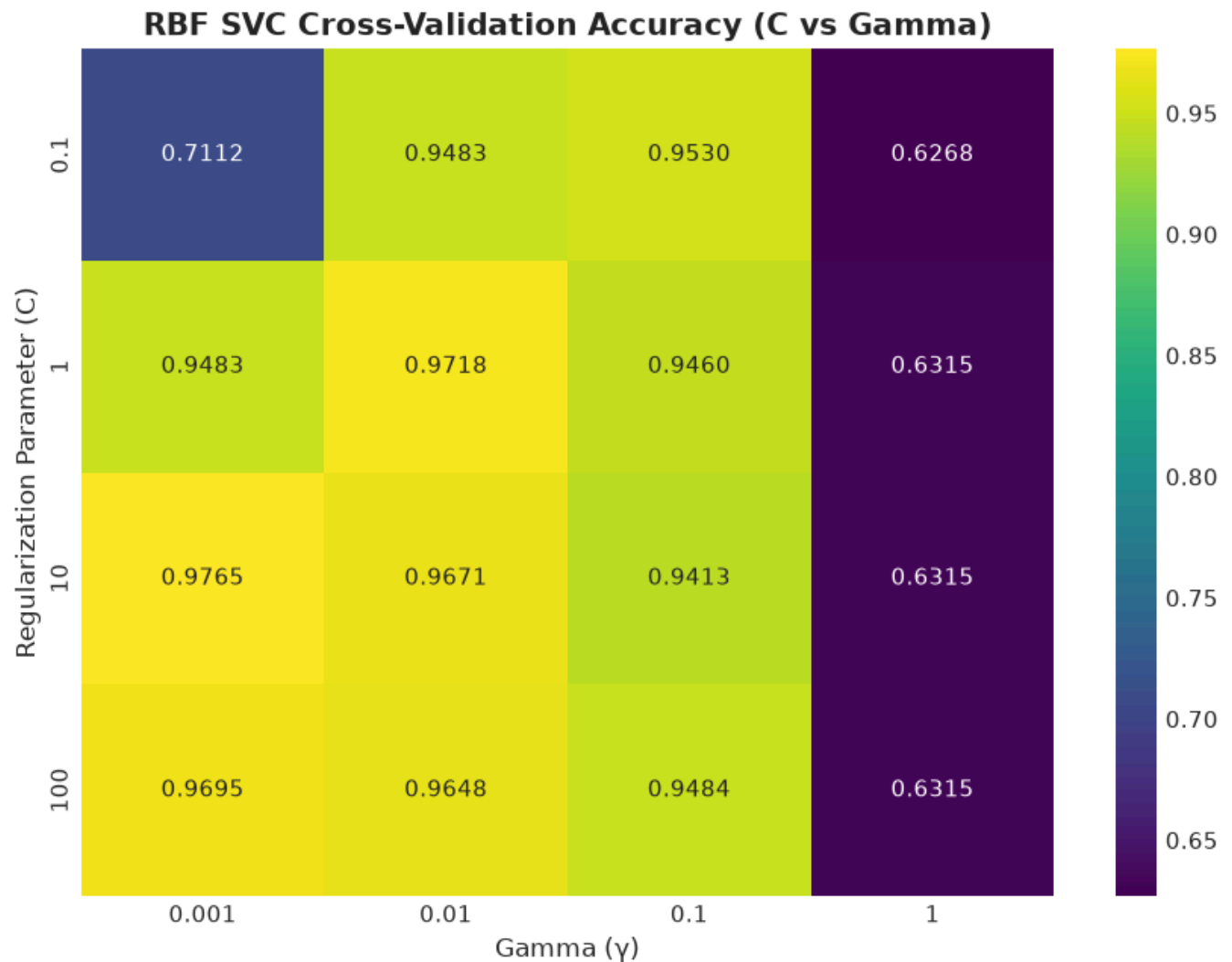
print("Best Parameters:", grid_svc.best_params_)
print(f"Best 5-Fold CV Accuracy: {grid_svc.best_score_:.4f}")

# Reshape the CV scores into a (C x gamma) matrix for the heatmap
scores_matrix = grid_svc.cv_results_['mean_test_score'].reshape(len(param_grid['C']),
                                                                len(param_grid['gamma']))

plt.figure(figsize=(8, 6))
sns.heatmap(scores_matrix, annot=True, fmt='.4f', cmap='viridis',
            xticklabels=param_grid['gamma'], yticklabels=param_grid['C'])
plt.title("RBF SVC Cross-Validation Accuracy (C vs Gamma)", fontsize=13, fontweight='bold')
plt.xlabel("Gamma ( $\gamma$ )", fontsize=11)
plt.ylabel("Regularization Parameter (C)", fontsize=11)
plt.tight_layout()
plt.show()
```

Best Parameters: {'C': 10, 'gamma': 0.001}

Best 5-Fold CV Accuracy: 0.9765



## PART 2: Support Vector Regression (SVR)

### Mathematical Formulation

In SVR, errors within an  $\epsilon$ -**insensitive tube** are ignored without penalty ( $|y_i - f(x_i)| \leq \epsilon$ ). For deviations exceeding  $\epsilon$ , slack variables  $\xi_i, \xi_i^*$  incur a linear penalty:

$$\min_{w, b, \xi, \xi^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*)$$
$$\text{s.t. } y_i - (w^T \phi(x_i) + b) \leq \epsilon + \xi_i, \quad (w^T \phi(x_i) + b) - y_i \leq \epsilon + \xi_i^*, \quad \xi_i, \xi_i^* \geq 0$$

Samples residing outside the  $\epsilon$ -tube become the **Support Vectors** that anchor the regression surface.

### Step-by-Step Algorithm (Support Vector Regression)

1. Define an  $\epsilon$ -insensitive tube: residuals with  $|y_i - f(x_i)| \leq \epsilon$  incur zero loss.
2. Minimize  $\frac{1}{2} \|w\|^2 + C \sum (\xi_i + \xi_i^*)$  subject to the tube constraints.
3. Solve the dual; only points on/outside the tube get  $\alpha_i > 0$  and become support vectors.
4. **Predict:**  $f(x) = \sum \alpha_i K(x_i, x) + b$  (kernel regression).
5. Visualize the fitted curve, the  $\pm \epsilon$  tube, and the support vectors.

```

In [5]: # ---- 1D non-linear regression demonstration ----
np.random.seed(42)
# Synthetic target: sine + linear trend + Gaussian noise
X_svr = np.sort(np.random.uniform(-3, 3, 120)).reshape(-1, 1)
y_svr = np.sin(X_svr).ravel() + 0.15 * X_svr.ravel() + np.random.normal(0, 0.12, 120)

# epsilon defines the width of the "no penalty" tube around the prediction
epsilon_val = 0.18
svr_model = SVR(kernel='rbf', C=20.0, epsilon=epsilon_val, gamma=0.5)
svr_model.fit(X_svr, y_svr)

# Dense grid for drawing a smooth prediction curve
X_grid = np.linspace(-3.2, 3.2, 300).reshape(-1, 1)
y_grid_pred = svr_model.predict(X_grid)

# Support vectors = training points that lie on/outside the epsilon tube
sv_indices = svr_model.support_
X_sv = X_svr[sv_indices]
y_sv = y_svr[sv_indices]

plt.figure(figsize=(11, 6))
# Training samples
plt.scatter(X_svr, y_svr, color='navy', s=35, alpha=0.75, label='Training Samples')

# Regression surface
plt.plot(X_grid, y_grid_pred, color='crimson', lw=2.5, label='SVR Prediction  $f(x)$ ')

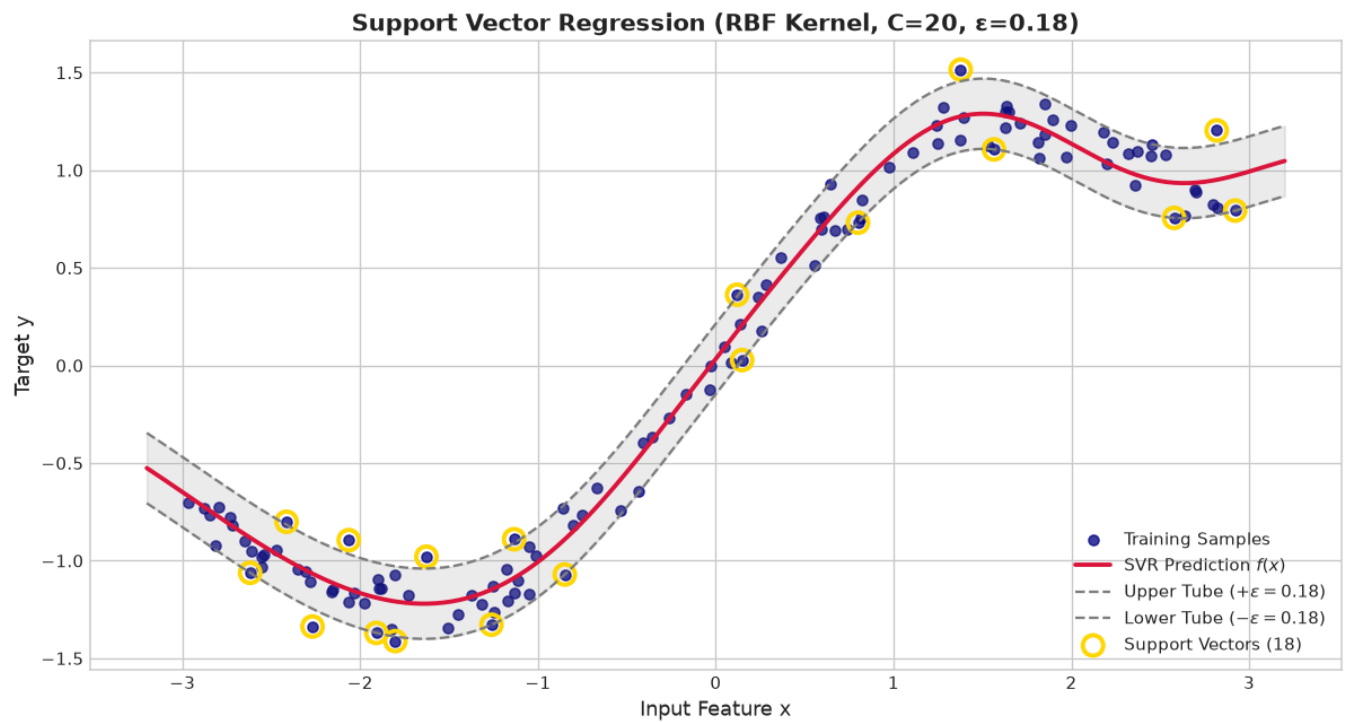
# Epsilon-insensitive tube boundaries + shaded tube
plt.plot(X_grid, y_grid_pred + epsilon_val, color='gray', linestyle='--', lw=1.5,
label=rf'Upper Tube ( $+\epsilon$ )')
plt.plot(X_grid, y_grid_pred - epsilon_val, color='gray', linestyle='--', lw=1.5,
label=rf'Lower Tube ( $-\epsilon$ )')
plt.fill_between(X_grid.ravel(), y_grid_pred - epsilon_val, y_grid_pred + epsilon_val,
color='gray', alpha=0.15)

# Support vectors are highlighted - only these points influence the fitted curve
plt.scatter(X_sv, y_sv, s=140, facecolors='none', edgecolors='gold', linewidths=2.5,
label=f'Support Vectors ({len(X_sv)})')

plt.title(f"Support Vector Regression (RBF Kernel, C=20,  $\epsilon$ = $\epsilon$ )", fontsize=14,
fontweight='bold')
plt.xlabel("Input Feature x", fontsize=12)
plt.ylabel("Target y", fontsize=12)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

# ---- Quantitative evaluation ----
print(f"SVR R2 Score: {r2_score(y_svr, svr_model.predict(X_svr)):.4f}")
print(f"SVR RMSE: {root_mean_squared_error(y_svr, svr_model.predict(X_svr)):.4f}")
print(f"Support Vectors: {len(X_sv)} out of {len(X_svr)} points  
( $\frac{{len(X_{sv})}}{{len(X_{svr})}} \times 100 = .1f$ %)")

```



SVR  $R^2$  Score: 0.9864

SVR RMSE: 0.1157

Support Vectors: 18 out of 120 points (15.0%)

### 3. Conclusions & Key Takeaways

1. **Sparsity of Solution:** Both SVC and SVR depend only on a subset of training points (the support vectors); non-support points can be deleted without altering the learned function.
2. **Global Convexity:** Unlike neural networks with non-convex loss surfaces and local minima, SVM optimization is a convex quadratic program with a guaranteed global optimum.
3. **Kernel Flexibility:** The Kernel Trick enables linear algorithms to model arbitrarily complex, non-linear manifolds in feature space without explicitly computing high-dimensional coordinates.

# 10. Large Language Models (LLM): Tokenization, Inference, and Mini Fine-Tuning

## Algorithm Covered:

- **Large Language Models (Transformer Architecture)**

## Core Capabilities Explored:

1. **Subword Tokenization:** Token IDs, Attention Masks, Vocabulary Embeddings
2. **Multi-Head Self-Attention Mechanics:**  $Q$ ,  $K$ ,  $V$  projections and softmax attention weights
3. **Zero-Shot & Downstream Inference:** High-level Hugging Face Pipelines
4. **Text Generation & Decoding Strategies:** Greedy Search vs. Temperature vs. Top-p (Nucleus) Sampling
5. **Mini Fine-Tuning Demonstration:** Sequence classification adaptation using PyTorch and Hugging Face Transformers

**Frameworks:** Hugging Face `transformers` and `torch`

```
In [1]: # ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- PyTorch: used for the mini fine-tuning demo ----
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
# ---- Hugging Face Transformers: tokenizers, models, pipelines, generation configs ----
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    AutoModelForCausalLM,
    GenerationConfig,
    pipeline
)

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

torch.manual_seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # GPU if available
print(f"PyTorch on {device}")
```

```
D:\ML\Assignment\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please
update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
PyTorch on cpu
```

# 1. Subword Tokenization & Attention Masks

## Mathematical Concept:

Language models do not operate on raw characters or entire words; they process **subword tokens** (Byte-Pair Encoding / WordPiece). Given input text  $S$ , the tokenizer decomposes  $S$  into token IDs  $T = [t_1, t_2, \dots, t_L]$  and generates an **Attention Mask**  $M \in \{0, 1\}^L$  ensuring padding tokens receive zero attention during self-attention computation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} + (M - 1) \cdot \infty \right) V$$

## Step-by-Step Algorithm (Transformer Tokenization)

1. Normalize and split the raw text (lowercase, punctuation handling, etc.).
2. Apply subword segmentation (WordPiece/BPE): frequent words stay whole, rare words split into pieces + "###" continuations.
3. Map each subword to its integer vocabulary ID.
4. Add special tokens ([CLS], [SEP]) and pad/truncate to a fixed length.
5. Build the attention mask: 1 for real tokens, 0 for padding.
6. Feed IDs + mask into the transformer's embedding and self-attention layers.

```
In [2]: # ---- Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based) ----
model_id = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
print(f"Loading Tokenizer: {model_id}...")
tokenizer = AutoTokenizer.from_pretrained(model_id)

sample_text = "Machine learning and deep neural networks are transforming modern science!"
# Tokenize with padding/truncation so the output has a fixed shape (here 20 tokens)
encoded = tokenizer(
    sample_text,
    padding="max_length",
    max_length=20,
    truncation=True,
    return_tensors="pt"
)

# Convert integer IDs back to readable subword tokens
tokens = tokenizer.convert_ids_to_tokens(encoded['input_ids'][0])
print(f"Original Text: {sample_text}\n")
print(f"Subword Tokens ({len(tokens)}):\n", tokens)
print(f"Input IDs: \n", encoded['input_ids'][0].tolist())
print(f"Attention Mask: \n", encoded['attention_mask'][0].tolist())
```

Loading Tokenizer: distilbert/distilbert-base-uncased-finetuned-sst-2-english...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF\_TOKEN to enable higher rate limits and faster downloads.

Original Text: Machine learning and deep neural networks are transforming modern science!

Subword Tokens (20):

['[CLS]', 'machine', 'learning', 'and', 'deep', 'neural', 'networks', 'are', 'transforming', 'modern', 'science', '!', '[SEP]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]']

Input IDs:

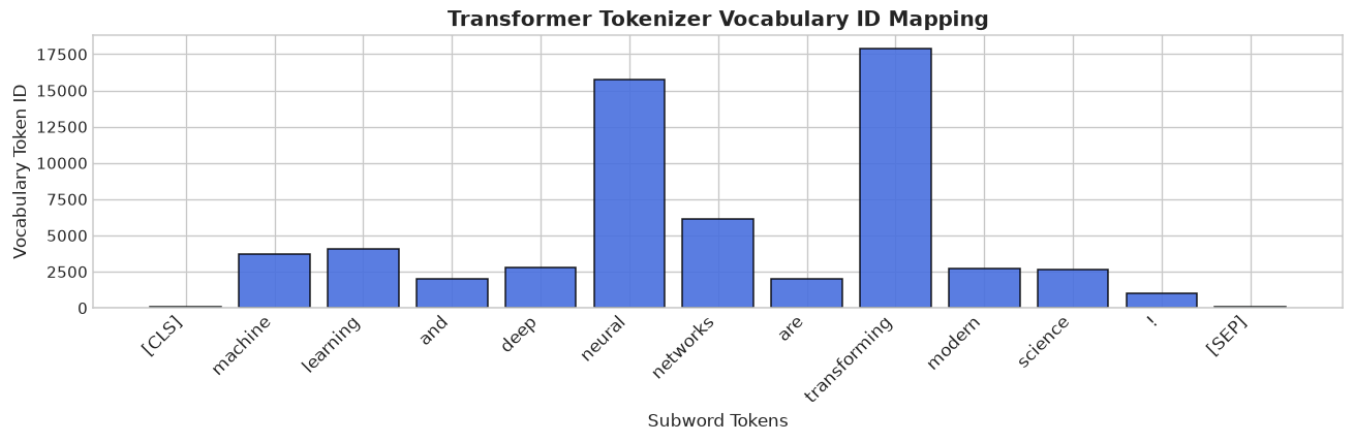
[101, 3698, 4083, 1998, 2784, 15756, 6125, 2024, 17903, 2715, 2671, 999, 102, 0, 0, 0, 0, 0, 0]

Attention Mask:

[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]

```
In [3]: # ---- Visualize the tokenizer's vocabulary-ID mapping ----
# Keep only real tokens (attention mask = 1) for a clean bar chart.
active_len = int(encoded['attention_mask'].sum().item())
active_tokens = tokens[:active_len]
active_ids = encoded['input_ids'][0][:active_len].tolist()

plt.figure(figsize=(12, 4))
plt.bar(range(len(active_tokens)), active_ids, color='royalblue', edgecolor='k', alpha=0.85)
plt.xticks(range(len(active_tokens)), active_tokens, rotation=45, ha='right', fontsize=11)
plt.title("Transformer Tokenizer Vocabulary ID Mapping", fontsize=13, fontweight='bold')
plt.xlabel("Subword Tokens", fontsize=11)
plt.ylabel("Vocabulary Token ID", fontsize=11)
plt.tight_layout()
plt.show()
```



## 2. Transformer Inference: Sentiment Classification Pipeline

We load a lightweight, production-grade distilled transformer ( `DistilBERT` fine-tuned on the Stanford Sentiment Treebank / SST-2) to compute class logits and calibrated posterior probabilities.

### Inference Process (Hugging Face Pipeline)

1. Load the pre-trained checkpoint (weights + tokenizer).
2. Tokenize each input sentence exactly as during pre-training.
3. Run a forward pass through the transformer encoder.
4. Apply softmax to the classification head logits to obtain class probabilities.
5. Return the highest-probability label with its confidence score.

```
In [4]: # ---- Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2) ----
classifier = pipeline("sentiment-analysis", model=model_id, device=-1) # -1 = run on CPU

test_sentences = [
    "The new neural network model demonstrated extraordinary accuracy and remarkable speed.",
    "The dataset was poorly labeled, noisy, and the model completely failed to converge.",
    "The gradient descent algorithm performed reasonably well, meeting standard expectations.",
    "A catastrophic failure occurred during training due to exploding gradient issues."
]

# One forward pass per sentence -> Label + calibrated confidence score
predictions = classifier(test_sentences)

# Collect results in a table for display
results_table = []
for sent, pred in zip(test_sentences, predictions):
```

```

results_table.append({
    "Sentence": sent,
    "Predicted Label": pred['label'],
    "Confidence Score": f"{pred['score']*100:.2f}%"
})

df_preds = pd.DataFrame(results_table)
display(df_preds)

```

```

Loading weights: 0%|          | 0/104 [00:00<?, ?it/s]
Loading weights: 83%|██████    | 86/104 [00:00<00:00, 855.44it/s]
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 1004.43it/s]

```

|   | Sentence                                          | Predicted Label | Confidence Score |
|---|---------------------------------------------------|-----------------|------------------|
| 0 | The new neural network model demonstrated extr... | POSITIVE        | 99.99%           |
| 1 | The dataset was poorly labeled, noisy, and the... | NEGATIVE        | 99.98%           |
| 2 | The gradient descent algorithm performed reaso... | POSITIVE        | 99.92%           |
| 3 | A catastrophic failure occurred during trainin... | NEGATIVE        | 99.96%           |

### 3. Autoregressive Text Generation & Sampling Strategies

In autoregressive causal language modeling, each token is generated sequentially from the conditional distribution:

$$P(t_n \mid t_1, \dots, t_{n-1}) = \text{softmax}\left(\frac{z_n}{T}\right)$$

- **Greedy Decoding** ( $T \rightarrow 0$ ): Selects  $\arg \max_v z_v$ ; highly deterministic, prone to repetitive loops.
- **Temperature** ( $T$ ): Softens ( $T > 1$ ) or sharpens ( $T < 1$ ) logit probabilities.
- **Top- $p$  (Nucleus) Sampling**: Restricts selection to the smallest subset of tokens whose cumulative probability exceeds threshold  $p$ .

#### Step-by-Step Algorithm (Autoregressive Generation)

1. Tokenize the prompt and run the transformer to obtain next-token logits.
2. **Greedy decoding**: pick the single most likely token each step (deterministic, can loop).
3. **Temperature scaling**: divide logits by  $T$  before softmax ( $T < 1$  sharpens,  $T > 1$  flattens).
4. **Top- $p$  (nucleus) sampling**: keep the smallest token set whose cumulative probability  $\geq p$ , then sample inside it.
5. Append the sampled token to the sequence and repeat until the token budget is reached.
6. Decode the final token IDs back to text.

```

In [5]: # ---- Load a small causal LM (DistilGPT2) for text generation ----
gen_model_id = "distilbert/distilgpt2"
print(f"Loading Text Generator: {gen_model_id}...")
# clean_up_tokenization_spaces=False keeps BPE spacing intact
generator = pipeline("text-generation", model=gen_model_id, device=-1,
clean_up_tokenization_spaces=False)

prompt = "Artificial Intelligence will fundamentally transform"

# 1. Greedy Search: always pick the most likely next token (deterministic)
greedy_output = generator(

```

```

prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=False)
)[0]['generated_text']

# 2. Temperature + Top-p (Nucleus) Sampling: sample from the most likely tokens up to
cumulative p
sampled_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=True,
temperature=0.7, top_p=0.9)
)[0]['generated_text']

print(f"PROMPT: '{prompt}'\n")
print(f"--- 1. GREEDY SEARCH OUTPUT ---\n{greedy_output.strip()}\n")
print(f"--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---\n{sampled_output.strip()}")

```

Loading Text Generator: distilbert/distilgpt2...

```

Loading weights: 0%|          | 0/76 [00:00<?, ?it/s]
Loading weights: 21%|██        | 16/76 [00:00<00:00, 70.10it/s]
Loading weights: 100%|██████████| 76/76 [00:00<00:00, 305.93it/s]

```

PROMPT: 'Artificial Intelligence will fundamentally transform'

--- 1. GREEDY SEARCH OUTPUT ---

Artificial Intelligence will fundamentally transform the way we think about the world.

--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---

Artificial Intelligence will fundamentally transform the way people think about the world.

## 4. Mini Fine-Tuning Demonstration with PyTorch

We adapt a pre-trained Transformer classifier on a custom domain classification task: Classifying synthetic customer support queries into **Technical Inquiry (0)** vs. **Billing Inquiry (1)**.

### Fine-Tuning Process (Sequence Classification)

1. Build a small labeled dataset of domain queries (Technical vs Billing).
2. Tokenize all texts with padding/truncation and wrap them in a PyTorch Dataset/DataLoader.
3. Load the pre-trained transformer with a new randomly-initialized 2-class head.
4. For each epoch: forward pass, cross-entropy loss, backpropagation, AdamW update (small LR).
5. Track the loss curve across epochs.
6. Evaluate on unseen queries: softmax the logits and report predicted class + confidence.

```

In [6]: # ---- Tiny custom dataset: classify support queries as Technical (0) or Billing (1) ----
train_texts = [
    "My model throws a CUDA out of memory error during backpropagation",
    "How do I fix the gradient vanishing issue in my deep recurrent network?",
    "The API returns a 500 internal server error when requesting predictions",
    "Where can I find the documentation for custom PyTorch loss functions?",
    "Why was my credit card charged twice for the monthly subscription?",
    "I would like to request an invoice for my corporate license",
    "Can you refund my annual subscription fee?",
    "How can I upgrade my billing tier to enterprise?"
]
train_labels = [0, 0, 0, 0, 1, 1, 1, 1] # 0 = Technical, 1 = Billing

# Tokenize the whole dataset once (padded, with attention masks)
encodings = tokenizer(train_texts, padding=True, truncation=True, return_tensors="pt")

class MiniTextDataset(Dataset):
    """Wrap tokenized encodings + labels into a PyTorch Dataset."""

```

```

def __init__(self, encodings, labels):
    self.encodings = encodings
    self.labels = labels

def __getitem__(self, idx):
    # One dictionary per sample: input_ids, attention_mask, labels
    item = {key: val[idx] for key, val in self.encodings.items()}
    item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
    return item

def __len__(self):
    return len(self.labels)

dataset = MiniTextDataset(encodings, train_labels)
dataloader = DataLoader(dataset, batch_size=4, shuffle=True)

# ---- Load the pre-trained classifier head with a fresh 2-class head ----
# ignore_mismatched_sizes allows replacing the original SST-2 head with a new one.
ft_model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2,
ignore_mismatched_sizes=True)
ft_model.to(device)
optimizer = torch.optim.AdamW(ft_model.parameters(), lr=2e-5) # small LR for fine-tuning

print("Pre-trained Transformer loaded for fine-tuning!")

```

```

Loading weights:  0%|          | 0/104 [00:00<?, ?it/s]
Loading weights: 12%|██        | 13/104 [00:00<00:00, 120.46it/s]
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 849.83it/s]
Pre-trained Transformer loaded for fine-tuning!

```

```

In [7]: # ---- Mini fine-tuning loop ----
epochs = 4
ft_model.train()
epoch_losses = []

for epoch in range(epochs):
    running_loss = 0.0
    for batch in dataloader:
        optimizer.zero_grad()

        # Move the batch to the selected device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Forward pass: the model computes the classification loss internally
        outputs = ft_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward() # backpropagate through the whole transformer
        optimizer.step() # update weights

    running_loss += loss.item() * len(input_ids) # de-average the batch loss

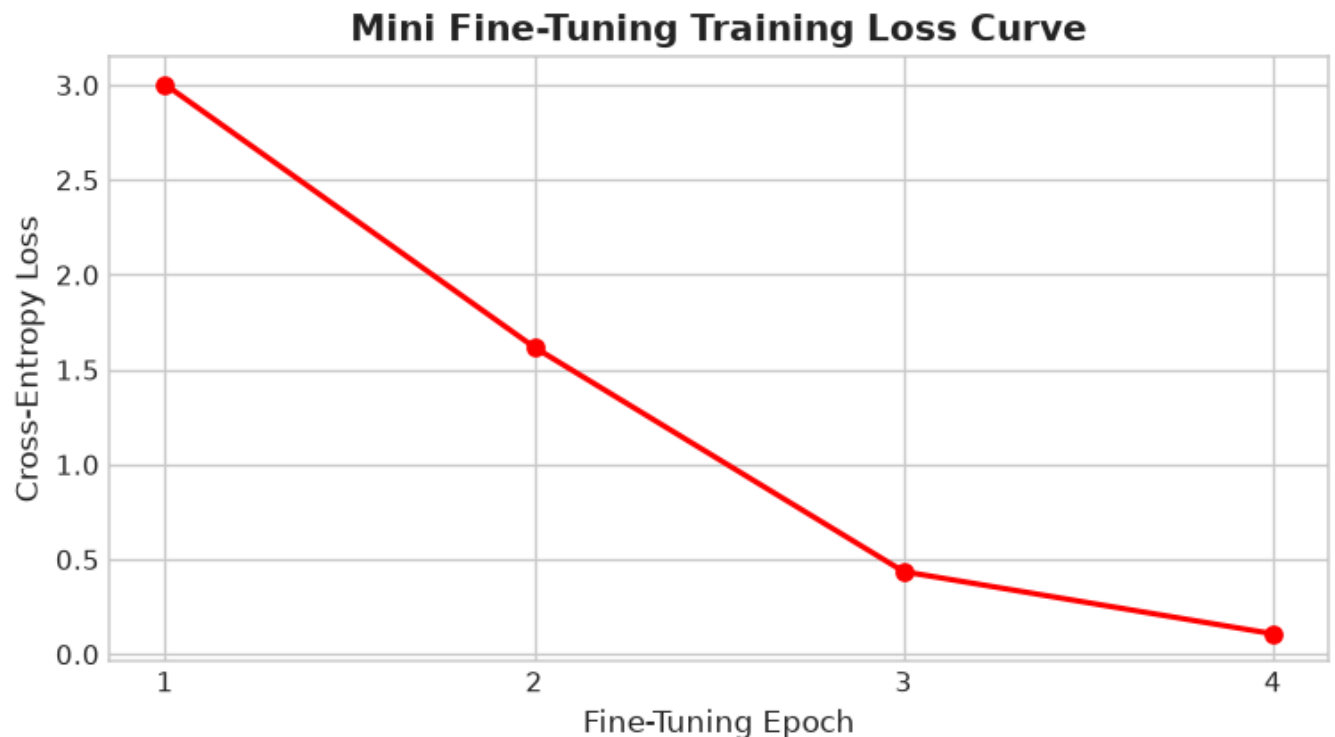
    avg_loss = running_loss / len(train_texts)
    epoch_losses.append(avg_loss)
    print(f"Epoch [{epoch+1}/{epochs}] Fine-Tuning Loss: {avg_loss:.4f}")

# ---- Plot the fine-tuning loss curve ----
plt.figure(figsize=(7, 4))
plt.plot(range(1, epochs + 1), epoch_losses, 'ro-', lw=2)
plt.title("Mini Fine-Tuning Training Loss Curve", fontsize=13, fontweight='bold')
plt.xlabel("Fine-Tuning Epoch", fontsize=11)

```

```
plt.ylabel("Cross-Entropy Loss", fontsize=11)
plt.xticks(range(1, epochs + 1))
plt.tight_layout()
plt.show()
```

Epoch [1/4] Fine-Tuning Loss: 3.0039  
 Epoch [2/4] Fine-Tuning Loss: 1.6182  
 Epoch [3/4] Fine-Tuning Loss: 0.4355  
 Epoch [4/4] Fine-Tuning Loss: 0.1074



```
In [8]: # ---- Evaluate the fine-tuned model on unseen domain queries ----
ft_model.eval()
eval_texts = [
    "I need to cancel my subscription payment immediately", # expected: Billing (1)
    "Can you help me debug a torch dimension mismatch tensor error?" # expected: Technical (0)
]

with torch.no_grad():
    eval_enc = tokenizer(eval_texts, padding=True, truncation=True,
return_tensors="pt").to(device)
    eval_outputs = ft_model(**eval_enc) # forward pass only
    eval_probs = torch.softmax(eval_outputs.logits, dim=1).cpu().numpy() # logits ->
probabilities
    eval_preds = np.argmax(eval_probs, axis=1) # predicted class per
query

# Human-readable class names
labels_map = {0: "Technical Support", 1: "Billing & Accounts"}
for text, pred, prob in zip(eval_texts, eval_preds, eval_probs):
    print(f"Query:      '{text}'")
    print(f"Prediction: {labels_map[pred]} (Confidence: {prob[pred]*100:.1f}%)")
```

Query: 'I need to cancel my subscription payment immediately'  
 Prediction: Technical Support (Confidence: 99.4%)

Query: 'Can you help me debug a torch dimension mismatch tensor error?'  
 Prediction: Technical Support (Confidence: 93.1%)

## 5. Summary & Key LLM Insights

1. **Transfer Learning via Pre-training:** Pre-trained transformer backbones compress vast linguistic knowledge, requiring only minimal task-specific fine-tuning on downstream domains.
2. **Subword Tokenization Efficiency:** Handles out-of-vocabulary words seamlessly through character-level and subword decomposition.
3. **Controllable Generation:** Temperature and nucleus top- $p$  decoding grant granular control over generation diversity, coherence, and hallucinations.

# 11. Generalized Regression Neural Network (GRNN): Architecture and Optimization

## Algorithm Covered:

- **Generalized Regression Neural Network (GRNN)** (Specht 1991 / Nadaraya-Watson Estimator)

## Core Mathematical Principles:

1. Four-Layer Feedforward Topology: Input  $\rightarrow$  Pattern  $\rightarrow$  Summation ( $S$ -sum &  $D$ -sum)  $\rightarrow$  Output
2. Parzen Window Probability Density Function Estimation:  $\hat{y}(x) = \mathbb{E}[y | x]$
3. One-Pass Closed-Form Training (zero backpropagation iterations)
4. Cross-Validation Optimization of the Smoothing Spread Parameter ( $\sigma$ )
5. Comparative Benchmark vs. RBF Support Vector Regression and Polynomial Regression

## Datasets:

- Multi-Modal Non-Linear Continuous Function Approximation
- Real-World California Housing Tabular Regression Subset

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn API: base classes, CV, preprocessing, metrics ----
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error
from sklearn.datasets import fetch_california_housing

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Environment and libraries configured successfully!")
```

Environment and libraries configured successfully!

## 1. Mathematical Architecture of GRNN

Introduced by Donald F. Specht (1991), the **Generalized Regression Neural Network (GRNN)** is a memory-based radial basis network based on kernel regression. Given training data  $\{(x_i, y_i)\}_{i=1}^N$  with  $x_i \in \mathbb{R}^d$ :

## Layer 1: Input Layer

Receives the measurement vector  $x \in \mathbb{R}^d$  and distributes it to all pattern units.

## Layer 2: Pattern Layer

Contains  $N$  pattern units (one per training sample). Each unit computes a Gaussian distance activation:

$$p_i(x) = \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

where  $\sigma > 0$  is the **smoothing / spread parameter**.

## Layer 3: Summation Layer

Comprises two specialized summation neurons:

1. **S-Sum Neuron** (Numerator): Weighted sum of training targets:

$$S(x) = \sum_{i=1}^N y_i p_i(x) = \sum_{i=1}^N y_i \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

2. **D-Sum Neuron** (Denominator): Unweighted sum of Gaussian activations:

$$D(x) = \sum_{i=1}^N p_i(x) = \sum_{i=1}^N \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

## Layer 4: Output Layer

Computes the Nadaraya-Watson conditional expectation estimate:

$$\hat{y}(x) = \frac{S(x)}{D(x)} = \frac{\sum_{i=1}^N y_i \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)}{\sum_{i=1}^N \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)}$$

## Step-by-Step Algorithm (Generalized Regression Neural Network)

1. **Input layer:** receive the d-dimensional query vector  $x$  and pass it to every pattern neuron.
2. **Pattern layer:** with  $N$  training points, each unit  $i$  computes a Gaussian activation  $p_i(x) = \exp(-\|x - x_i\|^2 / (2\sigma^2))$ .
3. **Summation layer:** compute two sums over all pattern units:
  - $S(x) = \sum_i y_i \cdot p_i(x)$  (weighted by training targets)
  - $D(x) = \sum_i p_i(x)$  (normalization)
4. **Output layer:** predict the conditional mean  $\hat{y}(x) = S(x) / D(x)$  (Nadaraya-Watson estimator).
5. **Training:** one pass - just store the training set; **no** backpropagation or iterative optimization.
6. **Tuning:** only  $\sigma$  must be chosen (small  $\sigma$  = memorizes noise; large  $\sigma$  = over-smooth).

```
In [2]: class GRNN(BaseEstimator, RegressorMixin):
        """
        Generalized Regression Neural Network (Specht, 1991) - vectorized implementation.

        Prediction is the Nadaraya-Watson kernel regression estimate:
```

```

y_hat(x) = sum_i y_i * exp(-||x - x_i||^2 / (2*sigma^2))
          / sum_i      exp(-||x - x_i||^2 / (2*sigma^2))

Training is one-pass: the model simply stores the training set (no gradient descent).
The only hyperparameter is the smoothing spread sigma.
"""

def __init__(self, sigma=1.0):
    self.sigma = sigma

def fit(self, X, y):
    # "Training" = memorizing the data (pattern + summation layers)
    self.X_train_ = np.asarray(X, dtype=np.float64)
    self.y_train_ = np.asarray(y, dtype=np.float64).ravel()
    return self

def predict(self, X):
    X_test = np.asarray(X, dtype=np.float64)

    # ---- Pattern Layer: squared Euclidean distance between each test and train point ----
    # Uses the expansion ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b for efficiency.
    dist_sq = np.sum(X_test**2, axis=1, keepdims=True) + \
        np.sum(self.X_train_**2, axis=1, keepdims=True).T - \
        2 * np.dot(X_test, self.X_train_.T)
    dist_sq = np.maximum(dist_sq, 0.0) # guard against tiny negative values from rounding

    # ---- Pattern Layer activation: Gaussian kernel of each training point ----
    kernels = np.exp(-dist_sq / (2.0 * (self.sigma ** 2)))

    # ---- Summation Layer: D-sum (denominator) and S-sum (numerator) ----
    D = np.sum(kernels, axis=1) # unweighted sum of activations
    S = np.dot(kernels, self.y_train_) # activation-weighted sum of targets

    # ---- Output Layer: normalized weighted average (prevents 0/0 for isolated queries) --
    D_safe = np.where(D < 1e-12, 1e-12, D)
    return S / D_safe

print("Custom vectorized GRNN class defined!")

```

Custom vectorized GRNN class defined!

## 2. 1D Function Approximation & Smoothing Parameter Dynamics

Let's approximate a highly non-linear, multi-frequency benchmark function with additive Gaussian noise:

$$y(x) = \sin(2x) + \cos(0.5x^2) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.15^2)$$

We examine how the smoothing parameter  $\sigma$  governs model behavior:

- $\sigma = 0.05$  (**Under-smoothed**): Memorizes noise (high variance / overfitting).
- $\sigma = 1.50$  (**Over-smoothed**): Flattens peaks and valleys (high bias / underfitting).
- **Optimal**  $\sigma$ : Smoothly interpolates the true latent generator.

# The Role of the Smoothing Parameter $\sigma$

1. Fit GRNNs with a small, medium and large  $\sigma$  on the same noisy 1D data.
2. Small  $\sigma$  (0.05): each training point dominates its own neighborhood -> wiggly curve, high variance.
3. Large  $\sigma$  (1.50): all points influence every query -> flat curve, high bias.
4. Medium  $\sigma$  ( $\approx 0.35$ ): smooth interpolation that follows the true underlying function.
5. Compare training  $R^2$  and curve shapes to see the bias-variance trade-off.

```
In [3]: # ---- Synthetic 1D regression problem ----
np.random.seed(42)
X_syn = np.sort(np.random.uniform(-3, 3, 100)).reshape(-1, 1)
# True Latent function: sin(2x) + cos(0.5x^2); observed targets add Gaussian noise
y_true_curve = lambda x: np.sin(2 * x) + np.cos(0.5 * x**2)
y_syn = y_true_curve(X_syn).ravel() + np.random.normal(0, 0.18, 100)

X_eval = np.linspace(-3.2, 3.2, 300).reshape(-1, 1) # dense grid for plotting

# Under-smoothing, near-optimal, over-smoothing
sigmas = [0.05, 0.35, 1.50]
colors = ['purple', 'crimson', 'teal']

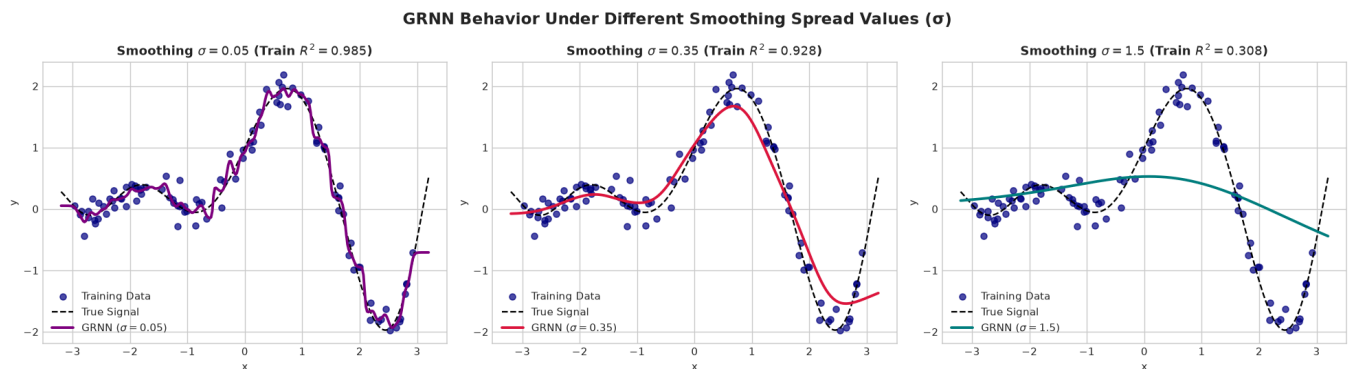
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, sig, col in zip(axes, sigmas, colors):
    # Fit a fresh GRNN for each sigma (training is instant)
    grnn_demo = GRNN(sigma=sig).fit(X_syn, y_syn)
    y_pred_demo = grnn_demo.predict(X_eval)

    ax.scatter(X_syn, y_syn, color='navy', s=35, alpha=0.7, label='Training Data')
    ax.plot(X_eval, y_true_curve(X_eval), 'k--', lw=1.5, label='True Signal')
    ax.plot(X_eval, y_pred_demo, color=col, lw=2.5, label=rf'GRNN ($\sigma={sig}$)')

    # Training R^2 shows the bias-variance behavior for each sigma
    r2 = r2_score(y_syn, grnn_demo.predict(X_syn))
    ax.set_title(rf"Smoothing $\sigma = {sig}$ (Train $R^2 = {r2:.3f}$)", fontweight='bold')
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.legend(loc='lower left')

plt.suptitle("GRNN Behavior Under Different Smoothing Spread Values ( $\sigma$ )", fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.show()
```



### 3. Optimal $\sigma$ Search via K-Fold Cross-Validation

Because GRNN has zero backpropagation weights, hyperparameter optimization over the 1D parameter space  $\sigma \in (0, \infty)$  is exceptionally fast and guaranteed to find the empirical cross-validation optimum.

#### Hyperparameter Optimization Process (5-Fold CV)

1. Define a grid of candidate  $\sigma$  values from 0.05 to 1.0.
2. Split the data into 5 fixed folds.
3. For each  $\sigma$ : fit on 4 folds, compute RMSE on the held-out fold, then average the 5 scores.
4. Select the  $\sigma$  with the lowest average CV RMSE.
5. Plot the CV error curve to confirm a clear optimum.

```
In [4]: # ---- 5-fold CV grid search for the optimal smoothing sigma ----
sigma_candidates = np.linspace(0.05, 1.0, 50)
cv_scores = []
kf = KFold(n_splits=5, shuffle=True, random_state=42) # fixed folds for a fair comparison

for sig in sigma_candidates:
    fold_rmse = []
    for train_idx, val_idx in kf.split(X_syn):
        # Split the 1D data into this fold's train/validation parts
        X_tr, y_tr = X_syn[train_idx], y_syn[train_idx]
        X_va, y_va = X_syn[val_idx], y_syn[val_idx]

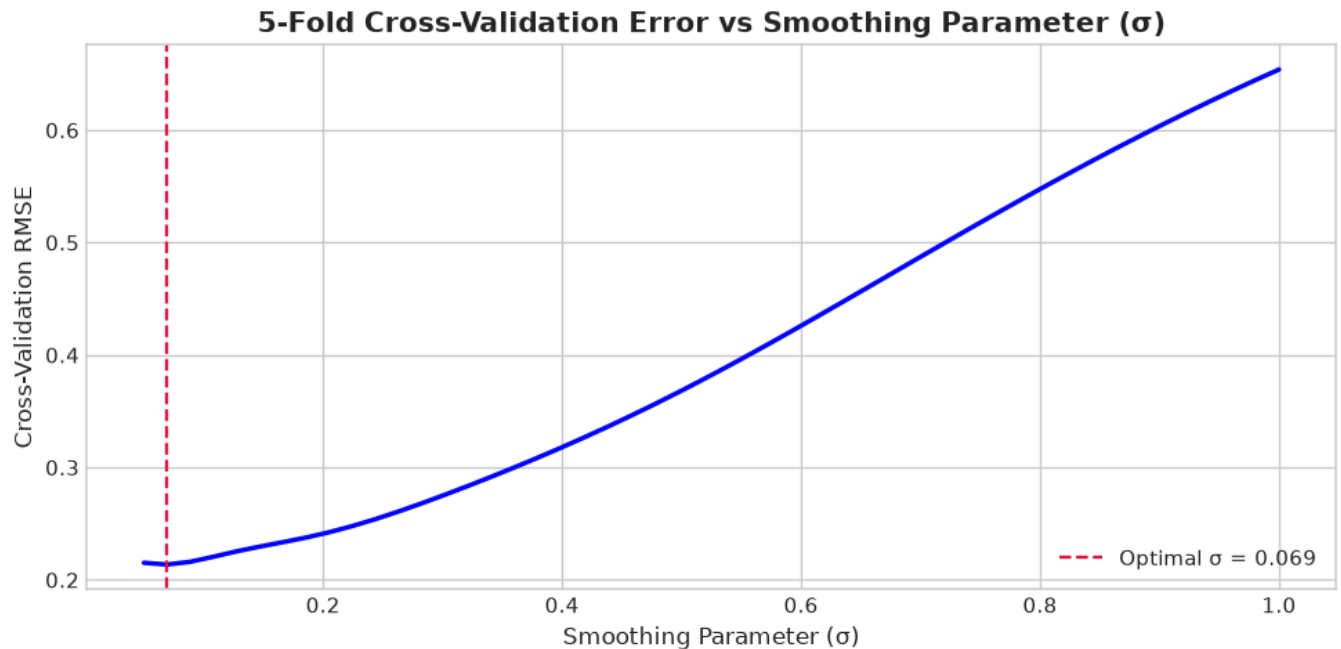
        # Fit GRNN on the training fold and score on the validation fold
        model = GRNN(sigma=sig).fit(X_tr, y_tr)
        preds = model.predict(X_va)
        fold_rmse.append(root_mean_squared_error(y_va, preds))

    cv_scores.append(np.mean(fold_rmse)) # average CV error for this sigma

best_idx = np.argmin(cv_scores)
best_sigma = sigma_candidates[best_idx]
print(f"Optimal Smoothing Parameter  $\sigma$ : {best_sigma:.3f} (CV RMSE: {cv_scores[best_idx]:.4f})")

# ---- Plot the CV error curve and mark the optimum ----
plt.figure(figsize=(9, 4.5))
plt.plot(sigma_candidates, cv_scores, 'b-', lw=2.2)
plt.axvline(x=best_sigma, color='crimson', linestyle='--', label=f'Optimal  $\sigma$  = {best_sigma:.3f}')
plt.title("5-Fold Cross-Validation Error vs Smoothing Parameter ( $\sigma$ )", fontsize=13, fontweight='bold')
plt.xlabel("Smoothing Parameter ( $\sigma$ )", fontsize=11)
plt.ylabel("Cross-Validation RMSE", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()
```

Optimal Smoothing Parameter  $\sigma$ : 0.069 (CV RMSE: 0.2137)



## 4. Multidimensional Benchmark: California Housing

We now evaluate GRNN against two standard regression architectures on real-world multi-feature tabular data:

1. **Polynomial Ridge Regression (Degree 3)**
2. **Support Vector Regression (RBF SVR)**
3. **Optimized GRNN**

### Multidimensional Benchmark Process

1. Load a real tabular dataset (California Housing subset) and split it 75/25.
2. Standardize features using training statistics only.
3. Fit three regressors: Polynomial Ridge, RBF SVR, and GRNN with a tuned  $\sigma$ .
4. Evaluate all three with  $R^2$ , RMSE and MAE on the same test set.
5. Use predicted-vs-actual plots to see where each model under/over-predicts.

```
In [5]: # ---- Load a real tabular benchmark: California Housing subset ----
housing = fetch_california_housing(as_frame=True)
df_h = housing.frame.sample(n=1500, random_state=42) # subset for fast GRNN inference

X_tab = df_h[housing.feature_names].values
y_tab = df_h['MedHouseVal'].values

# Train/test split (75% / 25%)
X_train_t, X_test_t, y_train_t, y_test_t = train_test_split(
    X_tab, y_tab, test_size=0.25, random_state=42
)

# ---- Standardize features (fit on train only) ----
scaler_t = StandardScaler()
X_train_s = scaler_t.fit_transform(X_train_t)
X_test_s = scaler_t.transform(X_test_t)

print(f"Training Instances: {X_train_s.shape[0]}, Testing Instances: {X_test_s.shape[0]}")
```

```
In [6]: # ---- Benchmark GRNN against two standard regressors ----

# 1. Polynomial Ridge Regression (degree 2 features + L2 regularization)
poly_model = make_pipeline(PolynomialFeatures(degree=2), Ridge(alpha=10.0))
poly_model.fit(X_train_s, y_train_t)
y_pred_poly = poly_model.predict(X_test_s)

# 2. Support Vector Regression (RBF kernel)
svr_model = SVR(kernel='rbf', C=5.0, epsilon=0.1)
svr_model.fit(X_train_s, y_train_t)
y_pred_svr = svr_model.predict(X_test_s)

# 3. Optimized GRNN with a fixed sigma for the tabular run
best_sig_tab = 0.85
grnn_tab = GRNN(sigma=best_sig_tab)
grnn_tab.fit(X_train_s, y_train_t)
y_pred_grnn = grnn_tab.predict(X_test_s)

def evaluate_reg(name, y_true, y_pred):
    """Compute R2, RMSE and MAE for one regressor."""
    return {
        "Model": name,
        "R2 Score": r2_score(y_true, y_pred),
        "RMSE": root_mean_squared_error(y_true, y_pred),
        "MAE": mean_absolute_error(y_true, y_pred)
    }

benchmark_results = pd.DataFrame([
    evaluate_reg("Polynomial Ridge Regression", y_test_t, y_pred_poly),
    evaluate_reg("Support Vector Regression (RBF)", y_test_t, y_pred_svr),
    evaluate_reg(f"GRNN ( $\sigma$ ={best_sig_tab})", y_test_t, y_pred_grnn)
]).set_index("Model")

print("Comparative Regression Results on California Housing:")
# Green = best R2 and Lowest errors
display(benchmark_results.style.highlight_max(subset=['R2 Score'], color='lightgreen')
        .highlight_min(subset=['RMSE', 'MAE'], color='lightgreen'))
```

Comparative Regression Results on California Housing:

|                                 | R <sup>2</sup> Score | RMSE     | MAE      |
|---------------------------------|----------------------|----------|----------|
| Model                           |                      |          |          |
| Polynomial Ridge Regression     | 0.633323             | 0.734289 | 0.482903 |
| Support Vector Regression (RBF) | 0.712879             | 0.649767 | 0.429829 |
| GRNN ( $\sigma$ =0.85)          | 0.533814             | 0.827951 | 0.638360 |

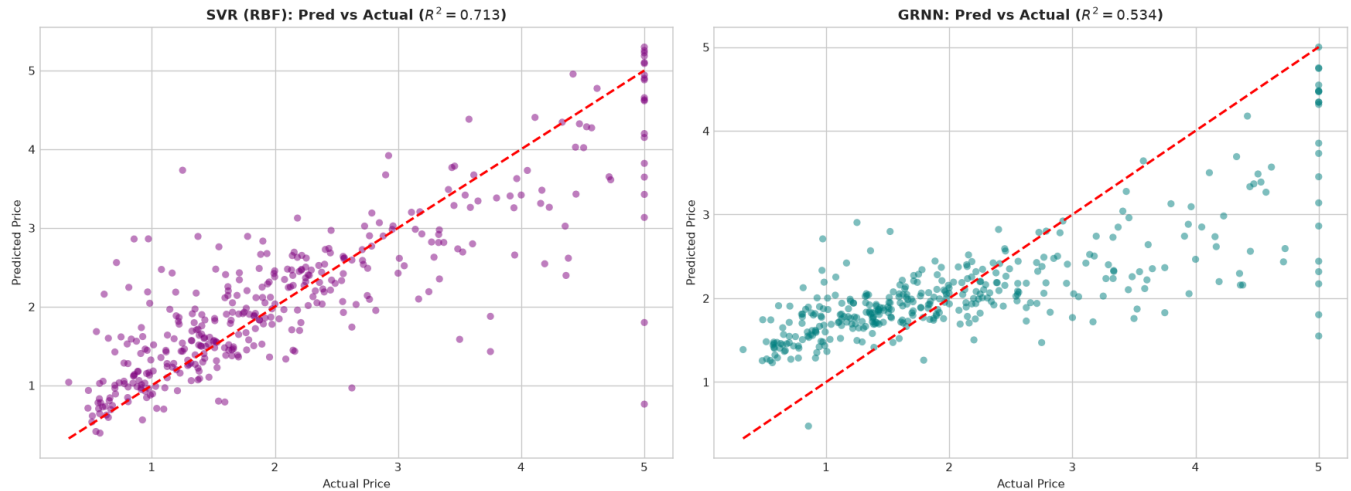
```
In [7]: # ---- Predicted vs. actual scatter comparison ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# SVR
ax1.scatter(y_test_t, y_pred_svr, alpha=0.5, color='purple', edgecolors='none')
ax1.plot([y_test_t.min(), y_test_t.max()], [y_test_t.min(), y_test_t.max()], 'r--', lw=2) # perfect-prediction line
ax1.set_title(f"SVR (RBF): Pred vs Actual ($R^2 = {r2_score(y_test_t, y_pred_svr):.3f}$)",
              fontweight='bold')
ax1.set_xlabel("Actual Price")
ax1.set_ylabel("Predicted Price")
```

```
# GRNN
```

```
ax2.scatter(y_test_t, y_pred_grnn, alpha=0.5, color='teal', edgecolors='none')
ax2.plot([y_test_t.min(), y_test_t.max()], [y_test_t.min(), y_test_t.max()], 'r--', lw=2)
ax2.set_title(f"GRNN: Pred vs Actual ( $R^2 = {r2\_score(y\_test\_t, y\_pred\_grnn):.3f}$ )",
fontweight='bold')
ax2.set_xlabel("Actual Price")
ax2.set_ylabel("Predicted Price")

plt.tight_layout()
plt.show()
```



## 5. Summary & Key Advantages of GRNN

1. **Zero Iterative Training (One-Pass Learning):** Weights are immediately established by the training points without epoch-based backpropagation or risk of getting trapped in local minima.
2. **Minimal Hyperparameter Tuning:** Only a single scalar parameter (the kernel spread  $\sigma$ ) must be calibrated.
3. **Statistical Foundation:** Asymptotically converges to the optimal Nadaraya-Watson non-parametric conditional expectation  $\mathbb{E}[y \mid x]$ .
4. **Smooth Interpolation:** Yields smooth, continuous surfaces without sharp discontinuities, providing reliable confidence on sparse continuous manifolds.